MODULE 1

Introduction to Data Warehouse

1.1 Data warehouse: Basic concepts

A data warehouse is a repository of information collected from multiple sources, stored under a unified schema, and usually residing at a single site.

"A data warehouse is a subject-oriented, integrated, time-variant, and non-volatile collection of data in support of management's decision-making process."

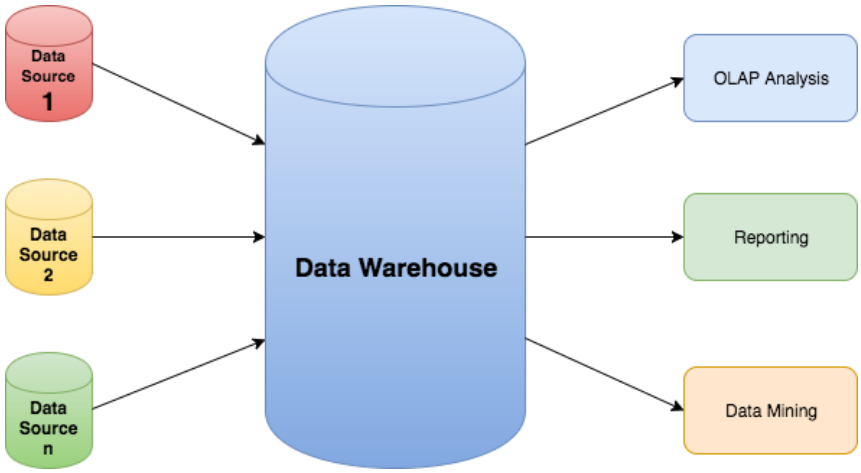
Following are the features of data warehouse:

Subject-Oriented: A data warehouse can be used to analyse a particular subject area. For example, "sales" can be a particular subject.

Integrated: A data warehouse integrates data from multiple data sources. For example, source A and source B may have different ways of identifying a product, but in a data warehouse, there will be only a single way of identifying a product.

Time-Variant: Historical data is kept in a data warehouse. For example, one can retrieve data from 3 months, 6 months, 12 months, or even older data from a data warehouse. This contrasts with a transactions system, where often only the most recent data is kept.

Non-volatile: Once data is in the data warehouse, it will not change. So, historical data in a data warehouse should never be altered.



1.2 Data warehousing

Data warehousing is the process of constructing and using data warehouses.

The construction of a data warehouse requires:

Data cleaning

Data integration

Data consolidation.

1.3 Differences between Operational Database Systems and Data Warehouses

- Online operational database systems are to perform online transaction and query processing. These systems are called **online transaction processing** (OLTP) systems.
- Data warehouse systems, serve users in the role of data analysis and decision making. Such systems can organize and present data in various formats. These systems are known as **online analytical processing** (OLAP) systems.

Comparison of OLTP and OLAP:

Features	**OLTP**	**OLAP**
Characteristic	operational processing	informational processing
Orientation	Transaction	Analysis
User	Clients, clerks, IT professionals	knowledge worker (e.g., manager executive, analyst)
DB design	entity-relationship (ER) model	star/snowflake
Data	Current data	historic
Summarization	primitive, highly detailed	summarized, consolidated
Unit of work	short, simple transaction	complex query
Access	read/write	mostly read
Focus	data in	information out

Number of records Accessed	Tens	Millions
Number of users	Thousands	Hundreds
DB size	GB to high-order GB	>=TB

1.4 Characteristics of OLAP

Users and system orientation:

An OLTP system is customer-oriented

An OLAP system is market-oriented

Data contents

An OLTP system manages current data

An OLAP system manages large amounts of historic data

Database design

An OLTP system adopts an entity-relationship (ER) data model and an application-oriented database design

An OLAP system typically adopts either a star or a snowflake model and a subject-oriented database design.

View

An OLTP system focuses mainly on the current data within an enterprise or department, without referring to historic data

OLAP systems also deal with information that originates from different organizations, integrating information from many data stores.

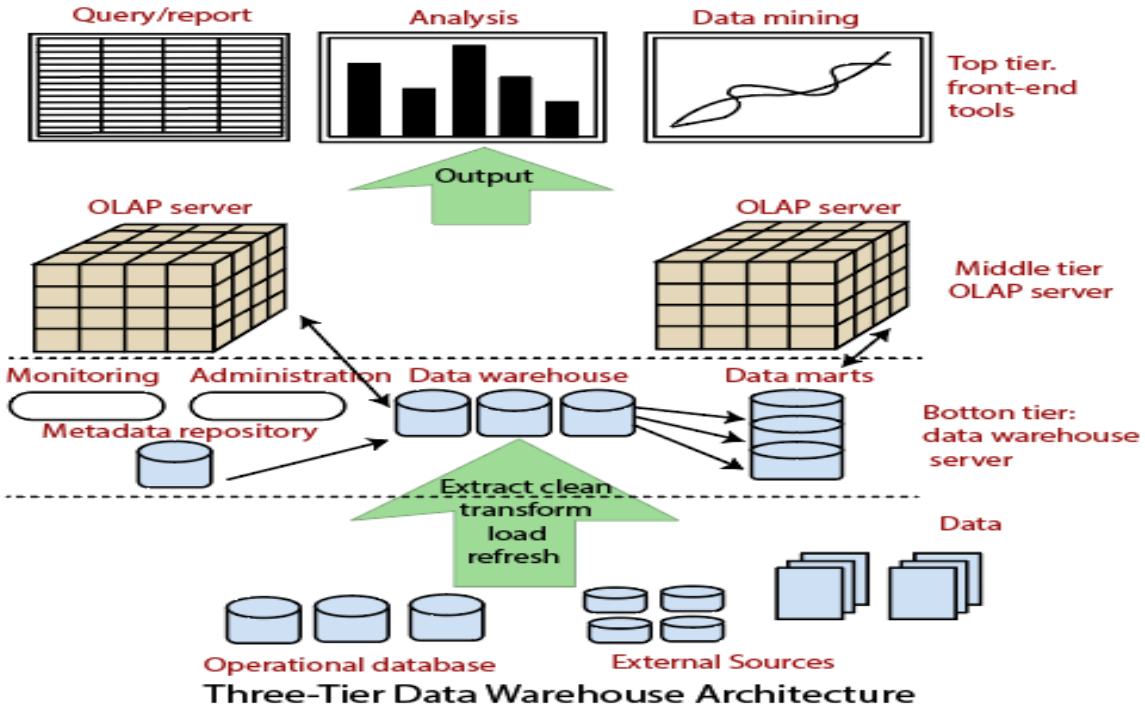
Access patterns

Access patterns of an OLTP system consist mainly of short, atomic transactions

Accesses to OLAP systems are mostly read-only operations

1.5 Data Warehousing: A Multitier Architecture

Data warehouses often adopt a three-tier architecture, as presented in Figure below



Three-Tier Data Warehouse Architecture

1. The bottom tier is a warehouse database server that is almost always a relational database system. Back-end tools and utilities are used to feed data into the bottom tier from operational databases or other external sources (e.g., customer profile information provided by external consultants).

2. The middle tier is an OLAP server that is typically implemented using either

(1) A relational OLAP(ROLAP) model (i.e., an extended relational DBMS that maps operations on multidimensional data to standard relational operations); or

(2) A multidimensional OLAP (MOLAP) model (i.e., a special purpose server that directly implements multidimensional data and operations).

3. The top tier is a front-end client layer, which contains query and reporting tools, analysis tools, and/or data mining tools (e.g., trend analysis, prediction, and so on).

Extraction, Transformation, and Loading

Data warehouse systems use back-end tools and utilities to populate and refresh their data.

These tools and utilities include the following functions:

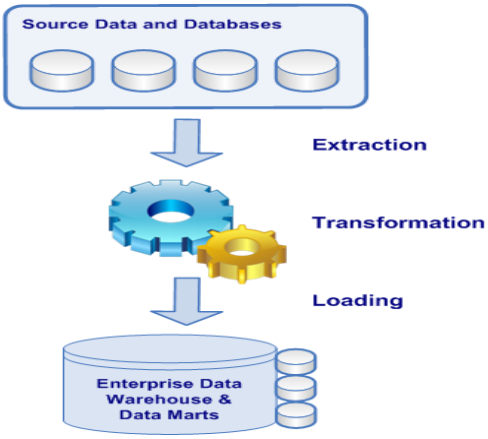
Data extraction

Data cleaning

Data transformation

Load

Refresh



Data extraction: Gathers data from multiple, heterogeneous, and external sources.

Data cleaning: Detects errors in the data and rectifies them when possible.

Data transformation: Converts data from legacy or host format to warehouse format.

Load: Sorts, summarizes, consolidates, computes, checks integrity, and imports the data into data warehouse

Refresh: Propagates the updates from the data sources to the warehouse.

1.6 Data Warehouse Models

From the architecture point of view, there are three data warehouse models

Enterprise Warehouse

Data Mart

Virtual Warehouse

→ Enterprise warehouse

An enterprise warehouse collects all of the information about subjects spanning the entire organization.

It provides corporate-wide data integration, usually from one or more operational systems or external information providers, and is cross-functional in scope.

It typically contains detailed data as well as summarized data, and can range in size from a few gigabytes to hundreds of gigabytes, terabytes, or beyond.

An enterprise data warehouse may be implemented on traditional mainframes, computer super servers, or parallel architecture platforms.

It requires extensive business modelling and may take years to design and build.

→ Data mart

A data mart contains a subset of corporate-wide data that is of value to a specific group of users. The scope is confined to specific selected subjects.

For example, a marketing data mart may confine its subjects to customer, item, and sales.

The data contained in data marts tend to be summarized.

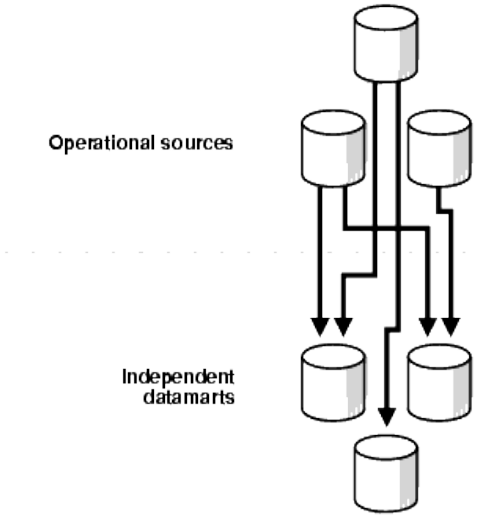
Data marts are usually implemented on low-cost departmental servers that are Unix/Linux or Windows based

Depending on the source of data, data marts can be categorized as

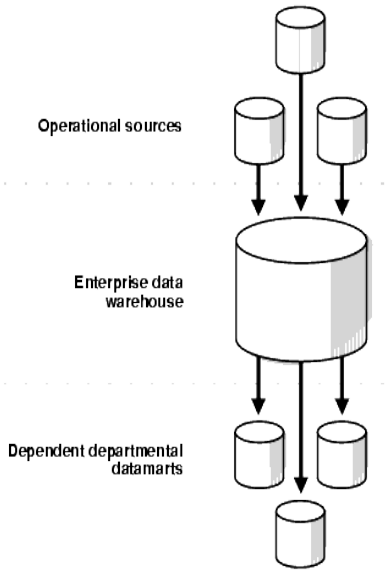
Independent data marts

Dependent data marts

Independent data marts are sourced from data captured from one or more operational systems or external information providers, or from data generated locally within a particular department or geographic area.



Dependent data marts are sourced directly from enterprise data warehouses



→ Virtual warehouse

A virtual warehouse is a set of views over operational databases.

For efficient query processing, only some of the possible summary views may be materialized.

A virtual warehouse is easy to build but requires excess capacity on operational database servers

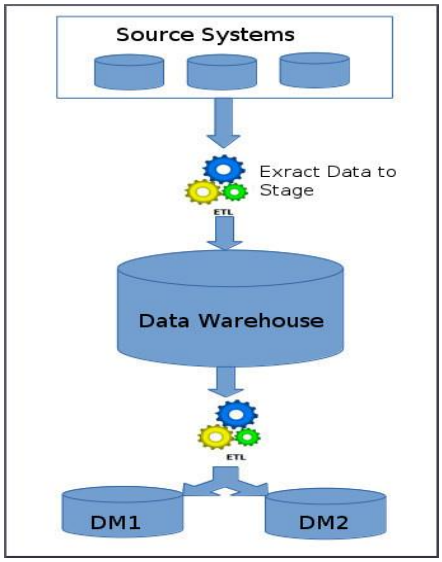
1.7 Data Warehouse Design Approaches

Top-down Data Warehouse Design Approach

In the top-down approach, the data warehouse is designed first and then data mart are built on top of data warehouse.

Steps that are involved in top-down approach:

- Data is extracted from the various source systems. The extracts are loaded and validated in the stage area. Validation is required to make sure the extracted data is accurate and correct. Use the ETL tools or approach to extract and push to the data warehouse.
- Data is extracted from the data warehouse. Various aggregation, summarization techniques are used to extract data and loaded back to the data warehouse.
- Once the aggregation and summarization is completed, various data marts extract that data and apply the some more transformation to make the data structure as defined by the data marts.



→It is Expensive

→It is inflexible to changing departmental changes

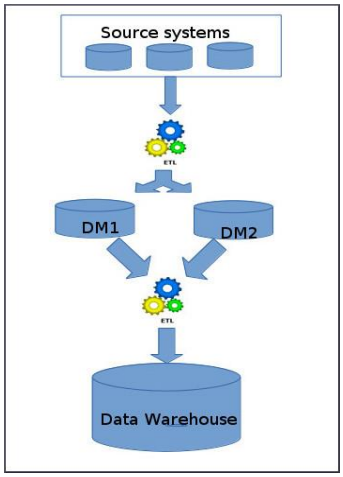
→Takes a long time to develop

Bottom-up Data Warehouse Design Approach

In this Approach, Data marts are directly loaded with the data from the source systems and then ETL process is used to load in to Data Warehouse.

Steps that are involved in bottom-up approach:

- The data flow in the bottom up approach starts from extraction of data from various source system into the stage area where it is processed and loaded into the data marts that are handling specific business process.
- After data marts are refreshed the current data is once again extracted in stage area and transformations are applied to create data into the data mart structure. The data is the extracted from Data Mart to the staging area is aggregated, summarized and so on loaded into EDW



→Provides flexibility

→low cost

→ Problem is when integrating various disparate data marts into a consistent enterprise data warehouse.

A recommended method for the development of data warehouse systems is to implement the warehouse in an **incremental and evolutionary** manner

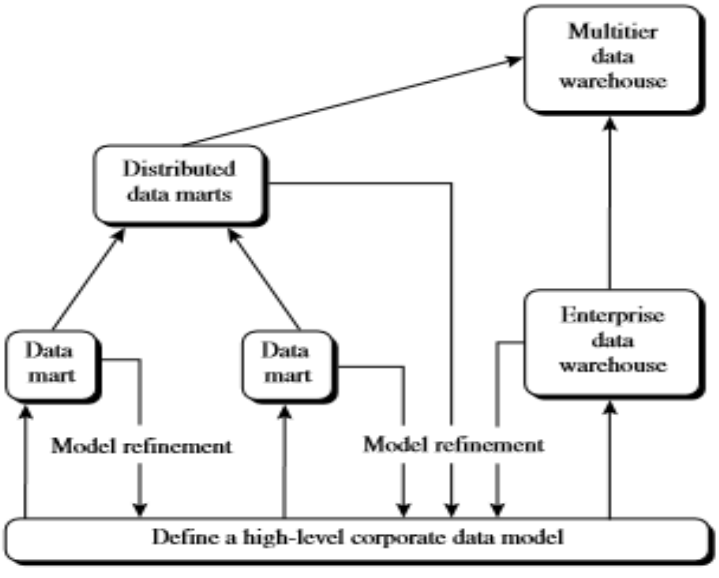


Figure 4.2 A recommended approach for data warehouse development.

First, a high-level corporate data model is defined within a reasonably short period (such as one or two months) that provides a corporate-wide, consistent, integrated view of data among different subjects and potential usages.

Second, independent data marts can be implemented in parallel with the enterprise warehouse based on the same corporate data model set noted before.

Third, distributed data marts can be constructed to integrate different data marts via hub servers.

Finally, a multitier data warehouse is constructed where the enterprise warehouse is the sole custodian of all warehouse data, which is then distributed to the various dependent data marts.

1.9 Data Cube and OLAP

A **data cube** allows data to be modelled and viewed in multiple dimensions.

It is defined by

dimensions

facts

Dimensions are the entities with respect to which an organization wants to keep records

For example:

AllElectronics may create a sales data warehouse in order to keep records of the store's sales with respect to the dimension's time, item, branch, and location.

Facts are numeric measures.

Examples of facts for a sales data warehouse include ***dollars sold*** (sales amount in dollars), ***units sold*** (number of units sold)

2-D Data cube

Table 1.1: 2-D View of Sales Data for *All Electronics* According to dimensions time and item, where sales are from branches located in the city Vancouver.

location = "Vancouver "				
	item (type)			
Time (quarter)	**Home entertainment**	**Computer**	**Phone**	**Security**
Q1	605	825	14	400
Q2	680	952	31	512
Q3	812	1023	30	501
Q4	927	1038	38	580

3-D Data cube (series of 2-D)

	location = "Chicago"				location = "New York"				location = "Toronto"				location = "Vancouver"			
	item				item				item				item			
	home				home				home				home			
time	ent.	comp.	phone	sec.	ent.	comp.	phone	sec.	ent.	comp.	phone	sec.	ent.	comp.	phone	sec.
Q1	854	882	89	623	1087	968	38	872	818	746	43	591	605	825	14	400
Q2	943	890	64	698	1130	1024	41	925	894	769	52	682	680	952	31	512
Q3	1032	924	59	789	1034	1048	45	1002	940	795	58	728	812	1023	30	501
Q4	1129	992	63	870	1142	1091	54	984	978	864	59	784	927	1038	38	580

Data for *All Electronics* According to *time*, *item*, and *location*

3-D Data cube

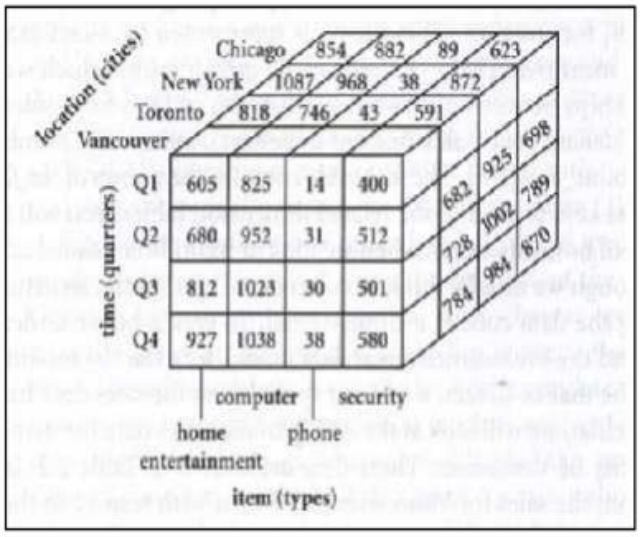


Figure 1.3: 3-D Data Cube

4-D Data Cube (series of 3-D)

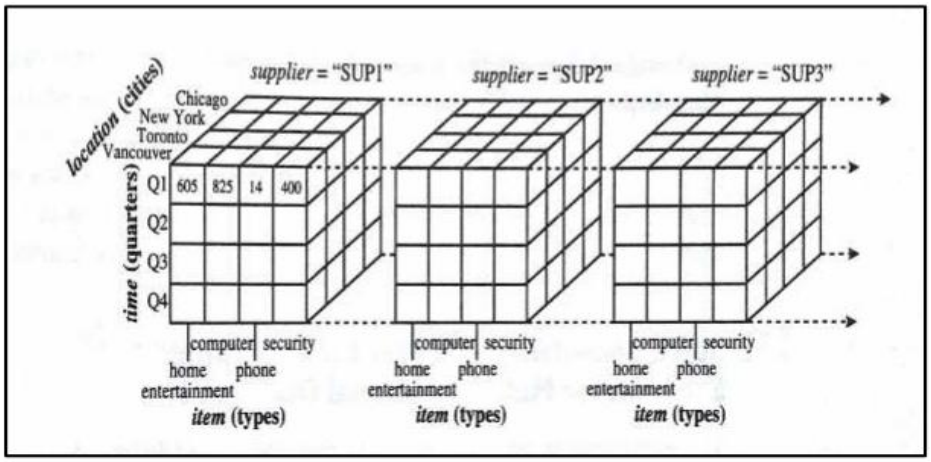


Figure 1.4: 4-D data cube representation of sales data, according to *time*, *item*, *location*, and *supplier*.

Multidimensional data model can be summarised as:

Given a set of dimensions, we can generate a **cuboid** for each of the possible subsets of the given dimensions. The result would form a **lattice of cuboids**

The 0-D cuboid, which holds the highest level of summarization, is called the apex cuboid

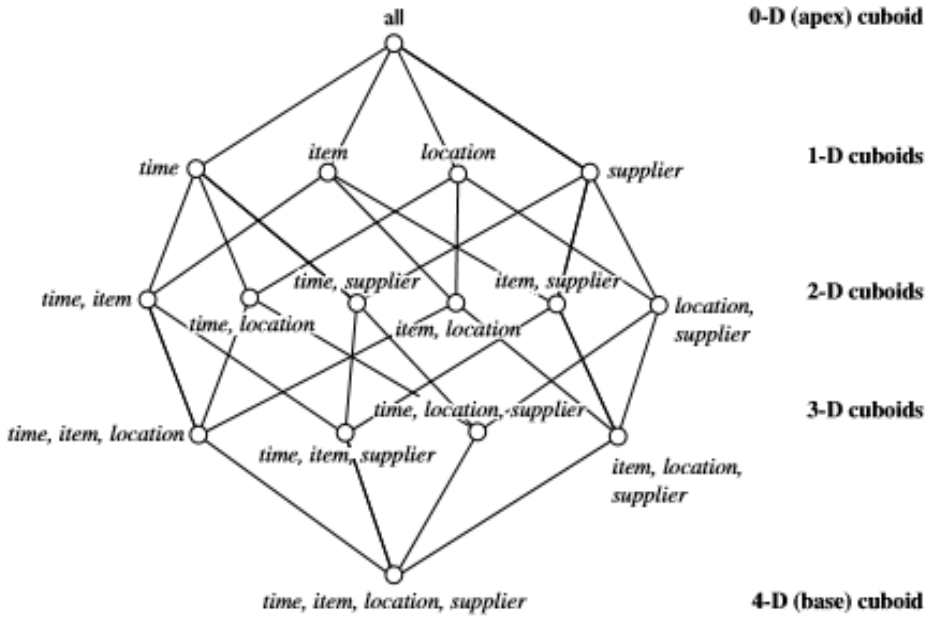


Figure 4.5 Lattice of cuboids, making up a 4-D data cube for *time*, *item*, *location*, and *supplier*. Each cuboid represents a different degree of summarization.

1.10 Schemas for Multidimensional Data Model

Data model for a data warehouse is a multidimensional model, which can exist in the form of:

Star schema

Snowflake schema

Fact constellation schema

Star schema: The most common modeling paradigm is the star schema, in which the data warehouse contains (1) a large central table (fact table) containing the bulk other data, with no redundancy, and (2) a set of smaller attendant tables (dimension tables), one for each dimension. The schema graph resembles a starburst, with the dimension tables displayed in a radial pattern around the central fact table.

Example:

1.10.1 Star schema.

A star schema for All Electronics sales is shown in Figure

Sales are considered along four dimensions: time, item, branch, and location. The schema contains a central fact table for sales that contains keys to each of the four dimensions, along with two measures: dollars sold and units sold. To minimize the size of the fact table, dimension identifiers (e.g., time key and item key) are system-generated identifiers.

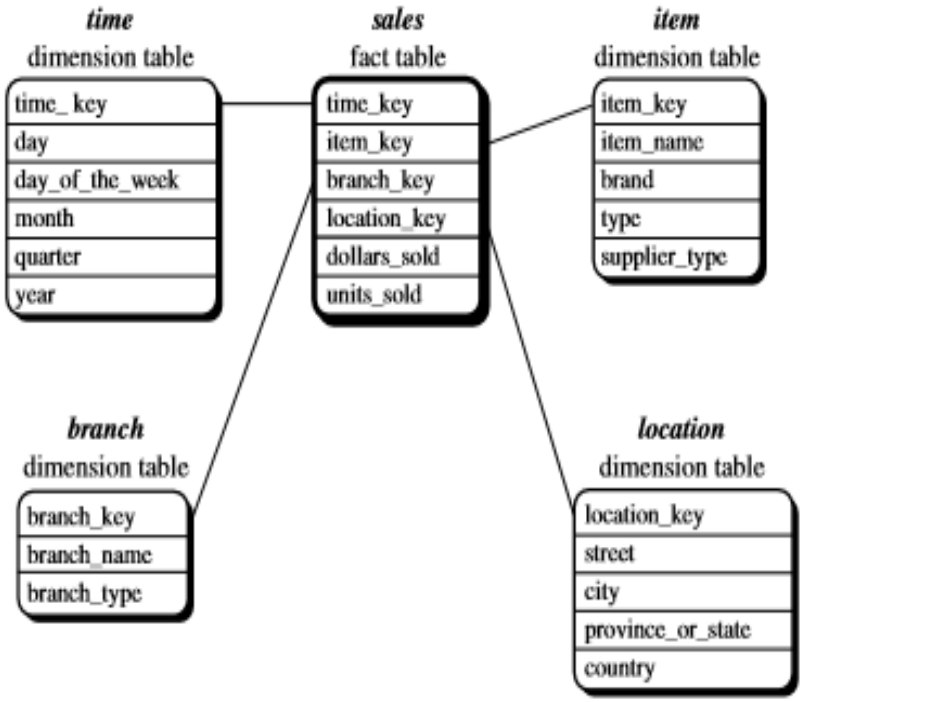


Figure 3.4 Star schema of a data warehouse for sales.

1.10.2 Snowflake schema:

The snowflake schema is a variant of the star schema model, where some dimension tables are normalized, thereby further splitting the data into additional tables.

The resulting schema graph forms a shape similar to a snowflake.

The major difference between the snowflake and star schema models is that the dimension tables of the snowflake model may be kept in normalized form to reduce redundancies.

Such a table is easy to maintain and saves storage space. However, this space savings is negligible in comparison to the typical magnitude of the fact table.

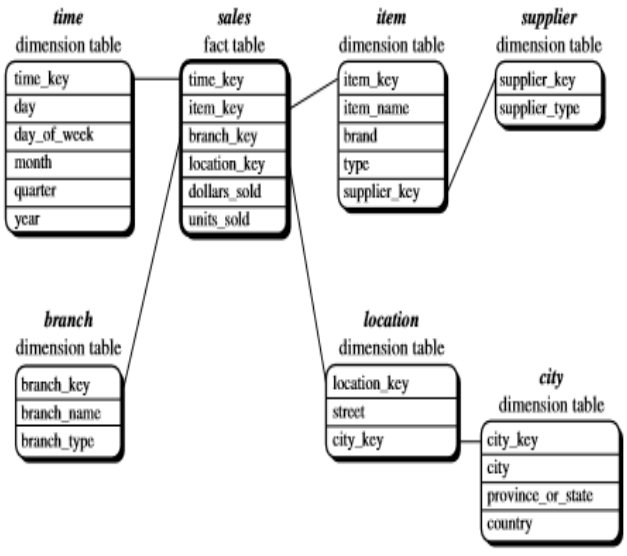


Figure 3.5 Snowflake schema of a data warehouse for sales.

The main difference between the two schemas is in the definition of dimension tables. The single dimension table for item in the star schema is normalized in the snowflake schema, resulting in new item and supplier tables. For example, the item dimension table now contains the attributes item key, item name, brand, type, and supplier key, where supplier key is linked to the supplier dimension table, containing supplier key and supplier type information. Similarly, the single dimension table for location in the star schema can be normalized into two new tables: location and city. The city key in the new location table links to the city dimension. Notice that, when desirable, further normalization can be performed on province or state and country in the snowflake schema.

1.10.3 Fact constellation:

Sophisticated applications may require multiple fact tables to share dimension tables. This kind of schema can be viewed as a collection of stars, and hence is called a galaxy schema or a fact constellation.

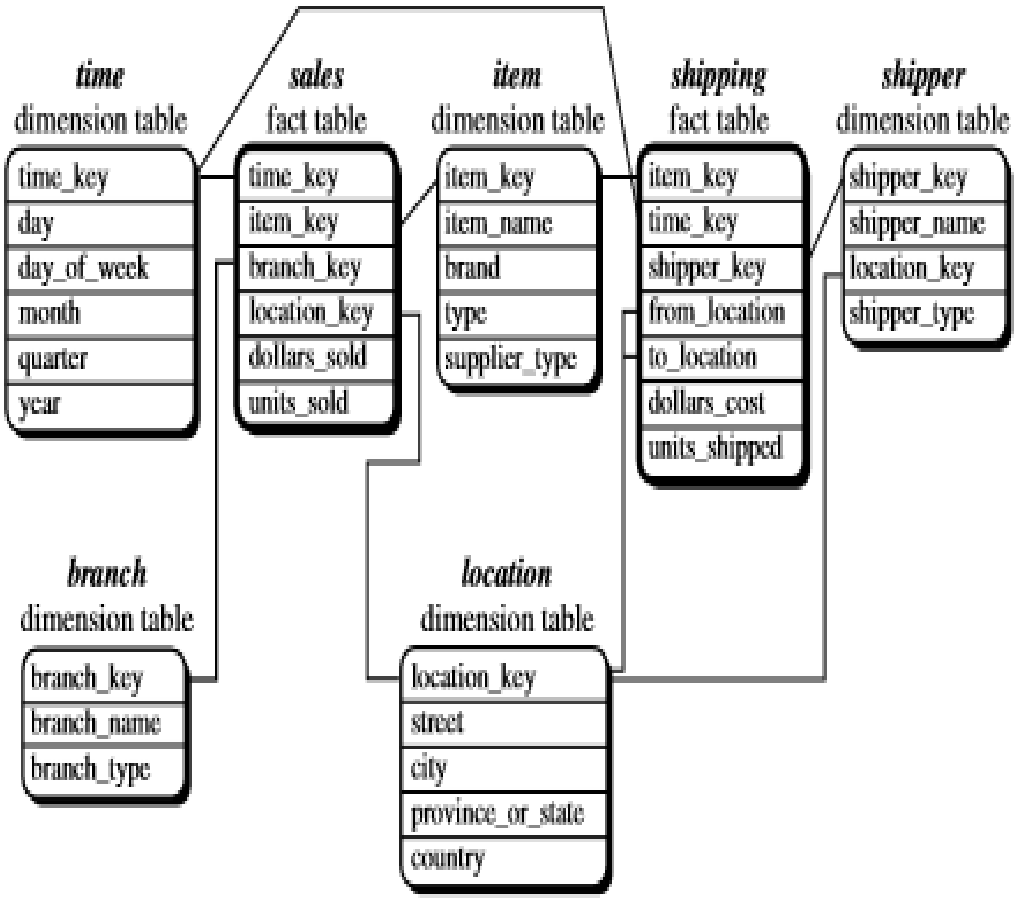


Figure 3.6 Fact constellation schema of a data warehouse for sales and shipping.

This schema specifies two fact tables, sales and shipping.

The sales table definition is identical to that of the star schema.

The shipping table has five dimensions, or keys—item key, time key, shipper key, from location, and to location— and two measures—dollars cost and units shipped.

A fact constellation schema allows dimension tables to be shared between fact tables.

For example, the dimension tables for time, item, and location are shared between the sales and shipping fact tables.

1.10.4 Difference between star and snowflakes

Comparison	Star schema	Snowflake schema
Structure of schema	Contains fact and dimension tables.	Contains sub-dimension tables including fact and dimension tables.
Use of normalization	Doesn't use normalization.	Uses normalization
Ease of use	Simple to understand and easily designed.	Hard to understand and design.
Space usage	More	Less
Foreign key used	Fewer	Large in number

1.11 Role of concept hierarchy

A **concept hierarchy** defines a sequence of mappings from a set of low-level concepts to higher-level, more general concepts.

Consider a concept hierarchy for the dimension location.

City values for location include Vancouver, Toronto, New York, and Chicago.

Each city can be mapped to the province or state to which it belongs. For example, Vancouver can be mapped to British Columbia, and Chicago to Illinois.

The states can in turn be mapped to the country to which they belong, such as Canada or the USA.

These mapping forms a concept hierarchy for the dimension location, mapping set of low level concepts(cities) to more general concepts(countries).

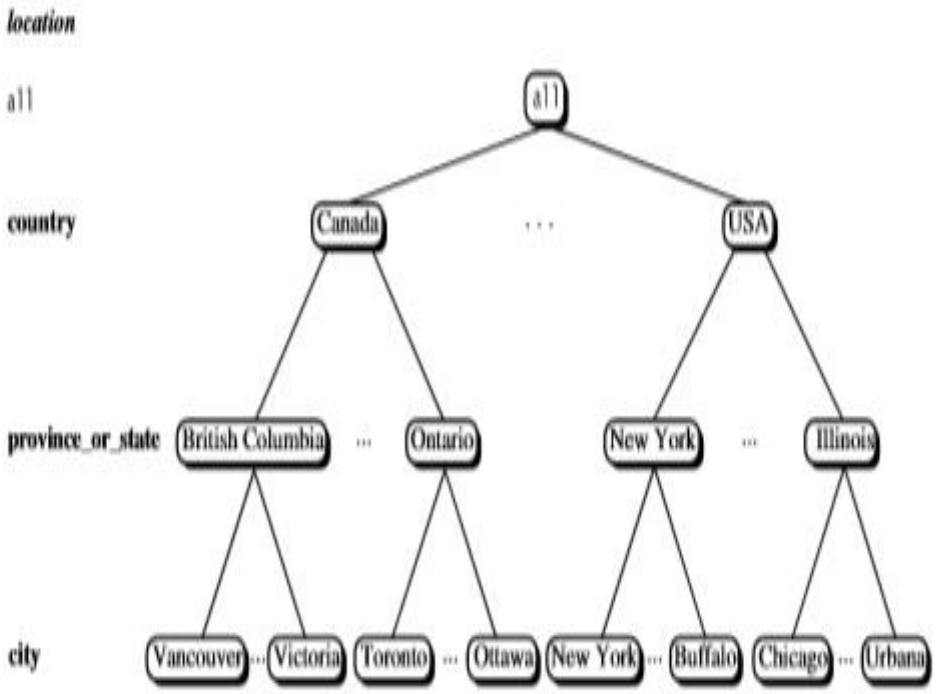


Figure 3.7 A concept hierarchy for the dimension *location*. Due to space limitations, not all of the nodes of the hierarchy are shown (as indicated by the use of "ellipsis" between nodes).

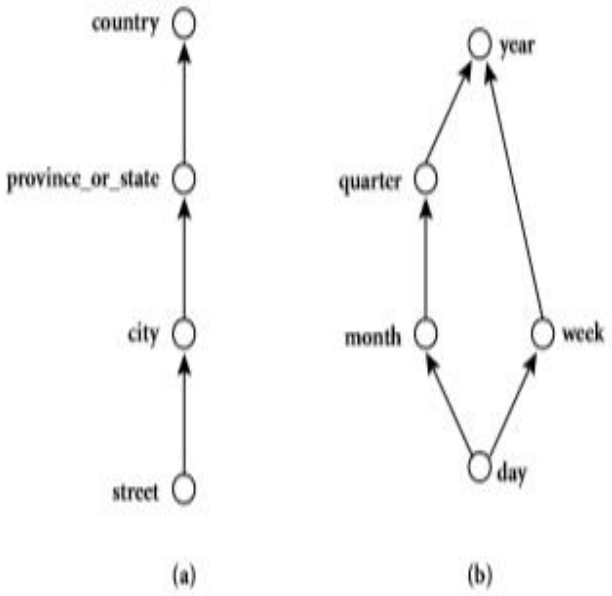


Figure 3.8 Hierarchical and lattice structures of attributes in warehouse dimensions: (a) a hierarchy for *location*; (b) a lattice for *time*.

In the above figure, "street<city<province_or_state<country".

Partial order for the time dimension based on the attributes day, week, month, quarter, and year is "day< {month<quarter; week}< year"

Concept hierarchy can also be defined in a **set-grouping hierarchy.**

Set-grouping hierarchy is shown below for the dimension price, where an interval ($X…$Y] denotes the range from $X(exclusive) to $Y(inclusive).

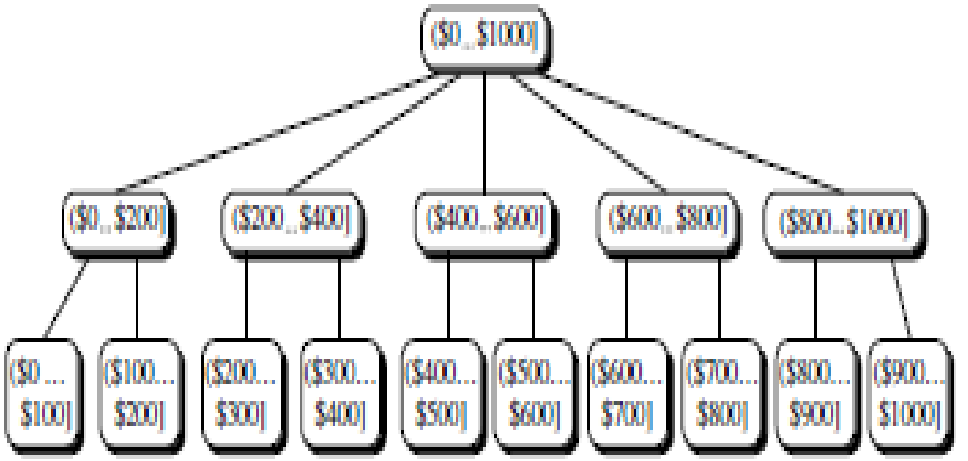


Figure 4.11 A concept hierarchy for *price*.

1.12 Measures: Their Categorization and computation

Measures can be organized into 3 categories based on the Aggregate functions used:

Distributive

Algebraic

Holistic

Distributive Aggregate function

An aggregate function is distributed, if it can be computed in a distributed manner

Suppose the data is partitioned into n sets.

We apply the function to each partition, resulting in n Aggregate values

If the result derived by applying the function to the n aggregate values is the same as that derived by applying the function to entire data set in distributive manner.

For Example, sum() can be computed for a data cube by first partitioning the cube into set of subcubes, computing sum() for each subcube, and then summing up the counts obtained for each subcube. Hence sum() is aggregate function.

Some good examples of distributive aggregate functions are:

count()

min()

max()

sum()

Algebraic Aggregate function

An aggregate function is algebraic, if it can be computed by an algebraic function M arguments(where M is bounded positive integer), each of which is obtained by applying a distributive aggregate function.

A measure is algebraic if it is obtained by applying an algebraic aggregate function

Example:

avg() can be computed by sum()/count()

where both sum() and count() are distributive aggregate function

Holistic Aggregate function

An aggregate function is holistic if there is no constant bound on storage size to describe a sub aggregate.

There does not exist any algebraic function with M arguments(where M is a constant) that characterizes the computation.

Example:

median()

mode()

rank()

1.13 OLAP operation

- Roll-up
- Drill-down
- Slice and dice
- Pivot (rotate)

Roll-up:

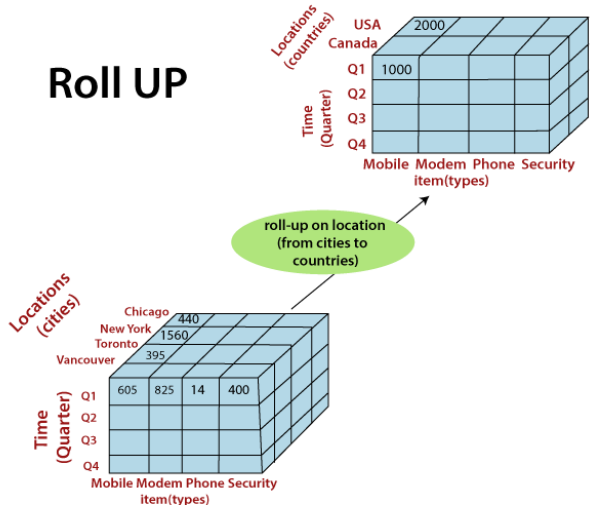
→Also called drill-up operation

→Performs aggregation on a data cube, either by ***climbing up a concept hierarchy or by dimension reduction***

→It navigates from more detailed data to less detailed data.

→Initially the concept hierarchy was "**street < city < state< country**".

→ On rolling up, the data is aggregated by ascending the location hierarchy from the level of city to the level of country.



Drill-down:

→Drill-down is the reverse of roll-up.

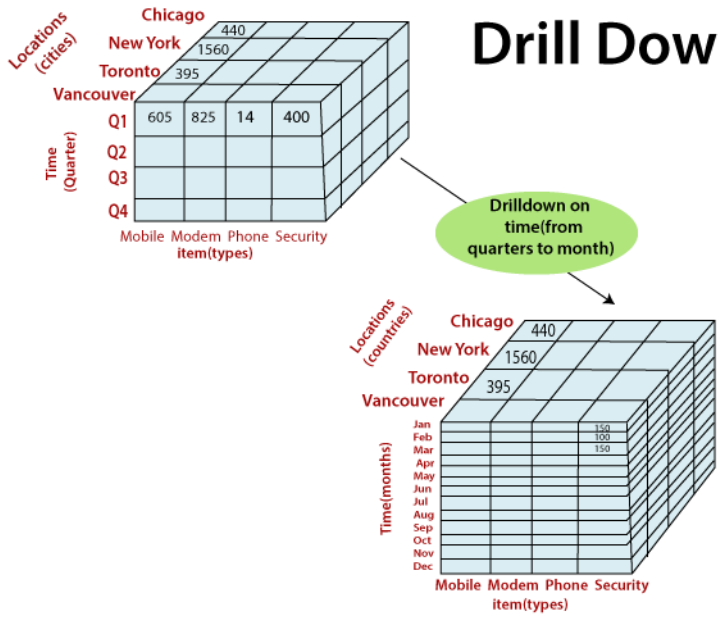
→It navigates from less detailed data to more detailed data.

→Drill-down can be realized by either ***stepping down a concept hierarchy***

→ Initially the concept hierarchy was "day < month < quarter < year."

→ On drilling down, the time dimension is descended from the level of quarter to the level of month.

→When drill-down is performed, one or more dimensions from the data cube are added.

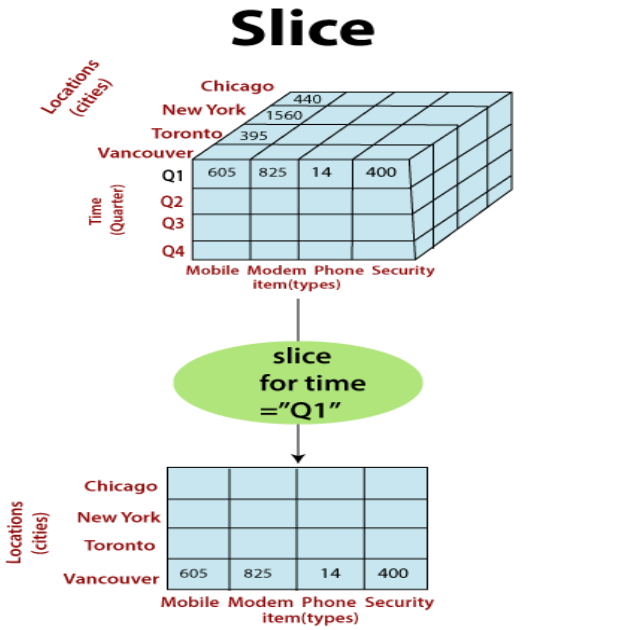


Slice and dice:

The slice operation selects one particular dimension from a given cube and provides a new sub-cube. Consider the following diagram that shows how slice works.

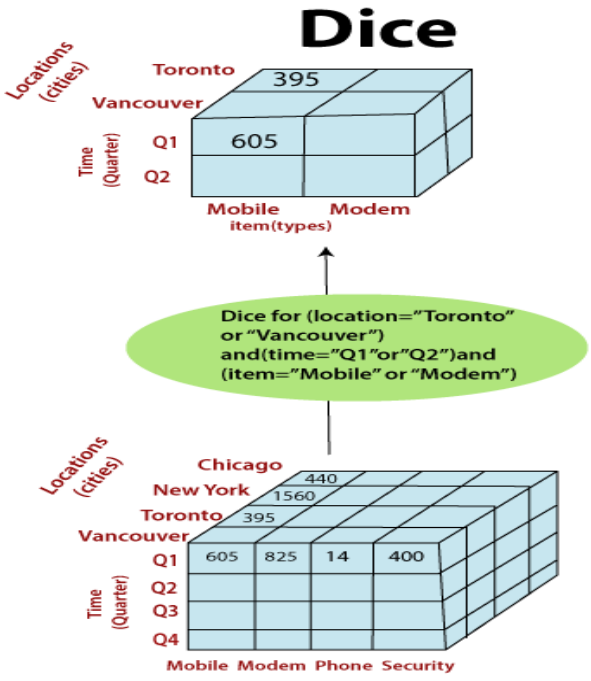
Here Slice is performed for the dimension "time" using the criterion time = "Q1".

It will form a new sub-cube by selecting one or more dimensions.



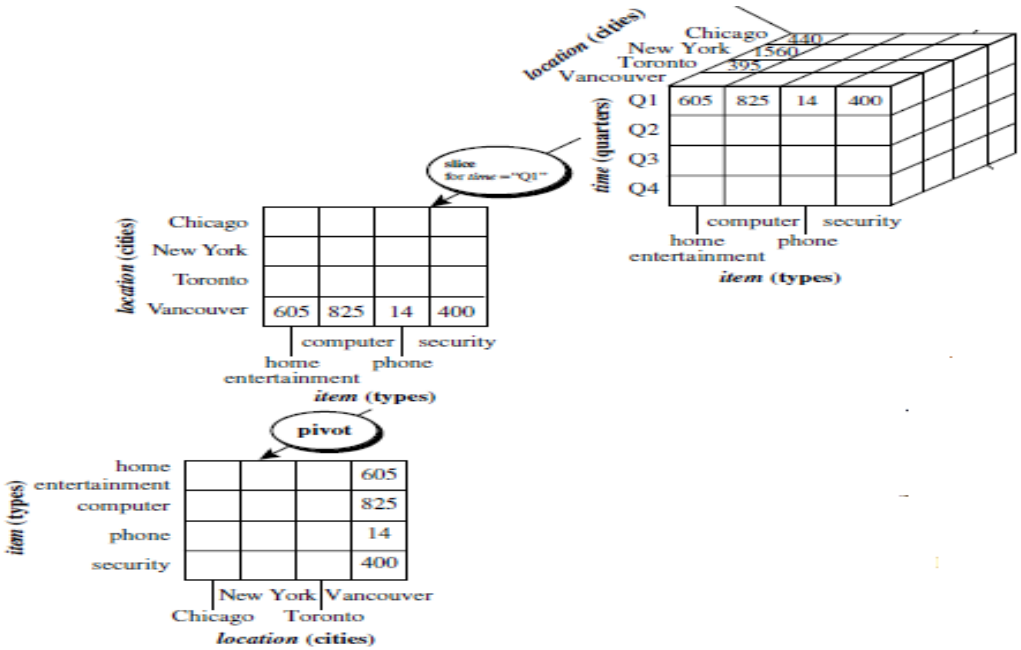
Dice selects two or more dimensions from a given cube and provides a new sub-cube.

Consider the following diagram that shows the dice operation. The dice operation on the cube based on the following selection criteria involves three dimension(location = "Toronto" or "Vancouver") (time = "Q1" or "Q2") (item =" Mobile" or "Modem")



Pivot (rotate):

- Visualization operation that rotates the data axes in view to provide an alternative data presentation


location (cities)
Chicago
New York
Toronto
Vancouver
440
1560
395
time (quarters)
Q1
Q2
Q3
Q4
605
825
14
400
computer
security
home entertainment
phone
item (types)
slice
for time = "Q1"
location (cities)
Chicago
New York
Toronto
Vancouver
605
825
14
400
computer
security
home entertainment
phone
item (types)
pivot
item (types)
home entertainment
computer
phone
security
605
825
14
400
New York
Vancouver
Chicago
Toronto
location (cities)

MODULE 2
Data warehouse Implementation

1.1 Efficient Data cube computation

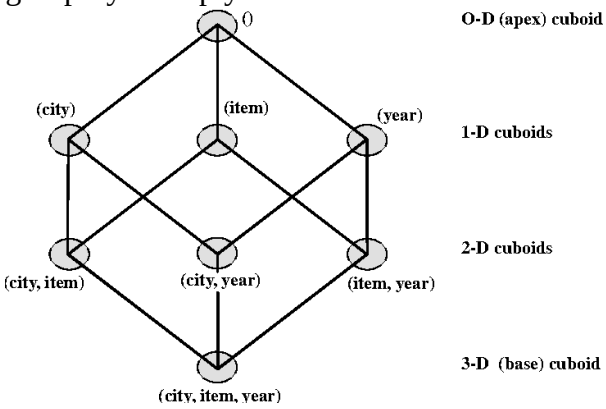
Data warehouse contains huge volume of data. OLAP servers demand that decision support queries be answered in the order of seconds.
Therefore, it is crucial for data warehouse system to support highly efficient data cube computation techniques, access methods and query processing techniques.
Multidimensional data analysis is the efficient computation of aggregation across many sets of dimensions
In SQL, aggregation is referred to as group-by's where Each group-by is represented by a cuboid. Set of group-by forms a lattice of cuboid

Compute cube operator

→ **Compute cube operator:** Computes aggregates over all the subsets of dimension
→This requires excessive storage space, especially for large no of dimensions

Example:
Suppose that you want to create a data cube for AllElectronics sales that contains:
city, item, year and sales_in_dollar.
You want to be able to analyse the data with the following queries:
"compute the sum of sales, grouping by city and item"
"compute the sum of sales, grouping by city"
"compute the sum of sales, grouping by item"

The total no of cuboid, or group-by's that can be computed for this data cube is 2^3 =8
The possible group-by's are as follows:
{(city, item, year), (city, item), (city, year), (item, year), (city), (item), (year), ()} where () means group-by is empty



→The base cuboid contains all three dimensions city, item and year.
→The apex cuboid or 0-D cuboid refers to the case where the group by is empty. It contains the total sum of all the sales
→ the base cuboid is the least generalized (most specific) of the cuboid
→ the Apex cuboid is the most generalized (least specific) of the cuboids
The statement such as
"compute cube sales_cube"
Would explicitly use to compute the sales aggregate cuboids for all 8 subset of the set[city, year, item] including the empty subset

Curse of dimensionality

→A Major challenge related to precomputation is that the required storage space may explode if all the cuboids in the data cube are precomputed

→Especially, when cube has many dimensions

Example:

Time is usually explored not only at one conceptual level (e.g. year) but rather at multiple conceptual levels such as in the hierarchy

"day<month<quarter<year".

For n dimensional data cube, the total number of cuboids that can be generated is:

total no. of cuboids= $\prod_{i=1}^{n}(Li + 1)$

where Li is the number of levels associated with the dimension i including top level all.

1.2 Partial materialization

Materialization is a precomputed table comprising aggregated data from fact and dimension table

There are three choices for data cube materialization:

→ No materialization

→ Full materialization

→ Partial materialization

No materialization:

Do not precompute any of the non-base cuboids.

This leads to computing expensive multidimensional aggregates on the fly, which can be extremely slow.

Full materialization:

Precompute all the cuboids.

The resulting lattice of computed cuboid is referred as "full cube".

This choice requires huge amount of memory space in order to store all precomputed cuboids

Partial materialization:

Selectively compute a proper subset of the whole set of possible cuboids.

We may compute the subset of the cube, which contains only those cells that satisfy some user-specified criterion.

1.3 Indexing OLAP data

→Indexing will be done on OLAP data to facilitate efficient data accessing.

→There are two ways of indexing OLAP data:

Bitmap indexing

Join indexing

Bitmap Indexing

→ Popular indexing method, because it allows quick searching in data cube

The bitmap index is an alternative representation of the record ID (RID) list. In the bitmap index for a given attribute, there is a distinct bit vector, By, for each value v in the domain of the attribute. If the domain of a given attribute consists of n values, then n bits are needed for each entry in the bitmap index (i.e., there are n bit vectors). If the attribute has the value v for a given row in the data table, then the bit representing that value is set to 1 in the corresponding row of the bitmap index. All other bits for that row are set to 0.

Bitmap indexing provides reduction in space and Input/Output space

Consider the following example:

Base table

RID	*item*	*city*
R1	H	V
R2	C	V
R3	P	V
R4	S	V
R5	H	T
R6	C	T
R7	P	T
R8	S	T

item bitmap index table

RID	H	C	P	S
R1	1	0	0	0
R2	0	1	0	0
R3	0	0	1	0
R4	0	0	0	1
R5	1	0	0	0
R6	0	1	0	0
R7	0	0	1	0
R8	0	0	0	1

city bitmap index table

RID	V	T
R1	1	0
R2	1	0
R3	1	0
R4	1	0
R5	0	1
R6	0	1
R7	0	1
R8	0	1

Note: H for "home entertainment," C for "computer," P for "phone," S for "security," V for "Vancouver," T for "Toronto."

Consider the following example to compute the following query:

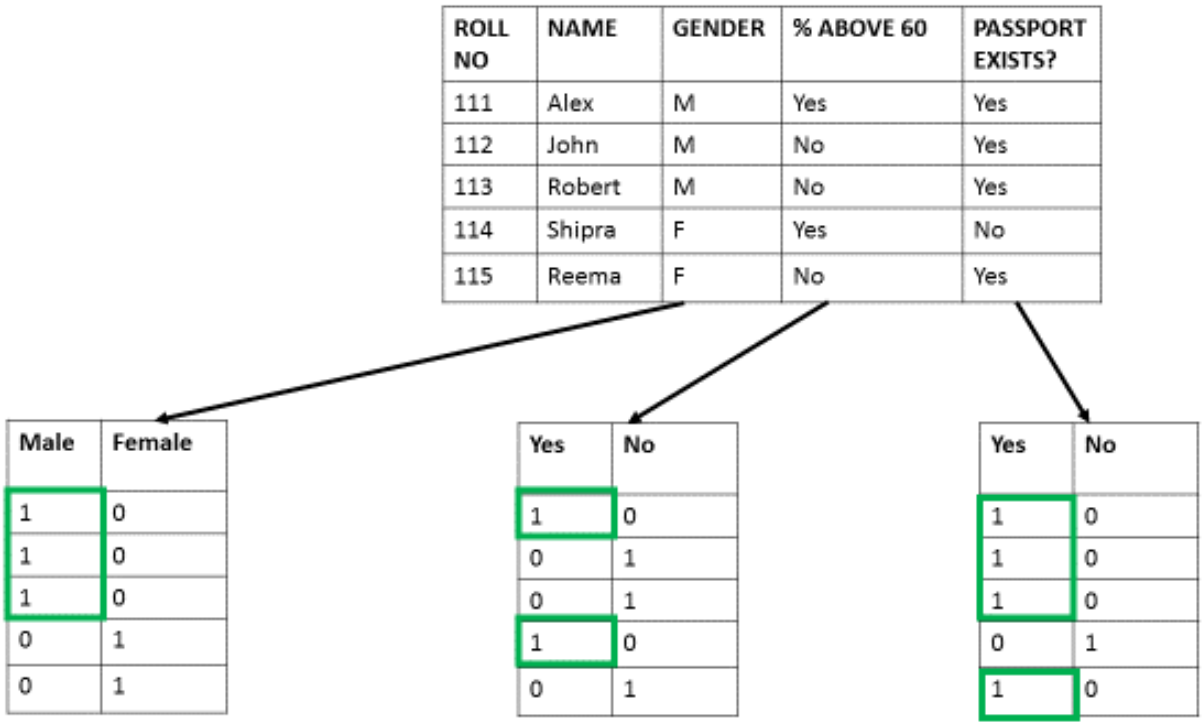
Select * from student where gender= M and %above 60= Yes and passport exists= Yes

ROLL NO	NAME	GENDER	% ABOVE 60	PASSPORT EXISTS?
111	Alex	M	Yes	Yes
112	John	M	No	Yes
113	Robert	M	No	Yes
114	Shipra	F	Yes	No
115	Reema	F	No	Yes

Male	Female
1	0
1	0
1	0
0	1
0	1

Yes	No
1	0
0	1
0	1
1	0
0	1

Yes	No
1	0
1	0
1	0
0	1
1	0



Select * from student where gender= M and %above 60= Yes and passport exists= Yes

ROLL NO	NAME	GENDER	% ABOVE 60	PASSPORT EXISTS?
111	Alex	M	Yes	Yes

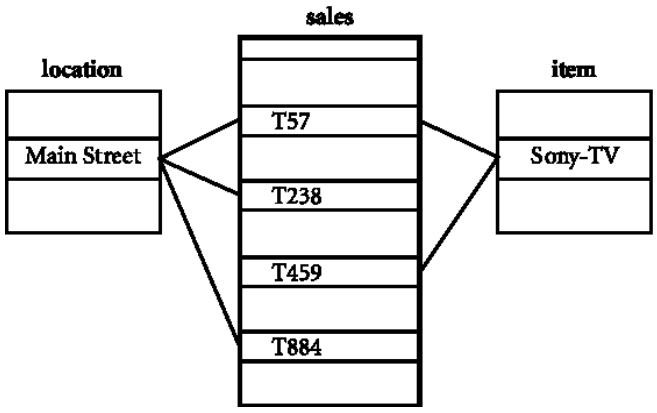
Join Indexing

Join indexing registers the joinable rows of two relations from a relational database.

Example:

If two relation R(RID, A) and S(B, SID) join on the attribute A and B, then the join index record contains the pair(RID, SID), where RID and SID are record identifier from the R and S relation. Join Index records can identify joinable tuples without performing costly join operations.

Consider an example of "AllElectronics" star schema of a join index relationship between the "sales" fact table and the "location" and "item" dimension tables



Above figure shows the linkage between sales fact table and location and item dimension table

Join index table for location/sales

location	sales_key
...	...
Main Street	T57
Main Street	T238
Main Street	T884
...	...

Join index table for item/sales

item	sales_key
...	...
Sony-TV	T57
Sony-TV	T459
...	...

Join index table linking two dimensions location/item/sales

location	item	sales_key
...	...	...
Main Street	Sony-TV	T57
...	...	...

To speed up query processing, the join indexing and bitmap indexing methods can be integrated to form "bitmapped join indices"

1.4 Efficient processing of OLAP Queries

The purpose of materialization cuboid and constructing OLAP index structure is to speed up query processing in data center.

Query processing in data cube should proceed as follows:

→Determine which operation should be performed on available cuboids

This involves transformation of any selection, projection, roll-up, drill down operation specified in the query into corresponding SQL and OLAP operation.

→Determine to which materialized cuboids the relevant operation should be performed.

This involves identifying all of the materialized cuboid that may potentially be used to answer the query

Example:

Suppose we create a data cube for AllElectronics

sales_cube[time, item, location]

Dimension hierarchy used are:

For time: day<month<quarter<year

For item: item_name<item_brand<type

For location: street<city<state<country

- Example
  - Let the query to be processed be on {*brand, province_or_state*} with the condition "*year = 2010*", and there are 4 materialized cuboids available:
    1) {*year, item_name, city*}
    2) {*year, brand, country*}
    3) {*year, brand, province_or_state*}
    4) {*item_name, province_or_state*} where *year = 2010*

    Which should be selected to process the query?
  - Finer-granularity data cannot be generated from coarser-granularity data
  - So, cuboid 2 cannot be used as *country* is more general concept than *province_or_state*
- Cuboids 1, 3 and 4 can be used to process the query as
  - they have the same set or superset of the dimensions in the query
  - the selection clause in the query can imply the selection in the cuboid
  - The abstraction levels for the *item* and *location* dimensions in these cuboids are at a finer level than *brand* and *province_or_state*, respectively
  - Cuboid 1 would cost most – *item_name* and *city* at lower level than *brand* and *province_or_state*
  - If few *year* values associated with *items* in the cube, but there are several *item_names* for each *brand*, then cuboid 3 is smaller than cuboid 4

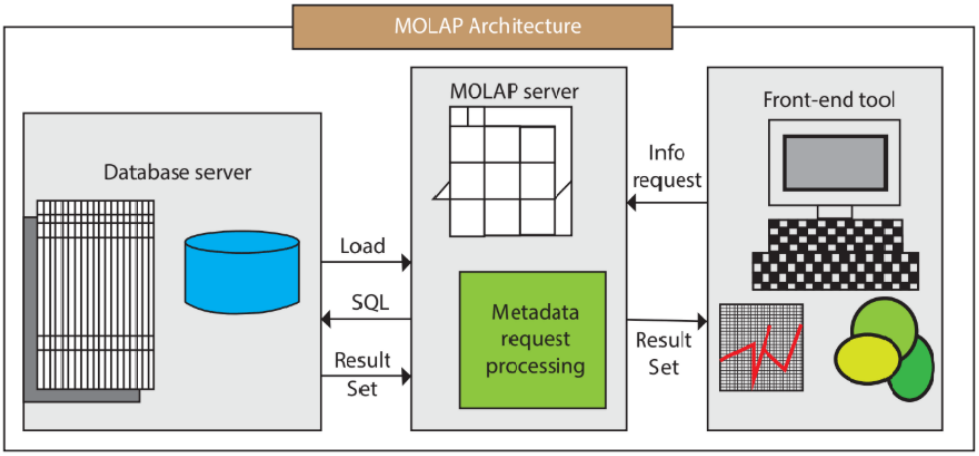
1.5 OLAP server architecture: ROLAP vs MOLAP vs HOLAP

Cubes in data warehouse are stored in three different modes:

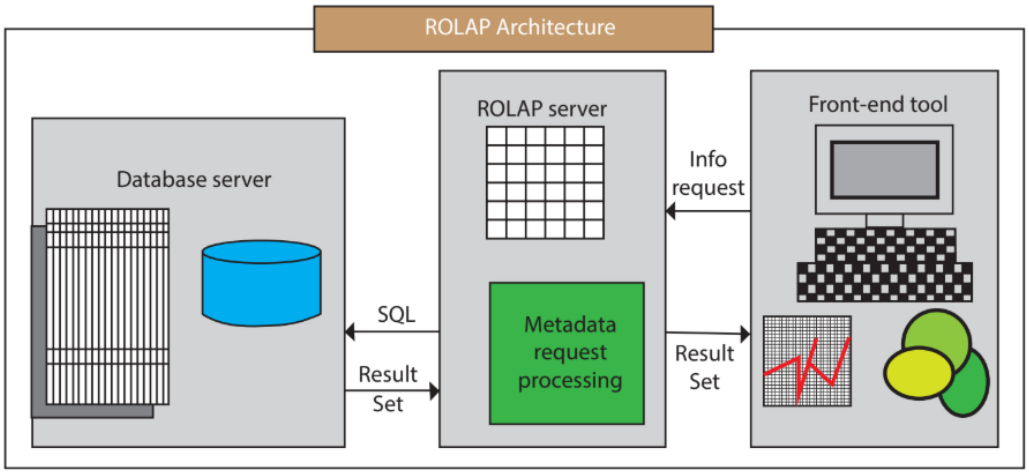
Multidimensional OLAP (MOLAP)
Relational OLAP (ROLAP)
Hybrid OLAP (HOLAP)

Multidimensional OLAP (MOLAP)

→ Traditional mode in OLAP analysis
→ In MOLAP, data is stored in form of multidimensional data cubes and not in relational database.
→ Advantage in MOLAP is it provides excellent query performance and cube are built for fast data retrieval.
→ Disadvantages of this model are that it can handle only limited amount of data.



Relational OLAP (ROLAP)



→ Data in this model is stored in relational data base.
→Advantages of this model is it can handle large volume of data i.e ROLAP is Highly scalable
→In ROLAP, both detailed data and summarized data are stored as shown in figure below.
→ Disadvantage is that performance is slow

RID	Item	◦◦◦	Day	Month	Quarter	Year	Sales
1001	TV		15	10	Q4	2003	250
1002	TV		23	10	Q4	2003	175
	◦◦◦			◦◦◦		◦◦◦	
5001	TV		all	10	Q4	2003	45,786

Hybrid OLAP (HOLAP) servers:

→The hybrid OLAP approach combines ROLAP and MOLAP technology, benefiting from the greater scalability of ROLAP and the faster computation of MOLAP
→ HOLAP server may allow large volumes of detail data to be stored in a relational database, while summary data are kept in a separate MOLAP store.

Difference between ROLAP and MOLAP

ROLAP	MOLAP
Stands for Relational Online Analytical Processing	Stands for Multidimensional Online Analytical Processing
Access of ROLAP is slow	Access of MOLAP is fast
Data is stored in relation table	Data is stored in multidimensional array
ROLAP is used for large volume of data	MOLAP is used for limited volume of data
DBMS facility is strong.	DBMS facility is weak.

1.6 Data mining

Data mining is a process of discovering interesting patterns and knowledge from large amount of data.

Data mining as a synonym for term: knowledge discovery from data or KDD (overall process of converting raw data into useful information)

Data mining has also opened up exciting opportunities for exploring and analyzing new types of data and for analyzing different types of data in new ways.

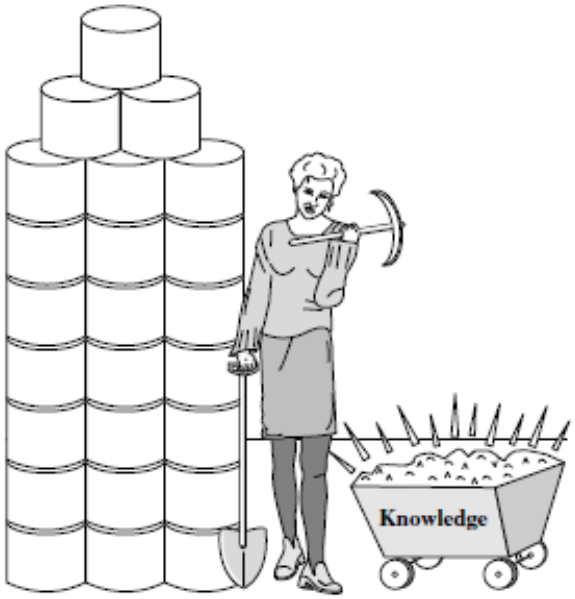
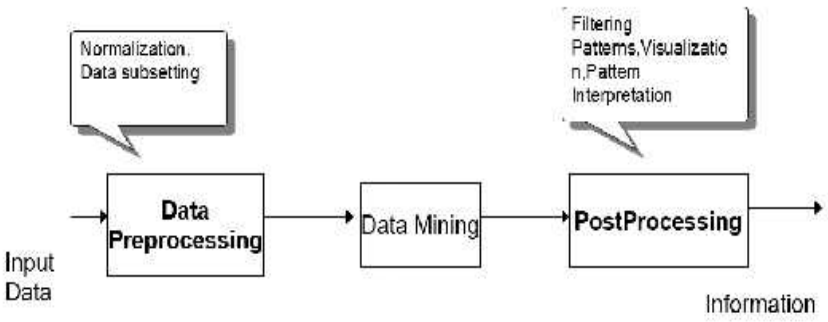


Figure 1.3 Data mining—searching for knowledge (interesting patterns) in data.

1.6 Data Mining and knowledge discovery:

Data mining is an integral part of knowledge discovery in database(KDD), which is an overall process of converting raw data into useful information.

This process consists of series of transitions from "Data Preprocessing" to "post processing" of data mining results.



The input data can be in different formats such as flat files, spread sheets, or relational tables.

The purpose of preprocessing is to transform the raw input data into an appropriate format for subsequent analysis.

Post processing steps ensures that only valid and useful results are incorporated into the decision support system.

KDD Process

The knowledge discovery process is shown in the figure below as an iterative sequence of the following steps:

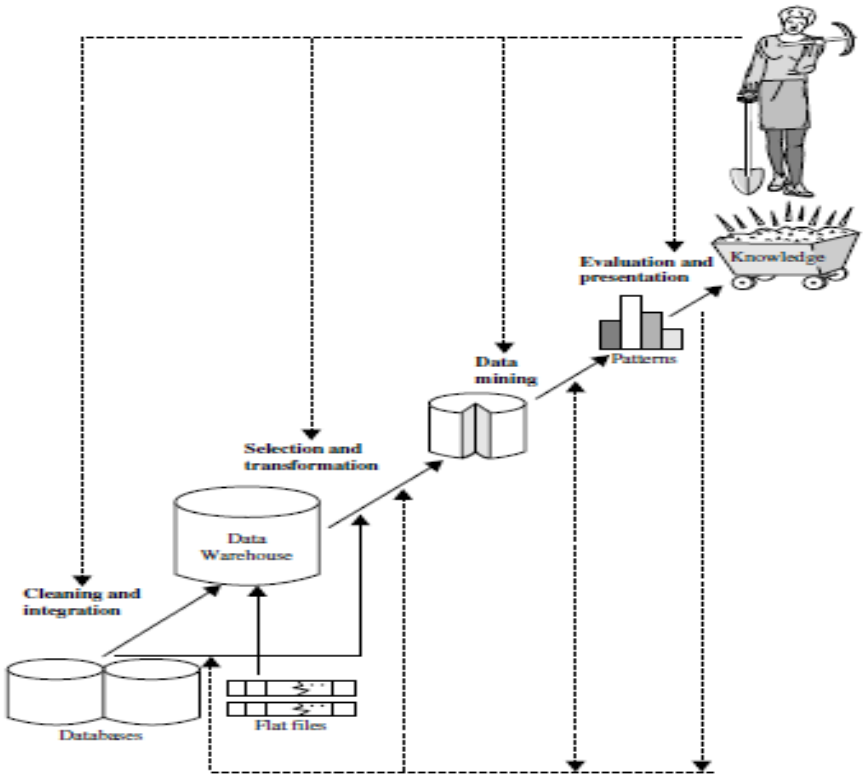


Figure 1.4 Data mining as a step in the process of knowledge discovery.

1. Data cleaning (to remove noise and inconsistent data)
2. Data integration (where multiple data sources may be combined)
3. Data selection (where data relevant to the analysis task are retrieved from the database)
4. Data transformation (where data are transformed and consolidated into forms appropriate for mining by performing summary or aggregation operations)
5. Data mining (an essential process where intelligent methods are applied to extract data patterns)
6. Pattern evaluation (to identify the truly interesting patterns representing knowledge based on interestingness measures)
7. Knowledge presentation (where visualization and knowledge representation techniques are used to present mined knowledge to users)

1.7 Motivating challenges of data mining

→Scalability
→High dimensionality
→Heterogeneous and complex data
→Data ownership and distribution
→Non-traditional analysis

Scalability:

→Because of the advances in data generation and collection, data set with the size of gigabytes, terabytes are becoming common.
→Massive data sets cannot fit into main memory.
→Need to develop scalable data mining algorithms to mine massive data sets.

High dimensionality:

→It is common to encounter data sets with hundreds or thousands of attributes.

→Example: data set that contains measurement of temperature at various location.
→Computational complexity increases as dimensionality increases
→Traditional data analysis techniques were developed for low dimensional data
→Need to develop data mining algorithm to handle high dimensionality

Heterogeneous and complex data:
→Traditional data analysis method deals with data sets containing attributes of same type
→Complex data set contains images, videos, text etc.
→Need to develop mining methods to handle complex data sets.

Data ownership and distribution:
→Data is not stored in one location, or owned by one organization
→Data is geographically distributed among resources belonging to multiple entities
→Need to develop distributed data mining algorithm to handle distributed data sets.

Non-traditional analysis:
→Traditional approach is based on hypothesis and test paradigm
→A hypothesis is proposed, an experiment is designed to gather the data, and then data is analyzed with respect to the hypothesis.
→This process is extremely labor intensive.
→Need to develop data mining methods to automate the process of hypothesis generation and evaluation

1.8 Data Mining Tasks

Data mining tasks are generally divided into two major categories:

Predictive tasks
Descriptive tasks

Predictive tasks:

The objective of this task is to predict the values of a particular attribute based on the values of other attributes.
The attribute to be predicted is called: **target or dependent variable**
Attribute used for making prediction are called: **explanatory or independent variable**

Descriptive tasks:

The objective is to derive patterns that summarize the relationship in the data.
Descriptive data mining task require post processing technique to validate and explain the results

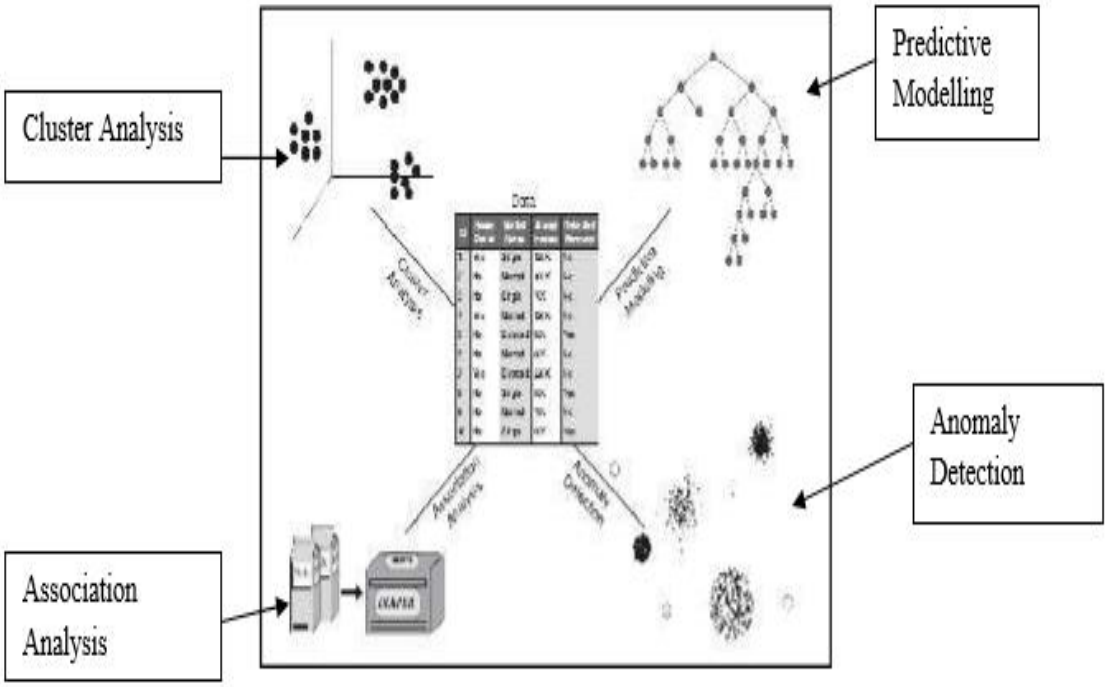


Figure 2.3: Data Mining Tasks

Four of the Core Data Mining tasks:
1) Predictive Modelling
2) Association analysis
3) Cluster analysis
4) Anomaly detection

Predictive Modelling

Process that uses data mining and probability to forecast outcomes.

There are two types of predictive modelling tasks:

Classification
Regression

Classification: It is used for discrete target variables

Ex: Web user will make purchase at an online bookstore is a classification task, because the target variable is binary valued.

Regression: It is used for continuous target variables.

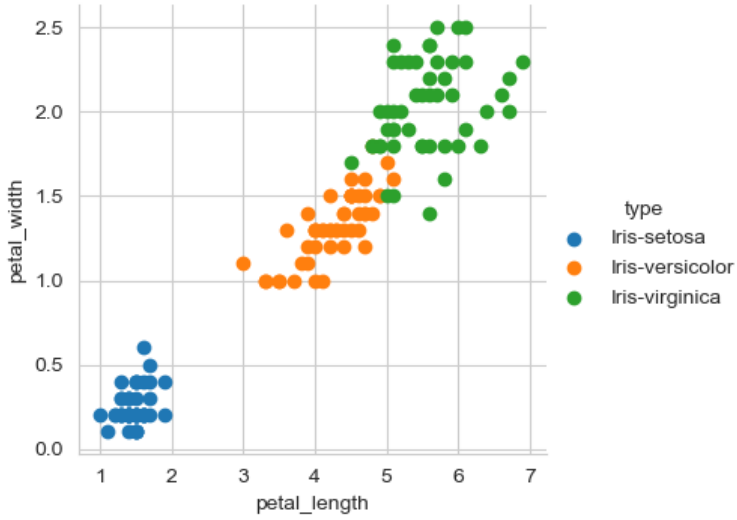
Ex: forecasting the future price of a stock is regression task because price is a Continuous values attribute

Example 1(Classification): Predicting the type of a flower

Consider a task of classifying an Iris flower as to whether it belongs to one of the following three Iris species:

Setosa
Versicolor
virginica

To perform this task, we need a data set containing the characteristics of various flowers of these three species.



Above figure shows a plot of petal width versus petal length for 150 flowers in Iris data set. Petal width is broken into the categories low, medium, high. Also petal length is broken into categories low, medium and high.
Based on these categories of petal width and length, following rules can be derived:
Petal width low and Petal length low implies Setosa
Petal width medium and Petal length medium implies Versicolour
Petal width high and Petal high low implies Virginica

Example 2(Regression):
Predicting the income of an Employee based on Education, Experience, and other demographic factors.

Association Analysis
The Goal of association analysis is to extract the most of interesting patterns in an efficient manner
Ex: Market based analysis: We may discover the rule that {diapers}➔{Milk}, which suggests that customers who buy diapers also tend to buy milk.
Below table illustrates point-of-sale data collected at the checkout counter of a grocery store.

Transaction ID	Items
1	{Bread, Butter, Diapers, Milk}
2	{coffee, Diapers, Cookies, Milk}
3	{Bread, Butter, Coffee, Diapers, Milk, Eggs}
4	{Bread, Butter, Salmon, Chicken}
5	{Eggs, Bread, Butter}

Cluster Analysis
Used to identify data objects that are similar to one another
Cluster analysis is used to find groups of closely related observations so that observations that belong to the same cluster are more similar to each other than observations that belong to other clusters
Example: Document clustering

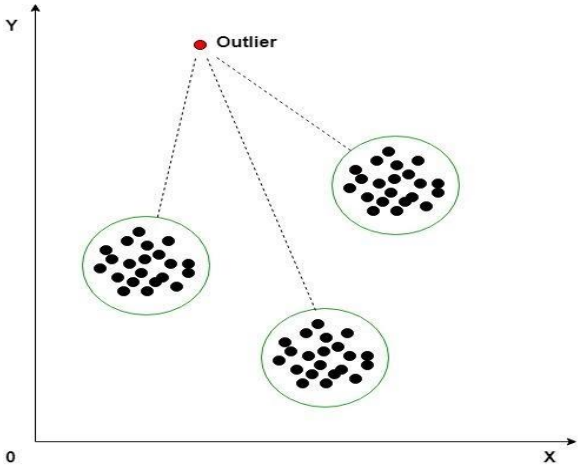
Article	Words
1	Dollar:1, industry:4, country:2, loan:3, government:2
2	Machinery:2, labor:3, market:4, industry:2, work:3, country:1
3	Job:5, inflation3, rise:2, jobless:2, market: 3, country:2
4	Patient:4, symptoms:2, drug:3, health:2, clinic:2, doctor:2
5	Death:2, cancer:4, drug:3, public:4, health:4, director:1

Collection of news articles shown in the table above can be grouped. Each article is represented as a set of word-frequency pair (w, c) where w is a word and c is the number of times the word appears in the article.

First cluster consists of first three articles, which correspond to the news about the economy while the second cluster contains the last four articles which corresponds to news about health care.

Anomaly Detection

It is the task of identifying observations whose characteristics are significantly different from the rest of the data. Such observations are known as anomalies or outliers.



Applications of anomalies are: fraud detection, network intrusion, unusual patterns of diseases

Example: Credit card fraud detection

A credit card company records transaction made by every credit card holder, along with personal information such as credit card limit, age, annual income, and address.

Since the number of fraudulent case is small compared to the number of legitimate transaction, anomaly detection technique can be applied to build a profile of legitimate transactions for the users. When a new transaction arrives, it is compared against the profile of the user. If the characteristics of the transaction are very different from the previously created profile, then the transaction is flagged as potentially fraudulent.

1.9 Types of Data

What is Data set?

Collection of data objects and their attributes

Other names for a data object are record, point, vector, pattern, event, case, sample, observation.

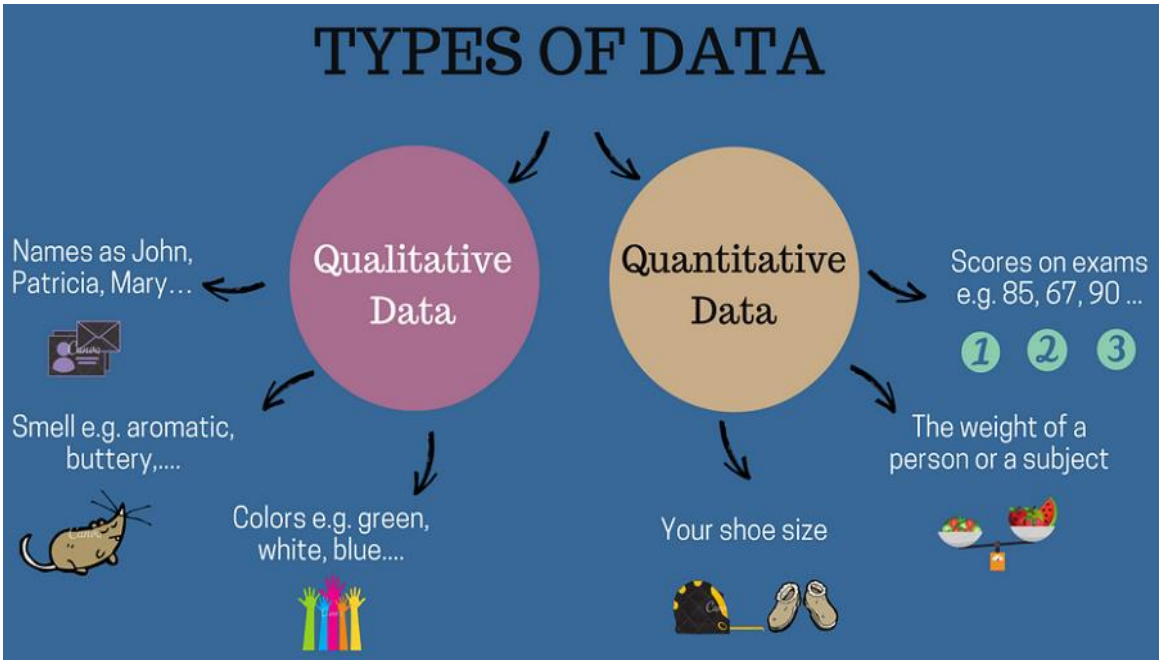
An attribute is a property or characteristic of an object

Data can be classified into two broad types:

- Qualitative
- Quantitative

Quantitative data can be counted, measured, and expressed using numbers.

Example: colours of eyes: black, brown, blue
exam result: pass or fail
Qualitative data is descriptive and conceptual.
Example: Height of a person
Shoe size
Scores in exam



1.10 Types of an Attribute

An Attribute is a property or characteristic of an object.
Different types of attributes:
Nominal
Ordinal
Interval
Ratio

Nominal Attributes

Nominal means "relating to names."
The values of a nominal attribute are just names of things
Nominal values provide only enough information to distinguish one object from another(=, ≠)
Example:
Suppose that hair color and marital status are two attributes
In our application, possible values for hair color are black, brown, blond, red, grey, and white.
The attribute marital status can take on the values single, married, divorced, and widowed. Both hair color and marital status are nominal attributes.
Another example of a nominal attribute is occupation, with the values teacher, dentist, programmer, farmer, and so on.

Ordinal Attributes (<, >, >=, <=)

An ordinal attribute is an attribute with possible values that have a meaningful order or ranking among them
Suppose that drink size corresponds to the size of drinks available at a fast-food restaurant. This attribute has three possible values: small, medium, and large. The values have a meaningful sequence
Other examples of ordinal attributes include grade (e.g., A+, A, A−, B+, and so on)

Professional ranks can be enumerated in a sequential order: for example, assistant, associate, specialist
Customer satisfaction has the following ordinal categories: 0: very dissatisfied, 1: somewhat dissatisfied, 2: neutral, 3: satisfied, and 4: very satisfied.

Interval-Scaled Attributes (+ or -)
Interval-scaled attributes are measured on a scale of equal-size units
Attributes allow us to compare and quantify the difference between values.
For example, a temperature of 20◦C is five degrees higher than a temperature of 15◦C.
Calendar dates are another example. For instance, the years 2002 and 2010 are eight years apart.

Ratio scaled attributes (* or /)
A measurement is ratio-scaled, we can speak of a value as being a multiple (or ratio) of another value
Years of experience (e.g., the objects are employees) and number of words (e.g., the objects are documents).

1.10.1 Describing Attributes by the number of values
Discrete Attribute:
A discrete attribute has a finite set of values.
Example: Marital status (single, married, divorced, widowed)
Binary attribute
Binary attribute is special case of discrete attribute and assume only two values (true/ false, yes/no, male/female, 0/1)
Example: result (pass, fail)
Continuous attribute
Continuous attribute is the one whose values are real numbers
Example: temperature, height or weight

1.11 **Characteristics of a Data Sets**
Three characteristics that apply to many data sets and have a significant impact on the data mining techniques:

Dimensionality
Sparsity
Resolution

Dimensionality
→ The dimensionality of a data set is the number of attributes that an object in a data possess.
→ Data with the small number of dimensions tends to be qualitatively different than moderate or high dimensional data.
→ Difficulties associated with high dimensional data are sometimes referred to as the curse of dimensionality
→ Example health care data

Sparsity
→For some data set, most attributes of an object have value 0
→ sparsity is an advantage because usually only the non-zero values need to be stored and manipulated

Resolution
→ It is frequently possible to obtain data at different levels of resolution

→Properties of data are different at different levels of resolution
→Example: The surface of the earth seems very uneven at a resolution of few meters, but is relatively smooth at a resolution of tens of kilometres.
→Pattern of the data depends on the resolution
→If the resolution is too fine, a pattern may not be visible or may be buried in noise.
→If the resolution is too coarse, the pattern may disappear

1.12 Types of data sets

Following are the types of data sets:

Record data
Graph based data
Ordered data

Record data

→ Collection of records (data objects), each of which consists of a fixed set of data fields(attributes)
→There is explicit relationship among records or data fields
→Record files are usually stored either in flat files or in relational database
→Types of record data are:

Transaction data
Data matrix
Document term matrix

Transaction data

Transaction data is a special type of record data where each record involves a set of items. Consider the grocery store. The set of products purchased by the customer constitutes a transaction. While individual products that were purchased are the items. This type of data is called market basket data.

Transaction ID	Items
1	**{Bread, soda, Milk}**
2	**{beer, bread}**
3	**{Beer, soda, Diapers, Milk}**
4	**{Bread, Butter, Chicken}**
5	**{soda, Bread, Butter}**

Data matrix

→Collection of data with same fixed set of numeric attributes
→ These data objects can be interpreted as m by n matrix, where there are m rows, one for each objects and n columns, one for each attribute.
→Also called a pattern matrix

Projection of x Load	Projection of y Load	Distance	Load	Thickness
10.23	5.27	15.22	27	1.2
12.65	6.25	16.22	22	1.1
13.54	7.23	17.34	23	1.2
14.27	8.43	18.45	25	0.9

(c) Data matrix.

Document term matrix

A special case of a data matrix in which the attributes are of the same type and are asymmetric. In this, documents are the rows of this matrix, while the term are the columns

	team	coach	play	ball	score	game	win	lost	timeout	season
Document 1	3	0	5	0	2	6	0	2	0	2
Document 2	0	7	0	2	1	0	0	3	0	0
Document 3	0	1	0	0	1	2	2	0	3	0

(d) Document-term matrix.

Graph based data

→Convenient and powerful representation of data

→There are two different types of graph based data

Data with Relationship among objects

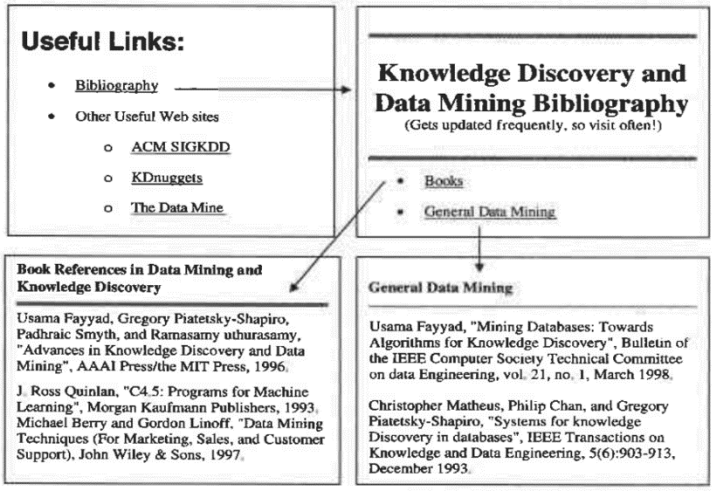
Data with objects that are graphs

Data with Relationship among objects:

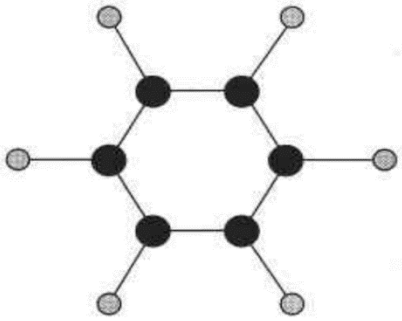
The relationship among the objects frequently convey important information's. In such cases, data is often represented as a graph.

Data objects are mapped to nodes of the graph, while relationship among the objects are mapped to links between objects.

Consider web pages on the world wide web, which contains both text and links to other pages.



(a) Linked Web pages.



(b) Benzene molecule.

Data with objects that are graphs:

If objects have structures, that is objects contain sub-objects that have relationships, then such objects are represented as graphs. For example, the structure of chemical compounds can be represented by a graph where the nodes are atoms and the links between node are chemical bonds. Above figure(b) shows a ball-and-stick diagram of the chemical benzene, which contains atoms of carbon (black) and hydrogen (grey).

Ordered data

→For some type of data, the attributes have relationship that involve order in time or space.

Ordered data

Different types of ordered data are:

Sequential data
Time series data
Spatial data

Sequential data:

→Also referred as Temporal data

→Each record has a time associated with it

→Consider a retail transaction data set that also stores the time at which the transaction took place.

→Using this information, it is possible to find patterns such as "people who buy DVD players tends to buy DVDs in the period immediately following the purchases"

Time	Customer	Items Purchased
t1	C1	A, B
t2	C3	A, C
t2	C1	C, D
t3	C2	A, D
t4	C2	E
t5	C1	A, E

Customer	Time and Items Purchased
C1	(t1: A,B) (t2:C,D) (t5:A,E)
C2	(t3: A, D) (t4: E)
C3	(t2: A, C)

(a) Sequential transaction data.

In the above table, there are five different times- t1, t2, t3, t4 and t5

Three different customers- C1, C2, and C3.

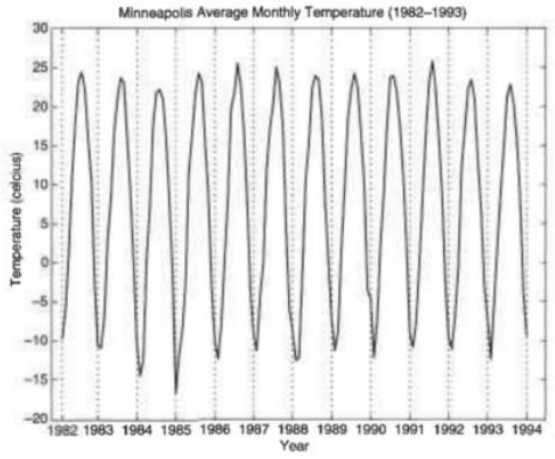
Five different items- A, B, C, D and E

In the top table, each row corresponds to the items purchased at a particular time by each customer. For instance, at time t3, customer C2 purchased items A and D. In the bottom table, the same information is displayed, but each row corresponds to a particular customer. Each row contains information on each transaction involving the customer, where a transaction is considered to be a set of items and the time at which those items were purchased. Example: Customer C3 bought items A and C at time t2.

Time series data:

→Time series data is a special type of sequential data in which each record is a time series

→Series of measurements taken over time



(c) Temperature time series.

Above figure shows a time series of the average monthly temperature for Minneapolis during the year 1982 to 1994.

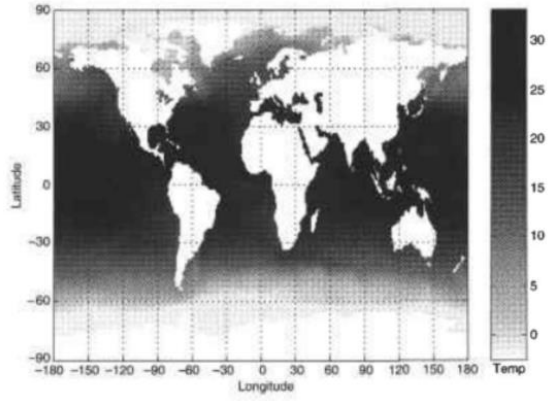
If two measurements are close in time, then the values of those measurements are often very similar which is termed as **temporal autocorrelation**.

Spatial Data:

→Some objects have spatial attributes such as positions and areas

→Example: Weather data

Objects that are physically close usually have similar values. For instance, two points on earth that are close to each other have similar values for temperature and rainfall.



(d) Spatial temperature data.

1.13 Data Quality

→Today's real world databases are highly susceptible to noise and inconsistent data due to their huge size

→Data mining applications focuses on:

* Detection and correction of data Quality problems (data cleaning)
* Use of algorithms that can tolerate poor data quality

Noise and artifacts

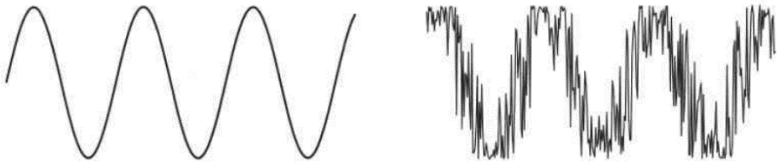
Noise is a random error or variance in a measured variable.

→It may involve the distortion of values or addition of spurious objects

Distortion of the data are referred as **artifacts**

Below figure shows a time series before and after it has been disrupted by random noise.

If more noise were added to the time series, its shape would be lost.



(a) Time series. (b) Time series with noise.

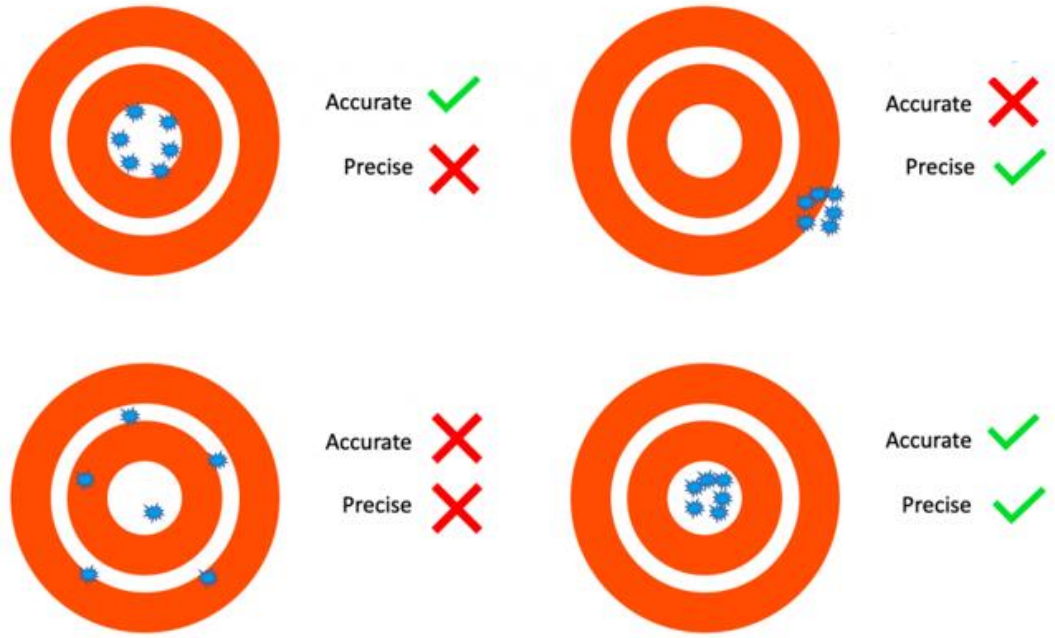
Precision, Bias and Accuracy

Precision: The closeness of repeated measurements to one another

Bias: Measure of how far the expected value is from the true value

Accurate: Closeness of the measurement to the true value of the quantity being measured.

Consider the following Example:



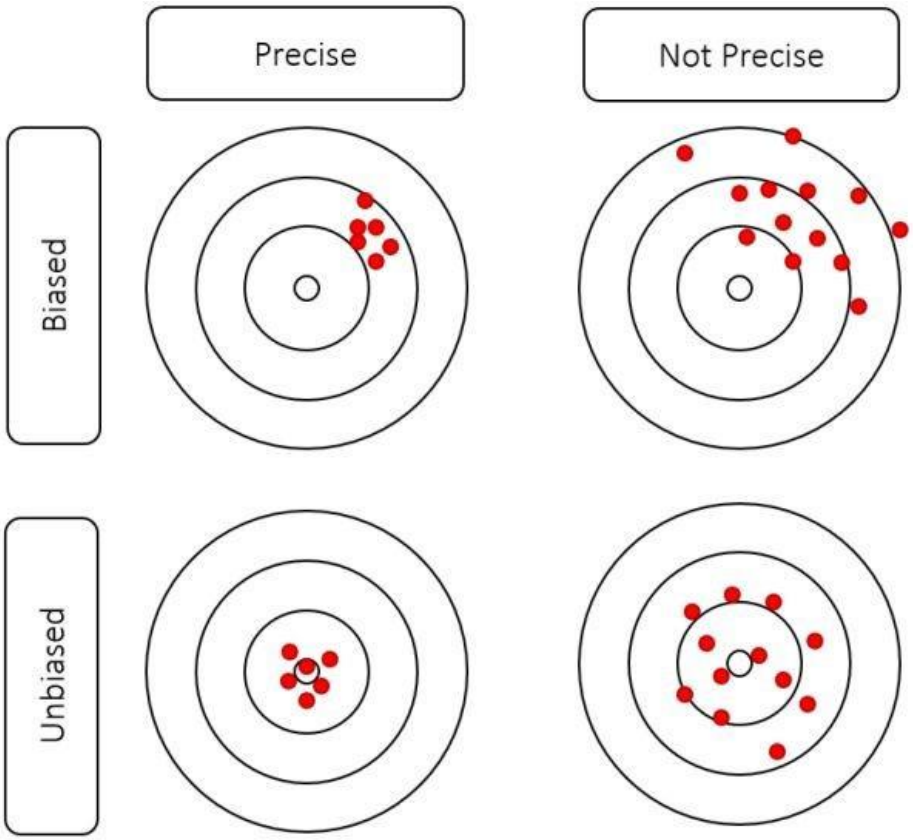


Figure 2.1: Precision & Bias

Outlier

Outliers are data objects that have characteristics that are different from most of the other data objects in the data set

→An outlier can cause serious problem in analysis

→Unlike noise, outliers may sometimes be of interest

→For example, in fraud and network intrusion detection, the goal is to find unusual objects from large number of normal ones

Missing values

→Not unusual for an object to be missing one or more attribute values

→In some cases, information is not collected

→Example: Some people decline to give their age or weight

Missing values should be taken into account during the data analysis.

There are several strategies for dealing with missing data.

Imagine that you need to analyse AllElectronics sales and customer data.

You note that many tuples have not recorded value for several attributes, such as customer income.

How can we fill in the missing values for this attribute?

1. Ignore the tuple or Attribute:

This method is not very effective, unless the tuple contains several attributes with missing values.

2. Fill in the missing value manually:

This approach is time consuming and may not be feasible given a large data set with many missing values.

3. Use a global constant to fill in the missing value:

Replace all missing attribute values by the same constant such as a label like *"Unknown".*

*4. Use a measure of central tendency for the attribute (e.g., the mean or median) to fill in the missing value***:**

For example, suppose that the average income of AllElectronics customers is $56,000. Use this value to replace the missing value for income.

5. Use the most probable value to fill in the missing value:

Using the other customer attributes in your data set, you may construct a decision tree to predict the missing values for *income.*

Inconsistent values

→Data can contain inconsistent values

→Example: Consider an address field where both a zip code and city are listed, but the specified zip code area is not contained in that city

→Some type of inconsistences are easy to detect

→Example: Person's height cannot be negative

→Once an inconsistency has been detected, it is sometimes possible to correct the data

Duplicate data

→A data set may include data objects that are duplicates or almost duplicate of one another

→To detect and eliminate such duplicates two main issues must be addressed:

* If there are two objects that actually represent a single object then the values of corresponding attributes may differ, these inconsistent values must be resolved.

* Care needs to be taken to avoid accidentally combining data objects that are similar, but not duplicates such as two distinct people with identical names

The term ***deduplication*** is often used to refer to the process of dealing with these issues

1.14 Data Preprocessing

Data Preprocessing is a Data mining technique that involves transforming raw data as understandable format

→ Real world data is often incomplete, inconsistent and likely to contain many errors

→ Data preprocessing resolves such issues

Important Approaches for data Pre Processing are:

Aggregation
Sampling
Dimensionality reduction
Feature subset selection
Feature creation
Discretization and binarization
Variable transformation

Aggregation

→"less is more"

→Aggregation is combining two or more objects into single object.

→Disadvantages of aggregation is the potential loss of interesting details

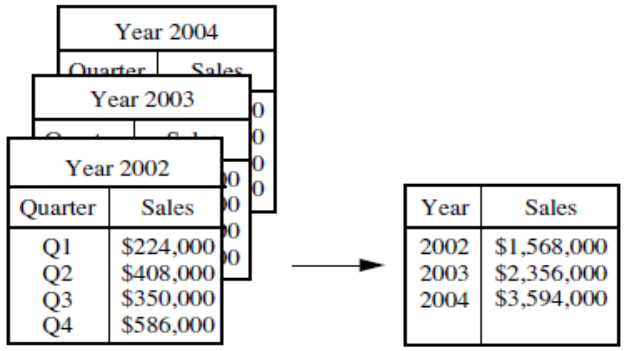
→Advantage: requires less memory, and processing time

Table 2.4. Data set containing information about customer purchases.

Transaction ID	Item	Store Location	Date	Price	...
⋮	⋮	⋮	⋮	⋮	
101123	Watch	Chicago	09/06/04	$25.99	...
101123	Battery	Chicago	09/06/04	$5.99	...
101124	Shoes	Minneapolis	09/06/04	$75.00	...
⋮	⋮	⋮	⋮	⋮	

Consider the above transaction, recording the daily sales of product in various store locations. For different days over the course of a year.

One way to aggregate transaction for this data set is to replace all the transaction of a single store with a single storewide transaction.
This reduces the hundreds or thousands of transactions that occurs daily at a specific store to a single daily transaction.
Quantitative attributes such as price, are typically aggregated by taking sum or an average.
A Qualitative attribute such as item can either be omitted or summarized as the set of all the items that were sold at that location.



Sales data for a given branch of *AllElectronics* for the years 2002 to 2004. On the left, the sales are shown per quarter. On the right, the data are aggregated to provide the annual sales.

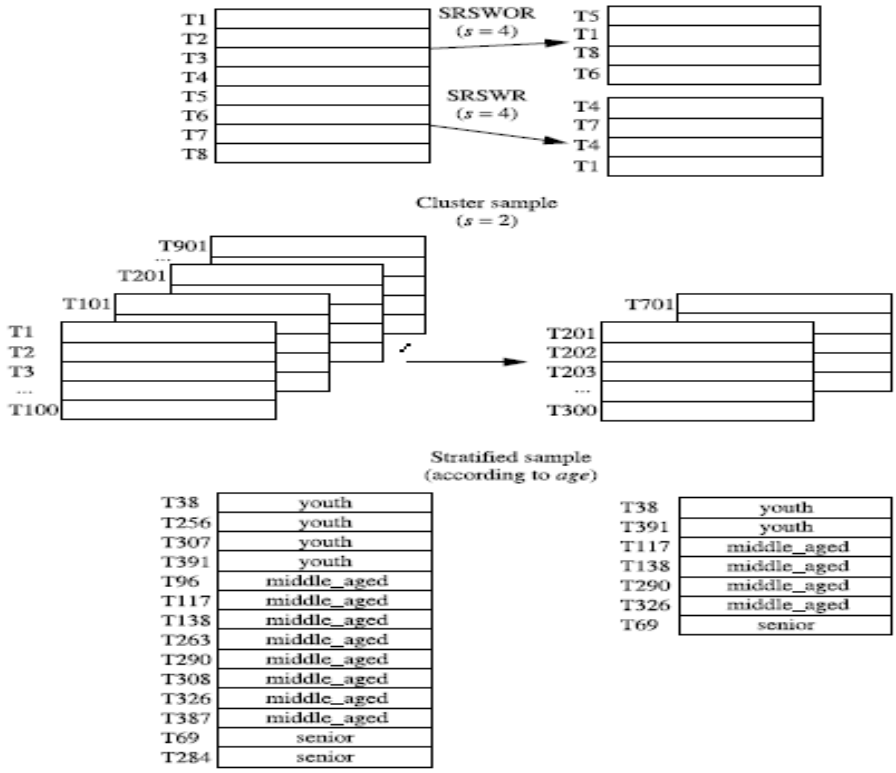
Sampling

Sampling can be used as a data reduction technique because it allows a large data set to be represented by a much smaller random data sample (or subset).
→ Approach used for selecting a subset of the data objects to be analyzed
→ Sampling can be very useful in data mining.

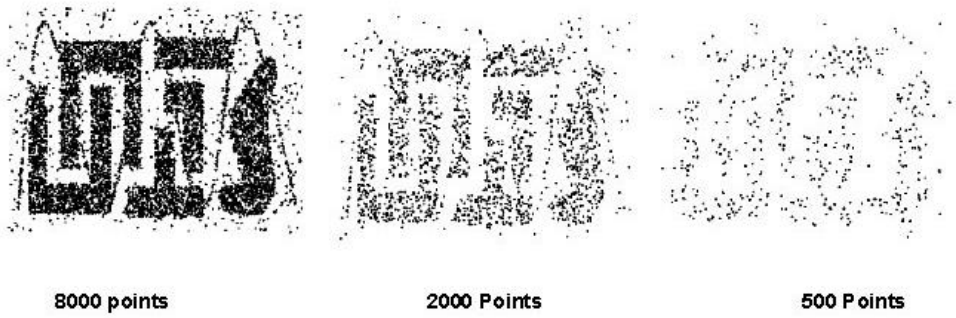
Sampling Approaches

→Simple random sample without replacement (SRSWOR)
→Simple random sample with replacement (SRSWR)
→Cluster sample
→Stratified sample

There is an equal probability of selecting any particular item
There are two variations on random sampling:
1) Sampling without replacement: As each item is selected, it is removed from the population
2) Sampling with replacement: Objects are not removed from the population as they are selected for the sample. In sampling with replacement, the same object can be picked up more than once
4) Cluster sample: Random documents are removed from the set of documents
3) Stratified sampling – Equal numbers of objects are drawn from each group though the groups are different size



Once a sampling technique has been selected, it is still necessary to choose the sample size. Larger sample size increase the probability that a sample will be representative, but they also eliminate much of the advantage of the sampling.
With smaller sample sizes, pattern may be missed or erroneous pattern can be detected.



Progressive sampling:
→Proper sampling size is difficult to determine
→So adaptive or progressive sampling schemes are sometimes used
→Start with a small sample and then increase the sample size until the sample of sufficient size has been obtained.
→This technique eliminates the need to determine the correct sample size initially.

Dimensionality Reduction
Data analysis becomes harder as the dimensionality of the data increases.
The term dimensionality reduction is often reserved for those techniques that reduce the dimensionality of a data set by creating new attributes that are a combination of old attributes.
Purpose:

o Avoid curse of dimensionality
o Reduce amount of time and memory required by data mining algorithms
o Allow data to be more easily visualized
o May help to eliminate irrelevant features or reduce noise

Curse of Dimensionality: The curse of dimensionality refers to various phenomena that arise when analysing and organizing data in high-dimensional spaces (often with hundreds or thousands of dimensions) that do not occur in low-dimensional settings such as the three-dimensional physical space of everyday experience.

Amount of data we need: Obviously when numbers of dimensions are more we need a bigger data set. Because only with large dataset we can get more information in higher dimensions. Therefore, as number of dimensions' increases amount of data we need to generalize accuracy grows exponentially.

Feature subset selection

Another way to reduce Dimensionality of data and this is not applicable for redundant and irrelevant features.

Redundant Feature

Duplicate information contained in one or more other attributes

Example: purchase price of product and sales tax paid contains same information

Irrelevant Feature

Contains information that is not useful for data mining task

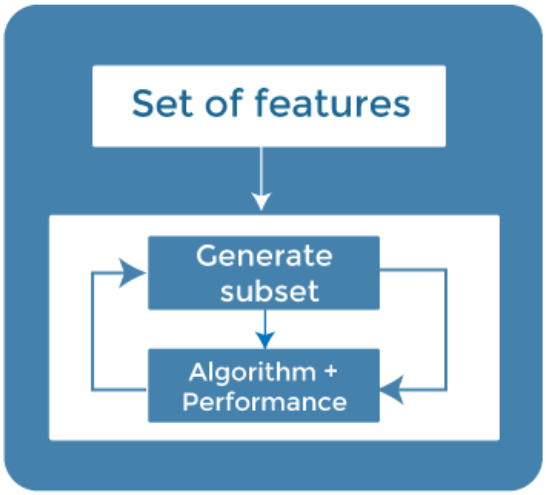
Example: Student ID is irrelevant to the task of predicting their marks

Three standard approaches to feature selection:

Embedded approach
Filter approach
Wrapper approach

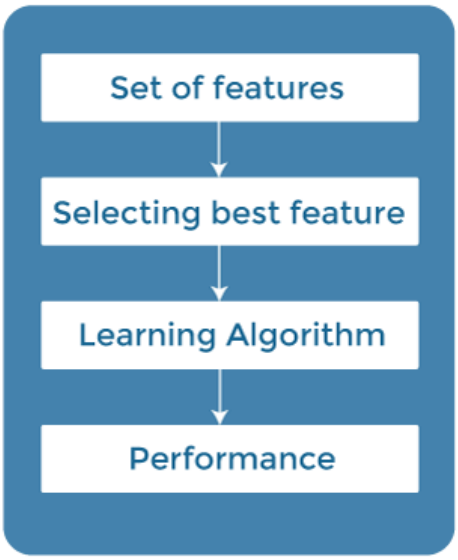
Embedded Approach

→Feature selection occurs naturally as part of the data mining algorithm.
→Algorithm itself decides which attribute to use and which to ignore



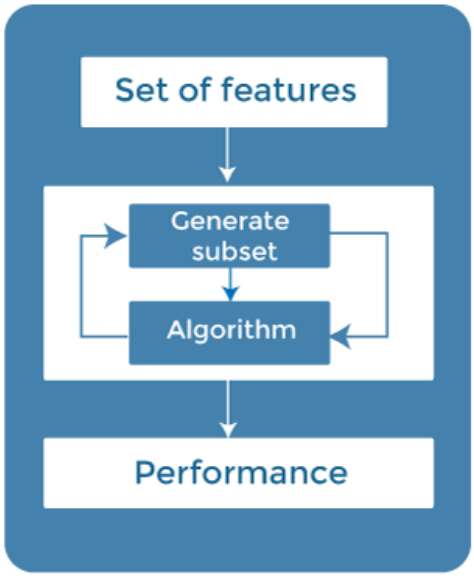
Filter Approaches

→Features are selected before data mining algorithm runs
→Approach independent of data mining task



Wrapper Approach

→ Method uses data mining algorithm as a black box to find best subset of attributes



An Architecture for feature subset selection

Flowchart of a feature subset selection process.

The feature selection process is viewed as consisting of four parts:
A measure for evaluating subset, a search strategy that controls the generation of a new subset of features, a stopping criteria and a validation procedure.
→ feature subset selection is a search over all possible subsets of features.
Many different types of search strategies can be used, but the search should be computationally inexpensive.

Feature Weighting:

→Alternative to eliminate features
→More important features are assigned a higher weight, while less important feature are given lower weight
→Weights are assigned based on the knowledge about the importance of features
→Features with larger weights play a more important role in the model.

Feature Creation

→From the original attributes, a new set of attributes can be created
→Number of new attributes can be smaller than the number of original attributes
→Such attributes can be generated using three methodologies:

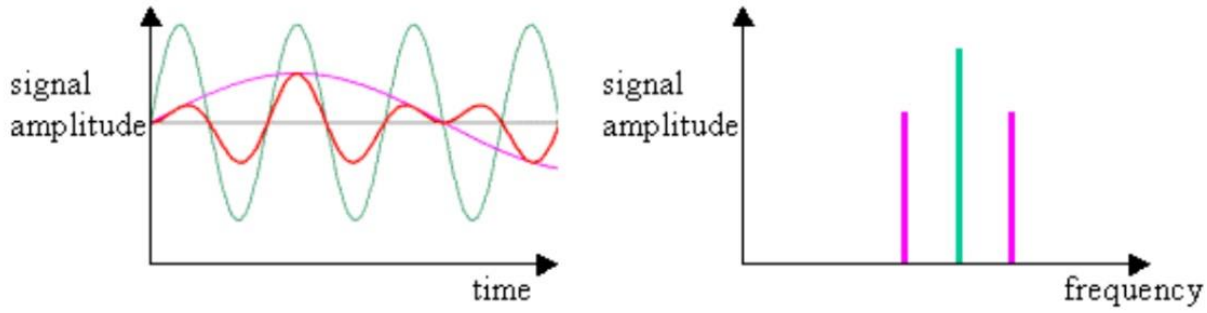
* Feature extraction
* Mapping data to a new space
* Feature construction

Feature Extraction:

→The creation of new set of features from the original raw data is known as feature extraction
→These new reduced set of features should then be able to summarize most of the information contained in the original set of features.
Example: extracting edges from images

Mapping data to a new space:

→ Sometimes a totally different view of data can reveal important and interesting features
Example:
Consider, for example, time series data, which often contains periodic patterns.
If there is only a single periodic pattern and not much noise then the pattern is easily detected.
If, on the other hand, there are a number of periodic patterns and a significant amount of noise is present, then these patterns are hard to detect.
Such patterns can be detected by applying a Fourier transform to the time series in order to change to a representation in which frequency information is explicit



Time domain **Frequency domain**

Feature construction:

→ Sometimes the features in the original data sets have the necessary information, but it is not in a form suitable for the data mining algorithm.
→In this situation, one or more new features constructed out of the original features can be more useful than the original features.

→ Example: dividing mass by volume to get density

Discretization and Binarization

→ **Discretization** is the process of converting a continuous attribute into an ordinal attribute.

→A potentially infinite number of values are mapped into a small number of categories.

→Example:

height measured as {low, medium, high}

age measured as {youth, middle-aged, senior}

→**Binarization** maps a continuous or categorical attribute into one or more binary variables

→Often convert a continuous attribute to a categorical attribute and then convert a categorical attribute to a set of binary attributes

Conversion of a categorical attribute to three binary attributes

Categorical Value	Integer Value	x_1	x_2	x_3
awful	0	0	0	0
poor	1	0	0	1
OK	2	0	1	0
good	3	0	1	1
great	4	1	0	0

Conversion of a categorical attribute to five asymmetric binary attributes

Categorical Value	Integer Value	x_1	x_2	x_3	x_4	x_5
awful	0	1	0	0	0	0
poor	1	0	1	0	0	0
OK	2	0	0	1	0	0
good	3	0	0	0	1	0
great	4	0	0	0	0	1

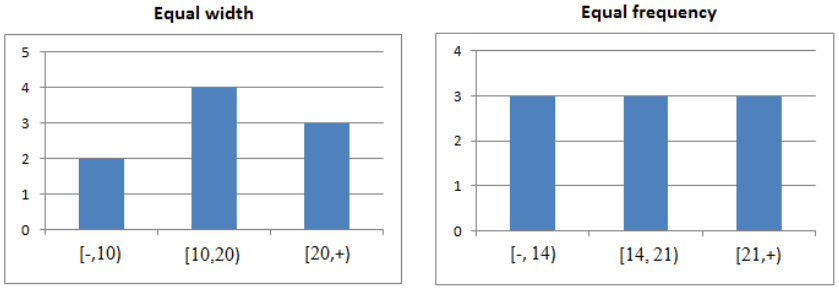
Unsupervised discretization methods:

Equal-width

Equal-frequency

Equal-width approach divides the range of the attributes into a user specified number of intervals each having the same width

Equal-frequency puts the same number of objects into each interval



Consider the following example for Equal Width Interval Discretization:

$$\text{Interval} = \frac{V_{max} - V_{min}}{k}$$

$$\text{Boundaries} = V_{min} + (i \times \text{Interval})$$

Table 1. Solved example for Equal Width Discretization

Data values	V1	V2	V3	V4	V5	V6	V7	V8
Original Continuous Values	10	50	15	20	12	25	40	30
Sorted values	10	12	15	20	25	30	40	50
Description	Here V_{min}= 10 V_{max}=50 let k= 2 So Interval=20 and Boundary=10+20=30							
Discrete Values	1	2	1	1	1	1	2	1

Consider the following example for Equal-Frequency Interval Discretization:

Table 2. Solved example for Equal Frequency Discretization

Data values	V1	V2	V3	V4	V5	V6	V7	V8
Original Continuous Values	10	50	15	20	12	25	40	30
Sorted values	10	12	15	20	25	30	40	50
Description	Let k= 2 So each interval will contain 8/2=4 data values							
Discrete Values	1	2	1	1	1	2	2	2

Variable transformation

An attribute transform is a function that maps the entire set of values of a given attribute to a new set of replacement values such that each old value can be identified with one of the new values

Two types of variable transformation:

Simple functional transformation

Normalization or Standardization

Simple functional transformation is applied to each value individually

If x is a variable, then Simple Functions include:

e^x

x^k

logx

1/x

$\sqrt{x}$

sinx

The goal of Normalization or Standardization is to make an entire set of values have a particular property

Min-max normalization:

$$v' = \frac{v - min_A}{max_A - min_A}(new\_max_A - new\_min_A) + new\_min_A.$$

Min-max normalization:

Suppose that the minimum and maximum values for the attribute *income* are $12,000 and $98,000, respectively.

We would like to map *income* to the range [0.0,1.0].

By min-max normalization, a value of $73,600 for *income* is transformed to

$$\frac{73{,}600-12{,}000}{98{,}000-12{,}000}(1.0-0)+0=0.716.$$

z-score normalization (*zero-mean normalization*):

$$v' = \frac{v-\bar{A}}{\sigma_A},$$

Suppose that the mean and standard deviation of the values for the attribute *income* are $54,000 and $16,000, respectively. With z-score normalization, a value of $73,600 for *income* is transformed to $\frac{73{,}600-54{,}000}{16{,}000}=1.225.$

1.15 Measures of Similarity and dissimilarity

Term ***proximity*** is used to refer to either similarity or dissimilarity between the objects

Similarity

Similarity between two objects is a numerical measure of the degree to which the two objects are alike

Similarities are usually non-negative and are often between 0(no similarity) or 1(complete similarity)

Dissimilarity

Dissimilarity between two objects is a numerical measure of the degree to which the two objects are different

Term **distance** is used as a synonym for dissimilarity

Dissimilarity sometimes fall in the interval [0,1], but it is also common for them to range from 0 to ∞

Similarities and dissimilarities for simple attributes

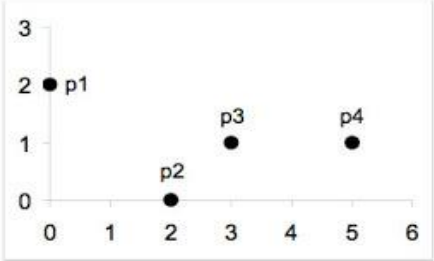
Attribute Type	Dissimilarity	Similarity
Nominal	$d = \begin{cases} 0 & \text{if } x = y \\ 1 & \text{if } x \neq y \end{cases}$	$s = \begin{cases} 1 & \text{if } x = y \\ 0 & \text{if } x \neq y \end{cases}$
Ordinal	$d = \lvert x-y\rvert/(n-1)$ (values mapped to integers 0 to $n-1$, where n is the number of values)	$s = 1-d$
Interval or Ratio	$d = \lvert x-y\rvert$	$s = -d$, $s = \frac{1}{1+d}$, $s = e^{-d}$, $s = 1-\frac{d-min\_d}{max\_d-min\_d}$

Dissimilarities between data objects

Euclidean distance:

Euclidean distance, d, between two points x and y, in one-, two-, three, or higher dimensional space is:

$$d(x,y) = \sqrt{\sum_{i=1}^{n}(x_i-y_i)^2}$$



point	x	y
p1	0	2
p2	2	0
p3	3	1
p4	5	1

	p1	p2	p3	p4
p1	0	2.828	3.162	5.099
p2	2.828	0	1.414	3.162
p3	3.162	1.414	0	2
p4	5.099	3.162	2	0

Distance Matrix

Euclidean distance equation is generalized by Minkowski distance:

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{k=1}^{n} |x_k - y_k|^r \right)^{1/r},$$

r is a parameter

Common examples of Minkowski distances:

r=1 City block/Manhattan, taxicab, L1 norm

r=2 Euclidean distance, L2 norm

r=∞ Supremum, Lmax or L∞ norm(maximum distance between any two attributes)

$$d(\mathbf{x}, \mathbf{y}) = \lim_{r \to \infty} \left(\sum_{k=1}^{n} |x_k - y_k|^r \right)^{1/r}.$$

Properties of Euclidean distance

If d(x,y) is the distance between two points, x and y, then the following properties hold:

1. Positivity:
   a) d(x,y) >=0 for all x and y
   b) d(x,y) =0 only if x=y
2. Symmetry:
   d(x,y)=d(y,x) for all x and y
3. Triangle Inequality:
   d(x,z)<=d(x,y) + d(y,z) for all points x,y,z

Similarity measure for binary data

Similarity measure between objects that contain only binary attributes are called similarity coefficients.

Let x and y be two objects that consists of n binary attributes

The comparison of two binary vector leads to the four quantities or frequencies:

f_{00} = the no of attributes where x is 0 and y is 0

f_{01} = the no of attributes where x is 0 and y is 1

f_{10} = the no of attributes where x is 1 and y is 0

f_{11} = the no of attributes where x is 1 and y is 1

Simple Matching Coefficient(SMC)

SMC is defined as:

$$SMC = \frac{\text{number of matching attribute values}}{\text{number of attributes}} = \frac{f_{11} + f_{00}}{f_{01} + f_{10} + f_{11} + f_{00}}.$$

Jaccard Coefficient

Suppose x and y are data objects that represent two rows of transaction matrix, then, If asymmetric binary attribute corresponds to an item in a store, then a 1 indicates that the item was purchased, while 0 indicates that the product was not purchased.

Jaccard Coefficient is symbolized by J, is given the following equation:

$$J= \frac{No\ of\ Matching\ presence}{no\ of\ attributes\ not\ involved\ in\ 00\ matches} = \frac{f11}{f01+f10+f11}$$

Calculate the SMC and J for the following two binary vectors:

x=(1,0,0,0,0,0,0,0,0,0)

y=(0,0,0,0,0,0,1,0,0,1)

f_{00} = 7 the no of attributes where x is 0 and y is 0

f_{01} = 2 the no of attributes where x is 0 and y is 1

f_{10} = 1 the no of attributes where x is 1 and y is 0

f_{11} = 0 the no of attributes where x is 1 and y is 1

$$SMC=\frac{f11+f00}{f01+f10+f11+f00} = \frac{0+7}{2+1+0+7}= 0.7$$

$$J= \frac{f11}{f01+f10+f11}= \frac{0}{2+1+0}= 0$$

Cosine Similarity

Documents are often represented as vectors, where each attribute represents the frequency with which a particular term(word) occurs in the document.

Cosine Similarity is one of the most common measure of document similarity

If x and y are two document vectors, then

$$\frac{x.y}{||x||\ ||y||}$$

. indicates vector dot product

x.y =$\sum_{k=1}^{n} xkyk$

||x|| is the length of vector x

||x||=$\sqrt{\sum_{k=1}^{n} x_k^2}$=$\sqrt{x.x}$

Calculate the cosine similarity for the following two data objects,

x=(3,2,0,5,0,0,0,2,0,0)

y=(1,0,0,0,0,0,0,1,0,2)

x.y=3*1+2*0+0*0+5*0+0*0+0*0+0*0+2*1+0*0+0*2=5

$$||x||= \sqrt{3*3+2*2+0*0+5*5+0*0+0*0+0*0+2*0+0*0+0*0}$$

=6.48

$$||y||=\sqrt{1*1+0*0+0*0+0*0+0*0+0*0+0*0+0*0+1*1+0*0+2*2}$$

=2.24

cos(x,y)= 0.31

MODULE 3
Association Analysis: Basic Concepts and Algorithms

1.1 Introduction

Many business enterprises accumulate large quantities of data from their day-to day operations, huge amounts of customer purchase data are collected daily at the checkout counters of grocery stores such data, commonly known as market basket transactions is as shown in Table 1.1.

TID	Items
1	{Bread, Milk}
2	{Bread, Diapers, Beer, Eggs}
3	{Milk, Diapers, Beer, Cola}
4	{Bread, Milk, Diapers, Beer}
5	{Bread, Milk, Diapers, Cola}

Table 1.1: Example of Market Basket transactions.

Each row in this table corresponds to a transaction, which contains a unique identifier labelled TID and a set of items bought by a given customer. Retailers are interested in analysing the data to learn about the purchasing behaviour of their customers. Such valuable information can be used to support a variety of business related applications such as marketing promotions, inventory management, and customer relationship management.

What is Association Analysis?

Association analysis is useful for discovering interesting relationships hidden in large amount of data. From Table 1.1 It is clear that the people who buy bread will also buy milk too.

{Bread} → {Milk}

1.2 Basic terminology used in association analysis

1.Binary Representation

Market basket data can be represented in a binary format where each row corresponds to a transaction and each column corresponds to an item.

An item can be treated as a binary variable whose value is one if the item is present in a transaction and zero otherwise

TID	bread	Milk	Diapers	Beer	Eggs	Cola
1	1	1	0	0	0	0
2	1	0	1	1	1	0
3	0	1	1	1	0	1
4	1	1	1	1	0	0
5	1	1	1	0	0	1

Table 1.2: Binary Representation of market based data

2. ItemSet: In association analysis, a collection of zero or more items is termed as -itemset.

For instance, {Beer, Diapers, Milk} is an example of a 3-itemset. The null (or empty) set is an itemset that does not contain any items.

3. Transaction Width: It is defined as the number of items present in a transaction. Important Property of ItemSet is: "Support and Count" which refers to the number of transactions that contain a particular itemset.
4. Association Rule: An association rule is an Implication expression of the form X→Y, where X and Y are disjoint itemsets. The strength of an association rule can be measured in terms of its support and confidence.
5. Support determines how often a rule is applicable to a given Data set
6. Confidence determines how frequently items in Y appear in transactions that contain X. Confidence, on the other hand, measures the reliability of the inference made by a rule.

$$\text{Support, } s(X \longrightarrow Y) = \frac{\sigma(X \cup Y)}{N};$$
$$\text{Confidence, } c(X \longrightarrow Y) = \frac{\sigma(X \cup Y)}{\sigma(X)}.$$

Consider the Association rule (Milk, Diapers)→(Beer).
From the above market basket data, the support count for (Milk, Diapers, Beer) is 2 and the total number of transactions is 5.
The rule's support is 2/5 = 0.4*100=40%
The rule's confidence is obtained by dividing the support count for {Milk. Diapers, Beer) by the support count for (Milk, Diapers).
Confidence: Since, there are 3 transactions that contain milk and diapers,
The confidence for this rule is 2/3 = 0.67*100=67%

1.3 Association rule discovery
Given a set of transactions T, the goal of association rule mining is to find all rules having

support ≥ minsup threshold

confidence ≥ minconf threshold

Apply the brute-force approach for mining association rules:

Brute force approach for mining association rule is to compute **support** and **confidence** for every possible rules

This approach is expensive because there are exponentially many rules that can be extracted from a data set.

The total number of possible rules extracted from a data set that contains D items is

$$R=3^d-2^{d+1}+1$$

Even for the small data set shown in Table 1.1, this approach requires us to compute the support and confidence for 36−27+ 1 = 602 rules. More than 80% of the rules are discarded after applying minsup = 20% and minconf = 50%, thus making most of the computations become wasted. To avoid performing needless computations, it would be useful to prune the rules early without having to compute their support and confidence values.

Commons strategy adapted by many association rule mining algorithm is to decompose the problem into two major approaches:

Frequent Itemset Generation
Rule Generation

Frequent Itemset Generation, whose objective is to find frequent itemsets. Frequent itemset is an itemsets whose support is greater than or equal to minsup threshold

Rule Generation, whose objective is to extract rules that satisfy both minsup and minconf. These rules are called strong rules.

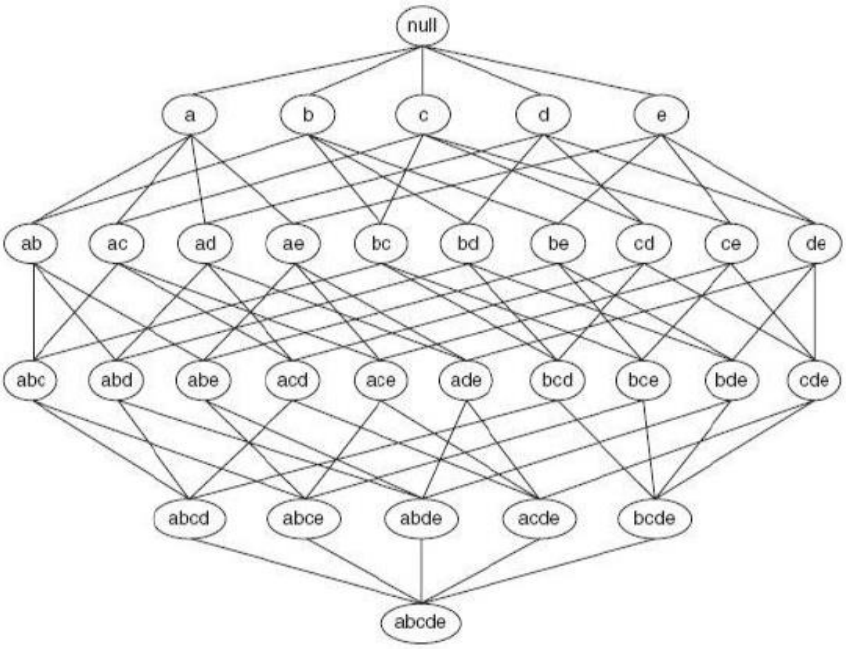
1.4 Frequent Itemset Generation

A lattice structure can be used to enumerate the list of all possible itemsets.

An itemset lattice for I = {a, b, c, d, e}. In general, a data set that contains k items can potentially generate up to 2^k-1 frequent itemsets, excluding the null set.

Because k can be very large in many practical applications, the search space of itemsets that need to be explored is exponentially large.

A Itemset lattice structure



A brute-force approach for finding frequent itemsets is to determine the support count for every candidate itemset in the lattice structure.

For example, the support for {Bread, Milk} is incremented three times because the itemset is contained in transactions 1, 4, and 5. Such an approach can be very expensive

There are several ways to reduce the computational complexity of frequent itemset generation:

→ Reduce the number of candidate itemset

The Apriori principle is an effective way to eliminate some of the candidate itemsets without counting their support values.

→ Reduce the number of comparisons

Instead of matching each candidate Itemset against every transaction, we can reduce the number of comparisons by using more advanced data structures, either to store the candidate itemsets or to compress the dataset

1.5 Apriori Principle

If an itemset is frequent, then all of its subsets must also be frequent.

Ex: suppose {c, d, e} is a frequent itemset. Clearly any transaction that contains {c, d, e} is a frequent item set. Clearly any transaction that contains {c, d, e} must also contain its subsets, {c, d}, {d, e}, {c}, {d}, and {e}.As a result, if {c, d, e} is frequent, then all subsets of {c, d, e} must also be frequent Figure 1.2.

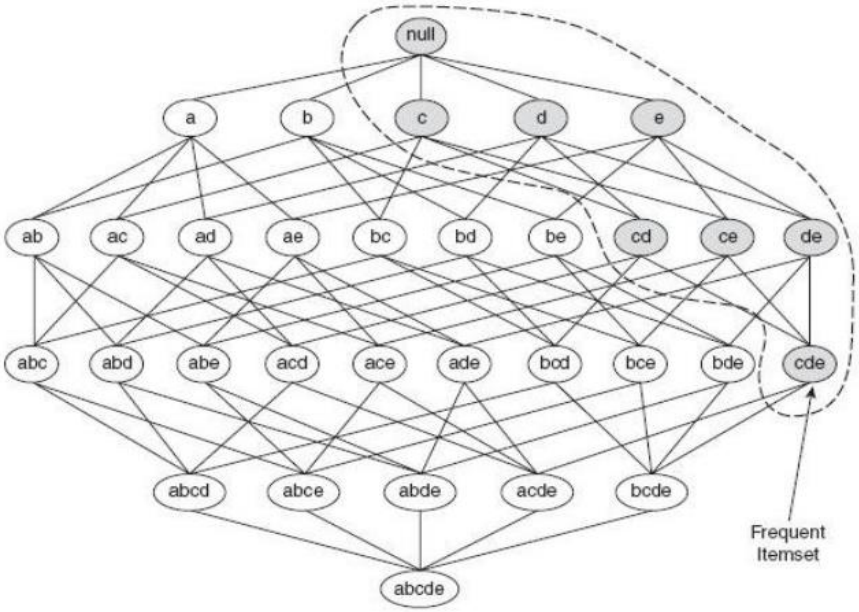


Figure 1.2: An illustration of the Apriori principle for frequent Itemset

Conversely if an items set such as {a, b} is infrequent then all of its supersets must also be infrequent too is shown in Figure 1.3

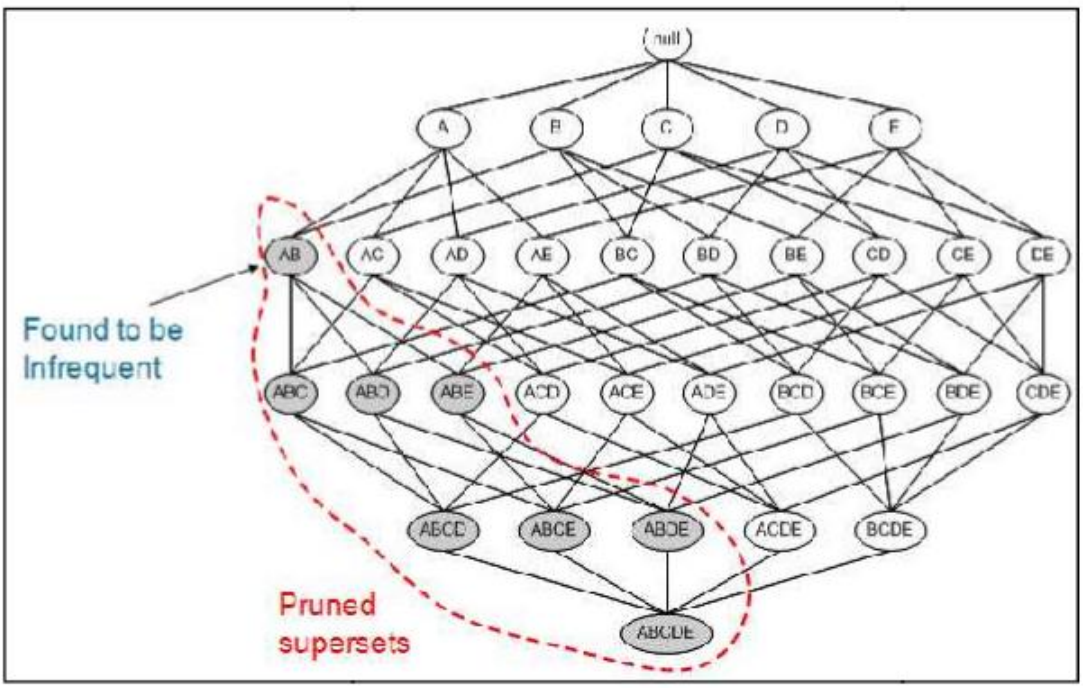


Figure 1.3: An illustration of the apriori principle for Infrequent Itemset

1.5.1 Frequent Itemset generation in Apriori Principle

Figure 1.4 provides a high-level illustration of the frequent itemset generation part of the Apriori algorithm for the transactions shown in Table 1.1.

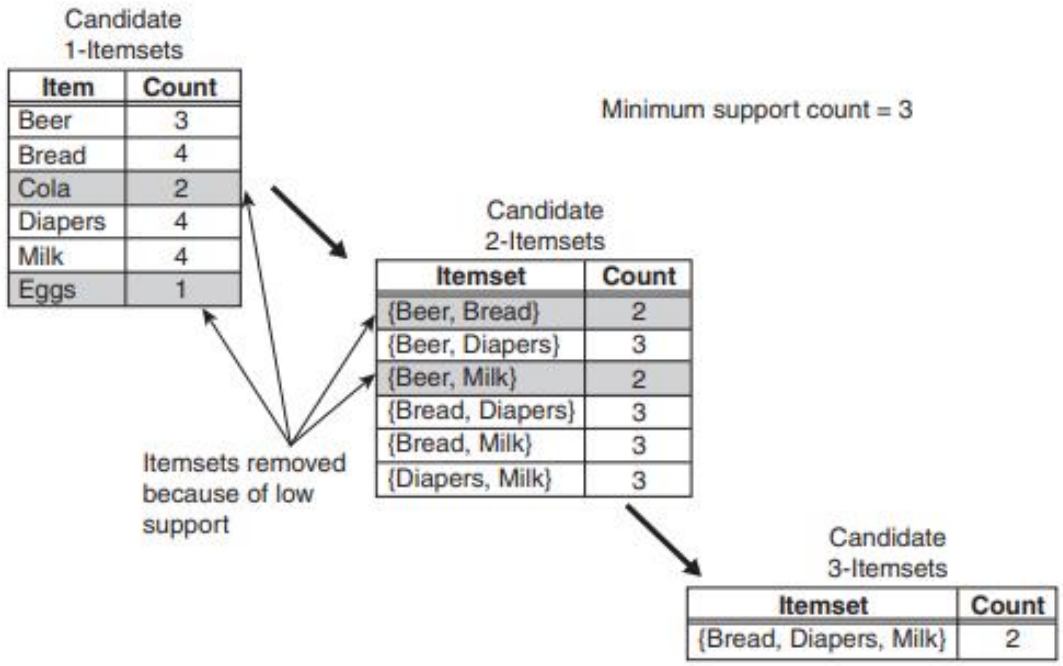


Figure 1.4: Illustration of frequent itemset generation using the Apriori algorithm

→We assume that the support threshold is 60%, which is equivalent to a minimum support count equal to 3.

→Initially, every item is considered as a candidate 1-itemset. After counting their supports, the candidate itemsets {Cola} and {Eggs} are discarded because they appear in fewer than three transactions.

→In the next iteration, candidate 2-itemsets are generated using only the frequent 1-itemsets

→Two of these six candidates, {Beer, Bread} and {Beer, Milk}, are subsequently found to be infrequent after computing their support values

→The remaining four candidates are frequent, and thus will be used to generate candidate 3-itemsets

→With the Apriori principle, we only need to keep candidate 3-itemsets whose subsets are frequent.

→The only candidate that has this property is {Bread, Diapers, Milk}. However, even though the subsets of {Bread, Diapers, Milk} are frequent, the itemset itself is not frequent.

Example 2:

Assume the support count is 2 and minimum confidence value is 60%. Generate the frequent itemset and strongs rules for the following transaction data

TID	Items
T1	1 3 4
T2	2 3 5
T3	1 2 3 5
T4	2 5
T5	1 3 5

C1

Itemset	Support
{1}	3
{2}	3
{3}	4
{4}	1
{5}	4

F1

Itemset	Support
{1}	3
{2}	3
{3}	4
{5}	4

Only Items present in F1

C2

Itemset	Support
{1,2}	1
{1,3}	3
{1,5}	2
{2,3}	2
{2,5}	3
{3,5}	3

F2

Itemset	Support
{1,3}	3
{1,5}	2
{2,3}	2
{2,5}	3
{3,5}	3

C3

Itemset	In F2?
{1,2,3}, {1,2}, {1,3}, {2,3}	NO
{1,2,5}, {1,2}, {1,5}, {2,5}	NO
{1,3,5},{1,5}, {1,3}, {3,5}	YES
{2,3,5}, {2,3}, {2,5}, {3,5}	YES

F3

Itemset	Support
{1,3,5}	2
{2,3,5}	2

Therefore, {1,3,5} and {2,3,5} are the frequent itemsets.

Rule Generation:

For I = {1,3,5}, subsets are {1,3}, {1,5}, {3,5}, {1}, {3}, {5}
For I = {2,3,5}, subsets are {2,3}, {2,5}, {3,5}, {2}, {3}, {5}

Considering {1,3,5}

1,3→ 5confidence= 2/3=66.6% (selected)
1,5→3 confidence=2/2=100% (selected)
3,5→1 confidence=2/3=66.6% (selected)

1→3,5 confidence=2/3=66.6% (selected)
3→1,5 confidence=2/4=50% (Rejected)
5→1,3 confidence=2/4=50% (Rejected)

Considering {2,3,5}

2,3→ 5confidence= 2/2=100% (selected)
2,5→3 confidence=2/3=66.6% (selected)
3,5→2 confidence=2/3=66.6% (selected)
2→3,5 confidence=2/3=66.6% (selected)
3→2,5 confidence=2/4=50% (Rejected)
5→2,3 confidence=2/4=50% (Rejected)

Pseudo code for Frequent Itemset generation of the Apriori Algorithm:

Method:

- Let k=1
- Generate frequent itemsets of length 1
- Repeat until no new frequent itemsets are identified
  - Generate length (k+1) candidate itemsets from length k frequent itemsets
  - Prune candidate itemsets containing subsets of length k that are infrequent
  - Count the support of each candidate by scanning the DB
  - Eliminate candidates that are infrequent, leaving only those that are frequent

Algorithm: Frequent itemset generation of the Apriori algorithm

```
k=1
F[1] = all frequent 1-itemsets
while F[k] != Empty Set:
    k += 1
    ## Candidate Itemsets Generation and Pruning
    C[k] = candidate itemsets generated by F[k-1]

    ## Support Counting
    for transaction t in T:
        C[t] = subset(C[k], t)     # Identify all candidates that belong to t
        for candidate itemset c in C[t]:
            support_count[c] += 1  # Increment support count

    F[k] = {c | c in C[k] and support_count[c]>=N*minsup}

return F[k] for all values of k
```

1.6 Candidate Generation and Candidate Pruning

1.6.1 Candidate Generation:

This operation generates new candidate k-itemset based on the frequent (k — 1)-itemsets found in the previous iteration.

Candidate Pruning: This operation eliminates some of the candidate k-itemsets using the support-based pruning strategy

Principles to generate candidate itemsets:

1. It should avoid generating too many unnecessary candidates.
2. It must ensure that the candidate set is complete i.e. no frequent items are left.

3. It should not generate the same candidate itemset more than once.

Several candidate generation procedures:

Brute force method
$F_{k-1} \times F_1$ Method
$F_{k-1} \times F_{k-1}$ Method

Brute-Force Method

The brute-force method considers every k-itemset as a potential candidate and then applies the candidate pruning step to remove any unnecessary candidates whose subsets are infrequent

Although candidate generation is rather trivial, candidate pruning becomes extremely expensive because a large number of itemsets must be examined.

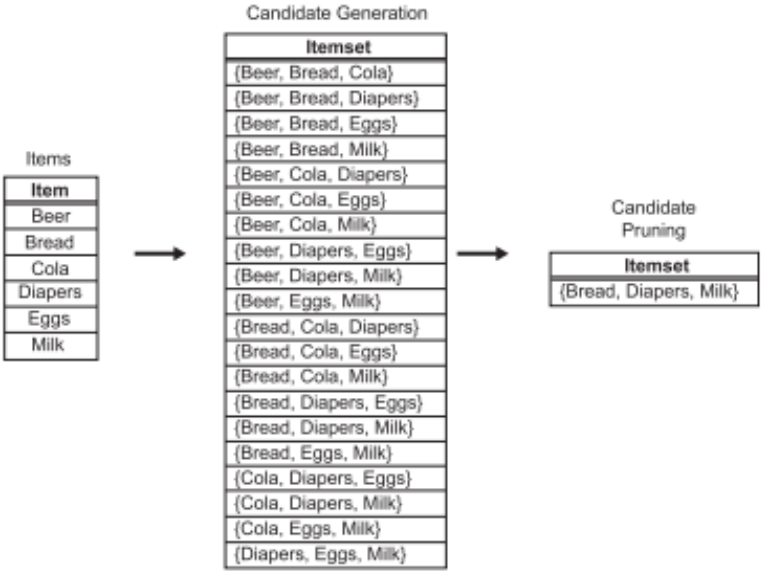


Figure 1.5: A brute-force method for generating candidate 3-itemsets.

$F_{k-1} \times F_1$ Method

An alternative method for candidate generation is to extend each frequent (k−1)-itemset with other frequent items.

→ Figure 1.6 illustrates how a frequent 2-itemset such as {Beer, Diapers} can be augmented with a frequent item such as Bread to produce a candidate 3-itemset {Beer, Diapers, Bread}.

→ Note that this approach does not prevent the same candidate itemset from being generated more than once.

→ For instance, {Bread, Diapers, Milk} can be generated by merging {Bread, Diapers} with {Milk}, {Bread, Milk} with {Diapers}, or {Diapers, Milk} with {Bread}.

→ One way to avoid generating duplicate candidates is by ensuring that the items in each frequent itemset are kept sorted in their lexicographic order.

→ For example, itemsets such as {Bread, Diapers}, {Bread, Diapers, Milk}, and {Diapers, Milk} follow lexicographic order as the items within every itemset are arranged alphabetically

→ Each frequent (k−1)-itemset X is then extended with frequent items that are lexicographically larger than the items in X. For example, the itemset {Bread, Diapers} can be augmented with {Milk} because Milk is lexicographically larger than Bread and Diapers.

→ However, we should not augment {Diapers, Milk} with {Bread} nor {Bread, Milk} with {Diapers} because they violate the lexicographic ordering condition

→ If the Fk−1 × F1 method is used in conjunction with lexicographic ordering, then only two candidate 3-itemsets will be produced in the example illustrated in Figure 1.6. {Beer, Bread, Diapers} and {Beer, Bread, Milk} will not be generated

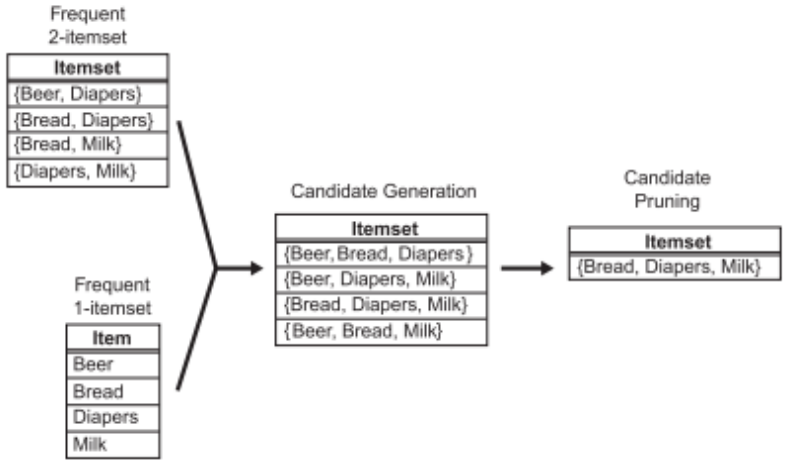


Figure 1.6: Generating and pruning candidate k-itemsets by merging a frequent (k−1)-itemset with a frequent item. Note that some of the candidates are unnecessary because their subsets are infrequent.

$F_{k-1} \times F_{k-1}$ Method

→ This candidate generation procedure merges a pair of frequent (k −1)-itemsets only if their first k −2 items, arranged in lexicographic order, are identical.

→ In Figure 1.7, the frequent itemsets {Bread, Diapers} and {Bread, Milk} are merged to form a candidate 3-itemset {Bread, Diapers, Milk}.

→ The algorithm does not have to merge {Beer, Diapers} with {Diapers, Milk} because the first item in both itemsets is different.

→ This example illustrates both the completeness of the candidate generation procedure and the advantages of using lexicographic ordering to prevent duplicate candidates.

→ Figure 1.7 shows that the Fk−1 × Fk−1 candidate generation procedure results in only one candidate 3-itemset.

→This is a considerable reduction from the four candidate 3-itemsets generated by the Fk−1 × F1 method.

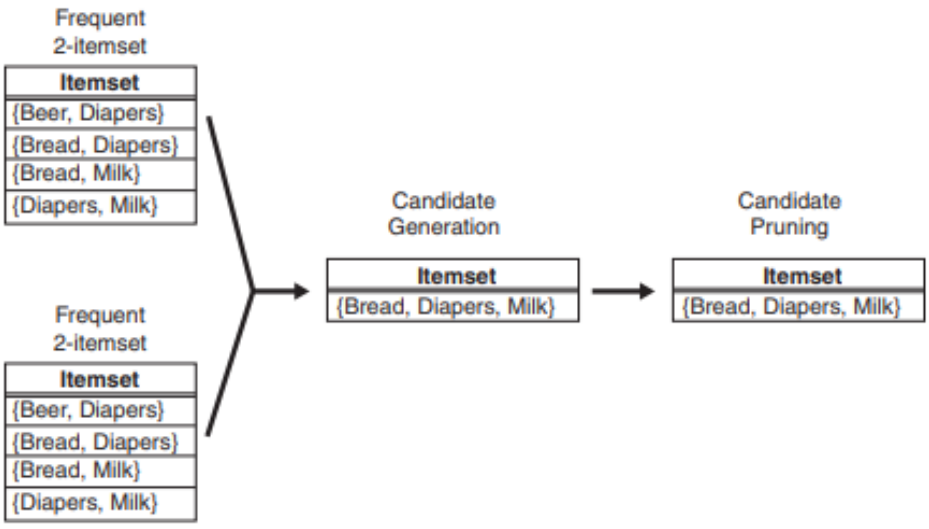


Figure 1.7: Generating and pruning candidate k-itemsets by merging pairs of frequent (k−1)-itemsets.

1.6.2 Support Counting

Support counting is the process of determining the frequency of occurrence for every candidate itemset that survives the candidate pruning step

→ One approach for doing this is to compare each transaction against every candidate itemset. This approach is computationally expensive, especially when the numbers of transactions and candidate itemsets are large.

→ An alternative approach is to enumerate the itemsets contained in each transaction and use them to update the support counts of their respective candidate itemsets.

→ To illustrate, consider a transaction t= {1,2,3,5,6}. There are 10 itemsets of size 3 contained in this transaction. Some of the itemsets may correspond to the candidate 3-itemsets under investigation, in which case, their support counts are incremented.

The Figure 3.8 below shows a systematic way for enumerating the 3-itemsets contained in t.

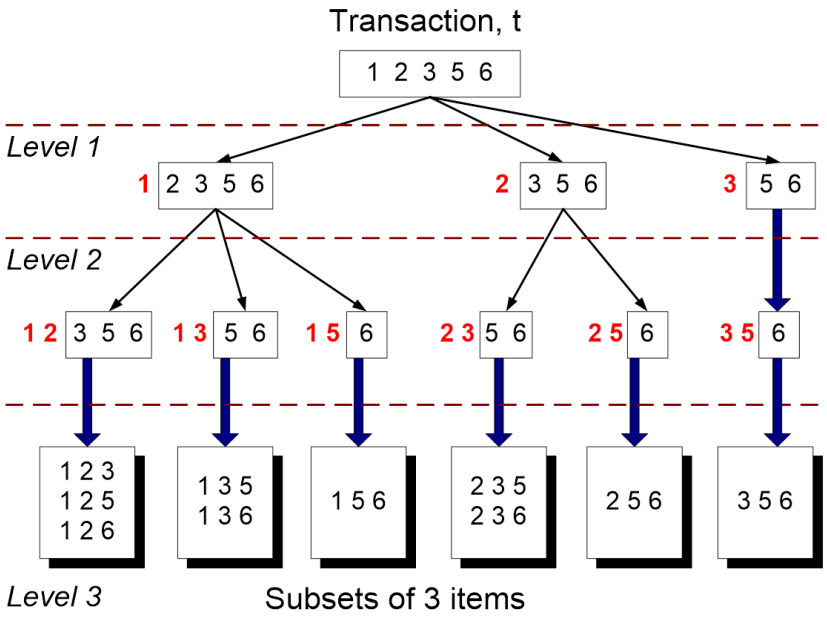


Figure 1.8: Enumerating subsets of three items from a transaction t.

→ Assuming that each itemset keeps its items in increasing lexicographic order, given t = {1,2,3,5,6}, all the 3-itemsets contained in t must begin with item 1, 2, or 3.

→ It is not possible to construct a 3-itemset that begins with items 5 or 6 because there are only two items in t whose labels are greater than or equal to 5.

→ 1 2356 represents a 3-itemset that begins with item 1, followed by two more items chosen from the set {2, 3, 5, 6}.

→ After fixing the first item, the prefix structures at Level 2 represent the number of ways to select the second item.

→ For example, 1 2 3 5 6 corresponds to itemsets that begin with prefix (1 2) and are followed by Items 3, 5, or 6.

→Finally, the prefix structures at Level 3 represent the complete set of 3 itemsets contained in t.

1.6.3 Support count using a hash tree

In the Apriori algorithm, candidate itemsets are partitioned into different buckets and stored in a hash tree. During support counting, itemsets contained in each transaction are also hashed into their appropriate buckets

→ That way, instead of comparing each itemset in the transaction with every candidate itemset, it is matched only against candidate itemsets that belong to the same bucket as shown in the figure 1.9

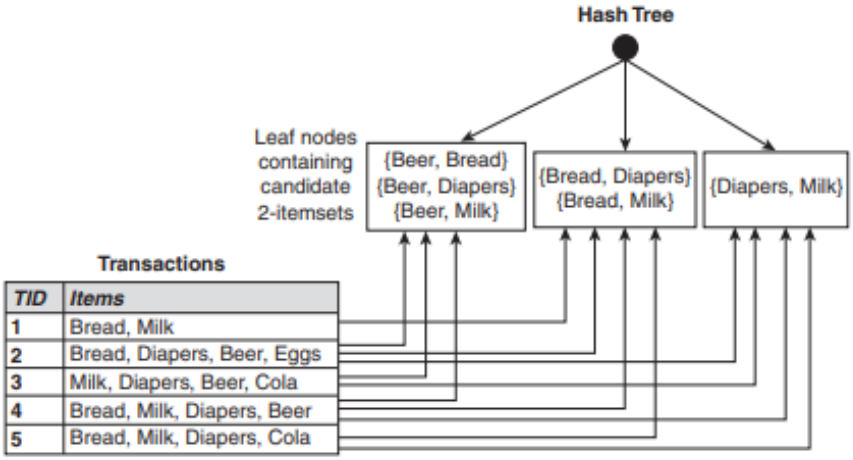


Figure 1.9: Counting the support of itemsets using hash structure.

Figure 1.10 shows an example of a hash tree structure.

Each internal node of the tree uses the following hash function, h(p)=(p − 1) mod 3, where mode refers to the modulo (remainder) operator, to determine which branch of the current node should be followed next.

For example, items 1, 4, and 7 are hashed to the same branch (i.e., the leftmost branch) because they have the same remainder after dividing the number by 3. All candidate itemsets are stored at the leaf nodes of the hash tree.

The hash tree shown in Figure 1.10 contains 15 candidate 3-itemsets, distributed across 9 leaf nodes.

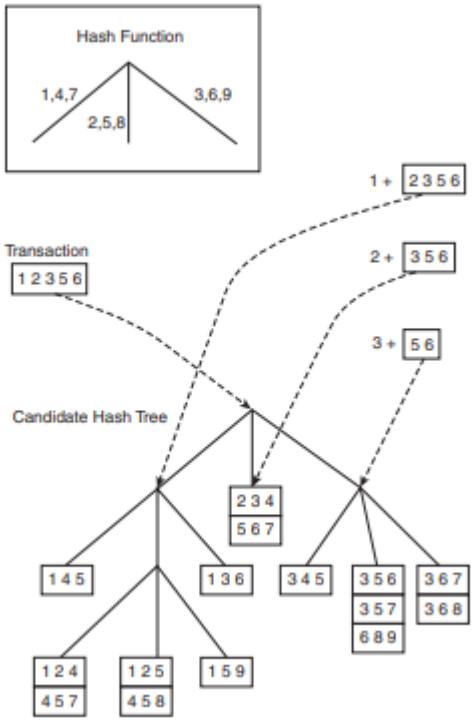


Figure 1.10: Hashing a transaction at the root node of a hash tree.

Consider the transaction, t = {1, 2, 3, 5, 6}. To update the support counts of the candidate itemsets, the hash tree must be traversed in such a way that all the leaf nodes containing candidate 3-itemsets belonging to t must be visited at least once.

At the root node of the hash tree, the items 1, 2, and 3 of the transaction are hashed separately. Item 1 is hashed to the left child of the root node, item 2 is hashed to the middle child, and item 3 is hashed

to the right child. At the next level of the tree, the transaction is hashed on the second item listed in the Level 2 tree structure shown in Figure 1.8.
For example, after hashing on item 1 at the root node, items 2, 3, and 5 of the transaction are hashed. Based on the hash function, items 2 and 5 are hashed to the middle child, while item 3 is hashed to the right child, as shown in Figure 1.11.
This process continues until the leaf nodes of the hash tree are reached. The candidate itemsets stored at the visited leaf nodes are compared against the transaction. If a candidate is a subset of the transaction, its support count is incremented. Note that not all the leaf nodes are visited while traversing the hash tree, which helps in reducing the computational cost. In this example, 5 out of the 9 leaf nodes are visited and 9 out of the 15 itemsets are compared against the transaction.

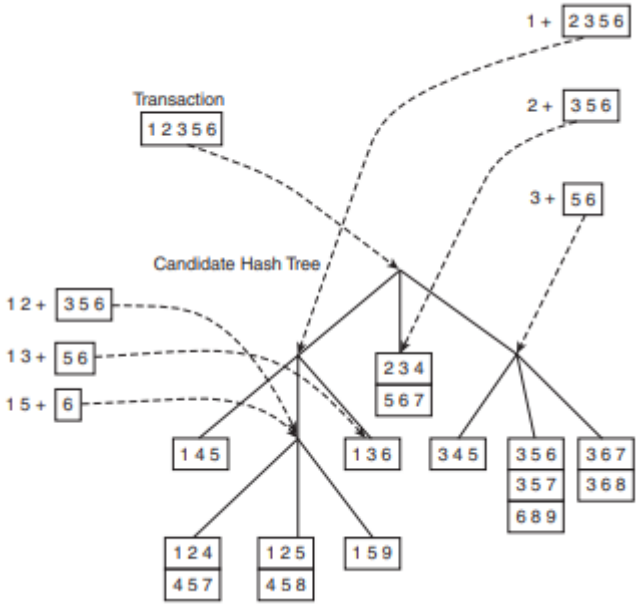


Figure 1.11: Subset operation on the leftmost subtree of the root of a candidate hash tree

1.7 Computational Complexity

The Computational complexity of the Apriory algorithm can be affected by the following factors:
Support Threshold: Lowering the support threshold often results in more itemsets being declared as frequent. This has an adverse effect on the computational complexity of the algorithm because more candidate itemsets must be generated and counted.

Number of Items (Dimensionality): As the number of items increases, more space will be needed to store the support counts of items. If the number of frequent items also grows with the dimensionality of the data, the computation and I/O costs will increase because of the larger number of candidate itemsets generated by the algorithm

Number of Transactions: Since the Apriori algorithm makes repeated passes over the data set, its run time increases with a larger number of transaction

Average Transaction Width: For dense data sets, the average transaction width can be very large. This affects the complexity of the Apriori algorithm in two ways.

- o First, the maximum size of frequent itemsets tends to increase as the average transaction width increases.
- o Second as the transaction width increases, more itemsets are contained in the transaction

Generation of frequent 1-itemsets: For each transaction, we need to update the support count for every item present in the transaction. Assuming that w is the average transaction width, this operation requires O(Nw) time, where N is the total number of transactions

Candidate generation: To generate candidate itemsets, pairs of frequent (k — l) itemsets arc merged to determine whether they have at least k - 2 items in common. Each merging operation requires at most k — 2 equality comparisons. In the best-case scenario, every merging step produces a viable candidate A-itemset. In the worst-case scenario, the algorithm must merge every pair of frequent (k-l)-itemsets found in the previous iteration

Support counting:

The cost for support counting is

$$O\left(N \sum_k \binom{w}{k} \alpha_k\right)$$

where w is the maximum transaction width
αk is the cost for updating the support count of a candidate k-itemset in the hash tree.

1.8 Rule Generation

An association rule can be extracted by partitioning the item set Y into two non-empty subsets, X and Y — X, such that X →Y — X satisfies the confidence threshold.

Note that all such rules must have already met the support threshold because they are generated from a frequent itemset.
Example:
Let X = {1,2,3} be a frequent itemset. There are six candidate association rules that can be generated from X: {1,2} →{3}, {1,3} → {2}, {2,3}→ {1}, {1} → {2,3}, {2}→{1,3}, and {3}→ {1,2}.
As each of their support is identical to the support for X, the rules must satisfy the support threshold.

1.8.1 Confidence-Based Pruning

Theorem

Let Y be an itemset and X is a subset of Y.

If a rule X → Y − X does not satisfy the confidence threshold, then any rule X' → Y − X', where X' is a subset of X, must not satisfy the confidence threshold as well.

Rule Generation in Apriori Algorithm

Initially, all the high-confidence rules that have only one item in the rule consequent are extracted. These rules are then used to generate new candidate rules.
For example, if {acd} —► {b} and {abd} —► {c} are high-confidence rules, then the candidate rule {ad} —► {bc} is generated by merging the consequents of both rules
If any node in the lattice has low confidence, then according to confidence based pruning Theorem, the entire subgraph spanned by the node can be pruned immediately.

Suppose the confidence for {bcd} —► {a} is low. All the rules containing item a in its consequent, including {cd} →{ab}, {bd} —► {ac}, {bc} —> {ad} and {d} → {abc} can be discarded is as shown in the figure 1.12

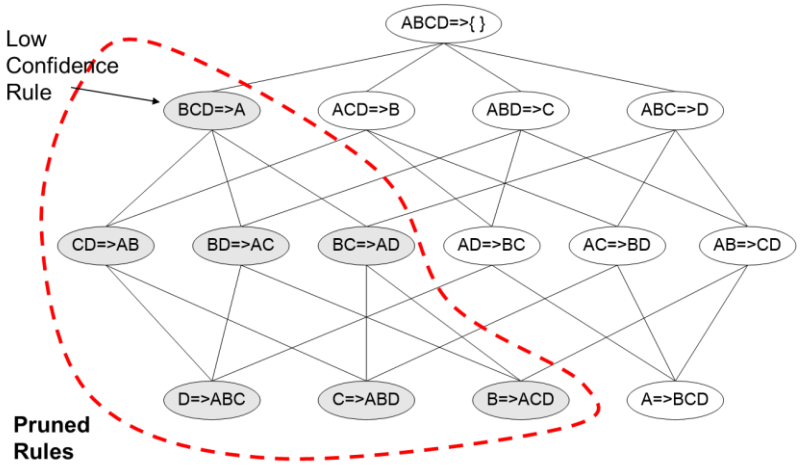


Figure 1.12: Pruning of Association rules using the confidence measure

1.9 Compact Representation of Frequent Itemset

In practice the number of frequent itemsets produced from a transaction data set that can be very large.It is useful to identify a small representative set of itemsets from which all other frequent itemsets can be derived.

Two approaches are

Maximal Frequent Itemset

Closed frequent itemsets.

Maximal Frequent Itemset

A maximum frequent itemset is defined as a frequent itemset for which none of its immediate supersets are frequent

To illustrate this concept, consider the itemset lattice shown in Figure 1.13

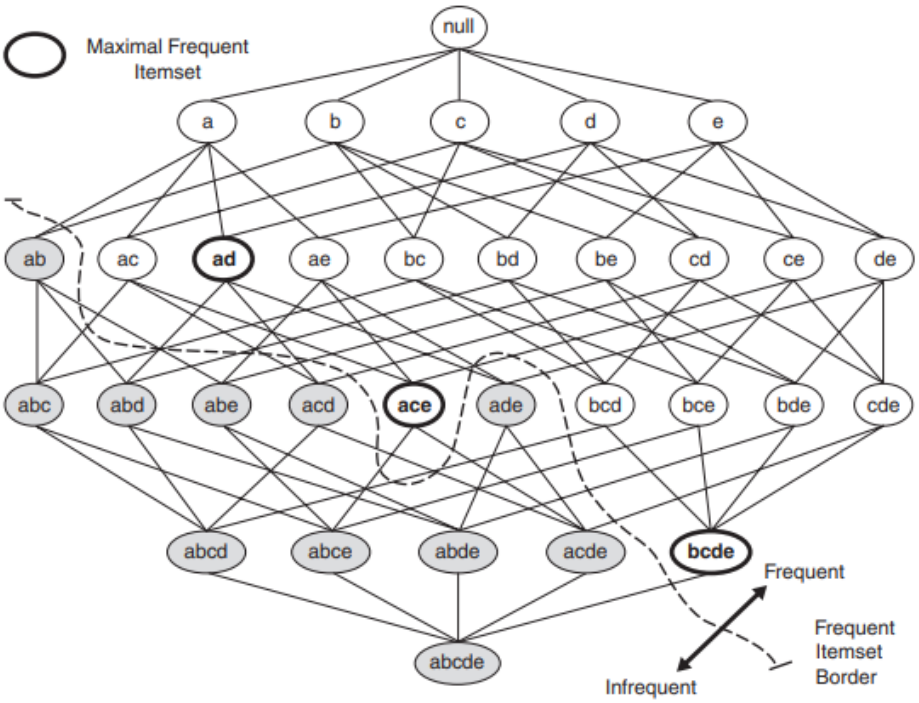


Figure 1.13: Maximal frequent itemset

The itemsets in the lattice are divided into two groups: those that are frequent and those that are infrequent. A frequent itemset border, which is represented by a dashed line, is also illustrated in the diagram. Every itemset located above the border is frequent, while those located below the border (the shaded nodes) are infrequent. Among the itemsets residing near the border, {a, d},{a, c, e}, and{b, c, d, e}are considered to be maximal frequent itemsets because their immediate supersets are infrequent. An itemset such as {a, d}is maximal frequent because all of its immediate supersets,{a, b, d},{a, c, d}and {a, d, e}, are infrequent. In contrast, {a, c}is non maximal because one of its immediate supersets {a, c, e}, is frequent.

Closed Frequent Itemsets

Closed Itemset: An itemset X is closed if none of its immediate supersets has exactly the same support count as X.

For example, since the node {b, c} is associated with transaction IDs 1, 2. and 3, its support count is equal to three. From the transactions given in this diagram, notice that every transaction that contains b also contains c.

Consequently, the support for {b} is identical to {b, c] and {b} should not be considered a closed itemset. Similarly, since c occurs in every transaction that contains both a and d, the itemset. {a, d} is not closed.

On the other hand,{ b, c} is a closed itemset because it does not have the same support count as any of its supersets is as shown in figure 1.14

Closed Frequent Itemset: An itemset is a closed frequent itemset if it is closed and its support is greater than or equal to minsup

Ex: {bc} is a closed frequent itemset because its support is 60%. Assuming that the support threshold is 40%. {b,c} is a closed frequent itemset because its support is 60%. The rest of the closed frequent itemsets are indicated by the shaded nodes

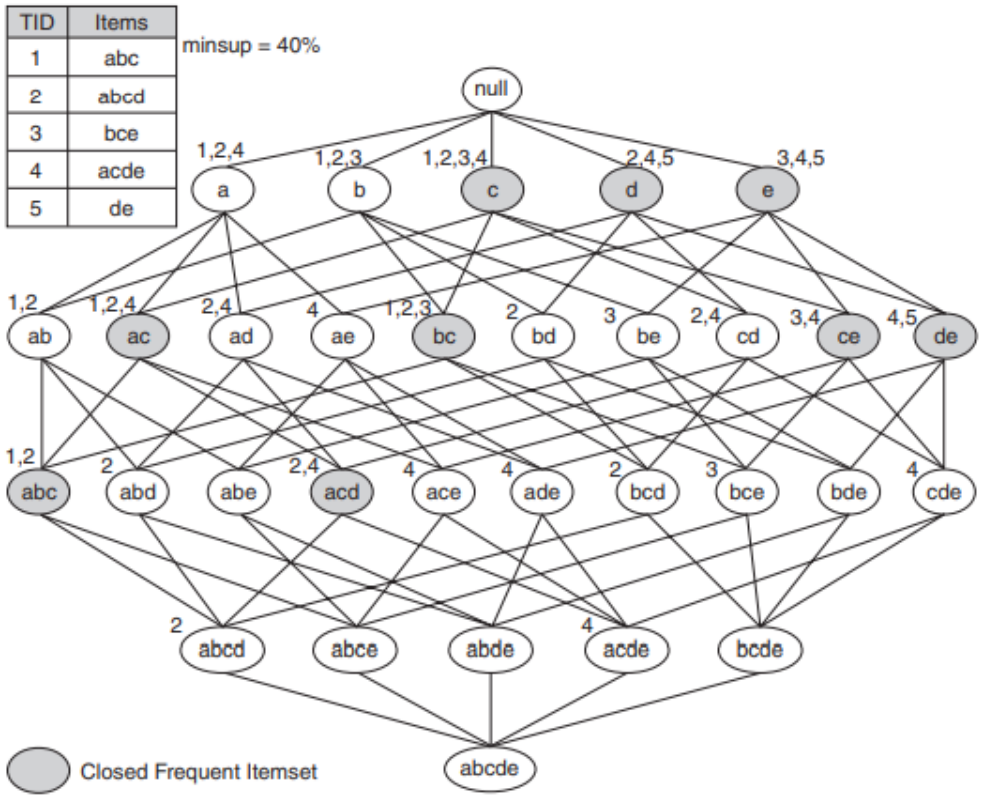


Figure 1.14: Closed Frequent itemset

1.10 Alternative Methods for Generating Frequent Itemsets

Several alternative methods have been developed to overcome these limitations and improve upon the efficiency of the Apriori algorithm.

Traversal of itemset lattice:

A search for frequent itemset can be conceptually viewed as a traversal on the itemset lattice. Search algorithm decides how the lattice structure is traversed during the frequent itemset generation process. Some search strategies are:

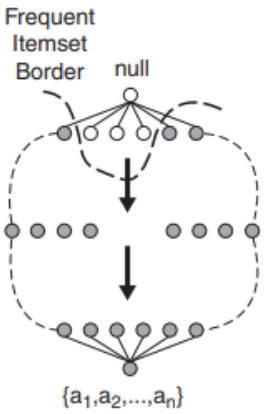
- General to specific vs specific to general
- Equivalence Classes
- Breadth-First versus Depth-First

General-to-Specific versus Specific-to-General

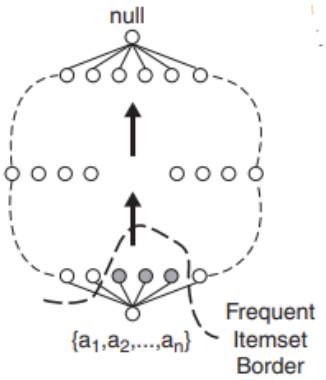
This general-to-specific search strategy is effective, provided the maximum length of a frequent itemset is not too long. The configuration of frequent item-sets that works best with this strategy is shown in Figure 1.15(a), where the darker nodes represent infrequent itemsets.

Alternatively, a specific-to-general search strategy looks for more specific frequent itemsets first,before finding the more general frequent itemsets. This strategy is use-ful to discover maximal frequent itemsets in dense transactions, where the frequent itemset border is located near the bottom of the lattice,as shown in Figure 1.15(b).

Another approach is to combine both general-to-specific and specific-to-general search strategies. This bidirectional approach requires more space to store the candidate itemsets, but it can help to rapidly identify the frequent itemset border.



(a) General-to-specific



(b) Specific-to-general

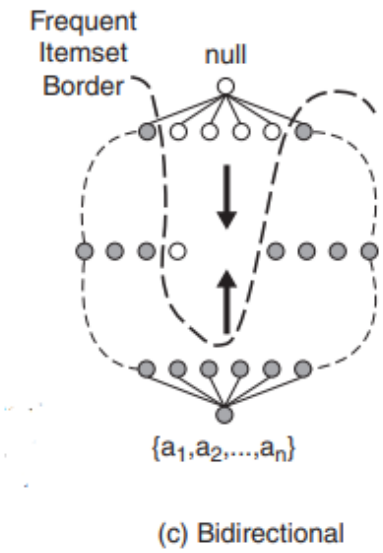


Figure 1.15: General to Specific, Specific to general and Bidirectional search

Equivalence classes

Another way is to first partition the lattice into disjoint groups of nodes (or equivalence classes).
A frequent itemset generation algorithm searches for frequent itemsets within a particular equivalence class first before moving to another equivalence class.
Equivalence classes can also be defined according to the prefix or suffix labels of an itemset.
In this case, two itemsets belong to the same equivalence class if theyshare a common prefix or suffix of length k . In the prefix-based approach,the algorithm can search for frequent itemsets starting with the prefix a before looking for those starting with prefixes b, c, and so on. Both prefix-based and suffix-based equivalence classes can be demonstrated using the tree-like structure shown in Figure 1.16

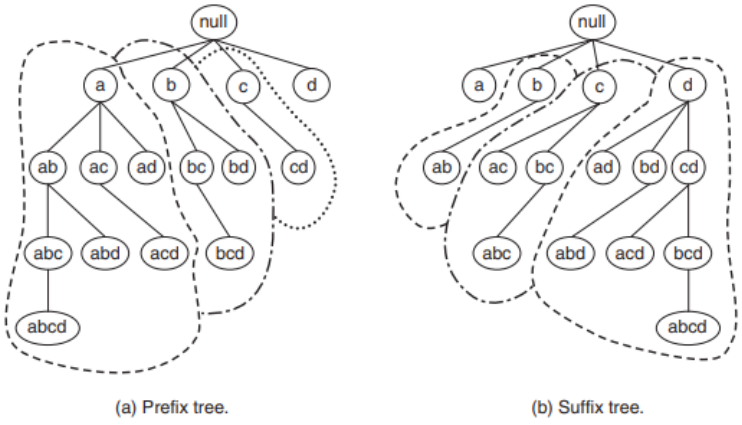


Figure 1.16: Equivalence classes based on the prefix and suffix labels on itemsets

BSF and DFS (Breadth first search and Depth first search)

The Apriori algorithm traverses the lattice in a breadth-first manner, as shown in Figure 1.17(a).
It first discovers all the frequent 1-itemsets, followed by the frequent 2-itemsets, and so on, until no new frequent itemsets are generated.
The lattice can also be traversed in a depth-first manner, as shown in Figures 1.17(b).

The algorithm progressively expands the next level of nodes, i.e., ab, abc, and so on, until an infrequent node is reached, say, abcd . It then backtracks to another branch, say, abce , and continues the search from there.
This approach allows the frequent itemset border to be detected more quickly than using a breadth-first approach

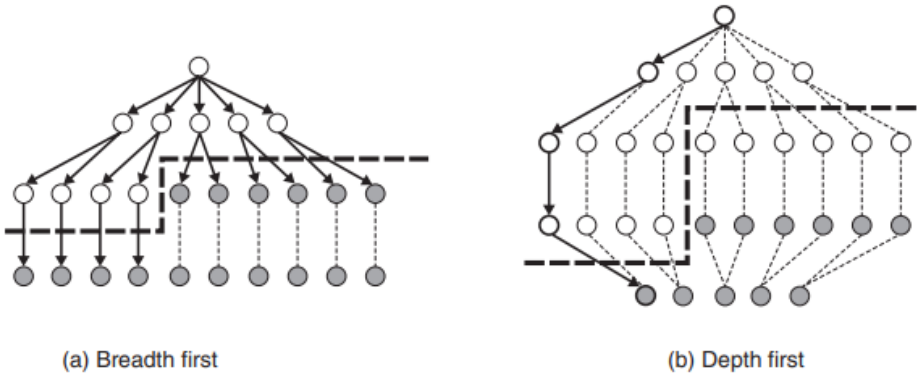


Figure 1.17: Representation of BFS and DFS

1.11 FP Growth Algorithm

FP-growth that takes a radically different approach to discovering frequent itemsets, it encodes the data set using a compact data structure called an FP-tree and extracts frequent itemsets directly from this structure

Apriori vs FP Growth Algorithm

Algorithm	Technique	Runtime	Memory usage	Parallelizability
Apriori	Generate singletons, pairs, triplets, etc.	Candidate generation is extremely slow. Runtime increases exponentially depending on the number of different items.	Saves singletons, pairs, triplets, etc.	Candidate generation is very parallelizable
FP-Growth	Insert sorted items by frequency into a pattern tree	Runtime increases linearly, depending on the number of transactions and items	Stores a compact version of the database.	Data are very inter dependent, each node needs the root.

FP-Tree Representation

An FP-tree is a compressed representation of the input data. It is constructed by reading the data set one transaction at a time and mapping each transaction onto a path in the FP -tree. As different transactions can have several items in common, their paths may overlap.
The more the paths overlap with one another, the more compression we can achieve using the FP-tree structure.

FP Growth Algorithm is divided into 5 simple steps as shown below:

1. The first step is to count all the items in whole transaction to get support count
2. Next we apply the threshold(minsup) we had previously set and remove items which poses support count less than minsup
3. Now we start the list of items with support count greater than minsup in descending order

4. Now, we build the FP tree, we go through each of the transaction and add all the items in the order they appear in our sorted list
5. In order to get the association now we go through every branch of the tree and only include in the association all the nodes whose count passed the threshold(minsup)

Consider the following example:

TID	List of Items
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

where minsup=2

Step 1: Count all the items in whole transaction to get support count

Itemset	Support count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

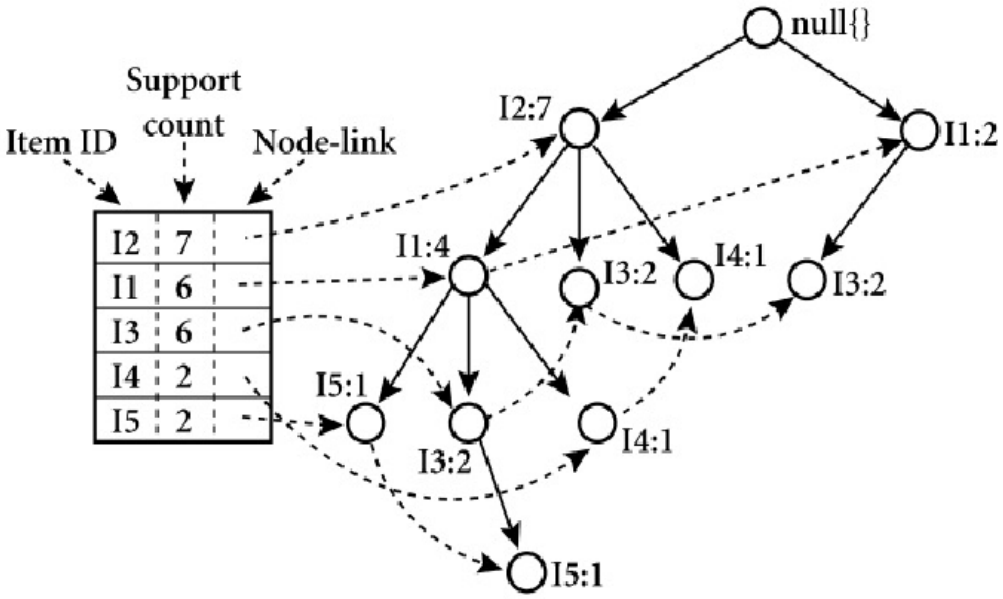
Step 2: Apply the threshold(minsup) and remove items which poses support count less than minsup. In the above table all items support count is greater than minsup=2. So all items are considered in FP tree.

Step 3: Arrange item in descending order of the support count

Itemset	Support count
{I2}	7
{I1}	6
{I3}	6
{I4}	2
{I5}	2

Step 4: FP tree construction, we go through each of transaction with following two condition:

* items whose support count is less than minsup should be removed from transaction
* Descending order item list for FP tree construction should be maintained.



Item	Conditional Pattern Base	Conditional FP- tree	Frequent Pattern Generated
I5	{{I2, I1: 1}, {I2, I1, I3: 1}}	<I2:2, I1:2>	{I2, I5: 2}, {I1,I5: 2}, {I2, I1, I5: 2}
I4	{{I2, I1: 1}, {I2: 1}}	<I2:2>	{I2, I4: 2}
I3	{{I2, I1: 2}, {I2: 2}, {I1: 2}}	<I2:4, I1:2>, <I1:2>	{I2, I3: 4}, {I1,I3: 4}, {I1, I2, I3: 2}
I1	{I2: 4}	<I2:4>	{I2, I1: 4}
I2	---	---	---

Frequent Itemset Generation in FP- Growth Algorithm

Consider the following example:

TID	Itemset
100	{ M, O, N, K, E, Y }
200	{ D, O, N, K, E, Y }
300	{ M, A, K, E }
400	{ M, U, C, K, Y }
500	{ C, O, K, I, E }

Minsup=3

Step1: Find the support count of every item in the transaction and remove the item having low support count < 3.In below example item D, A, U, N, C,T is having low support count so discard those item for next step and arrange the element in highest support count order.

item	Support count
M	3
O	3
N	2
K	5
E	4
Y	3
D	1
A	1
U	1
C	2
I	1

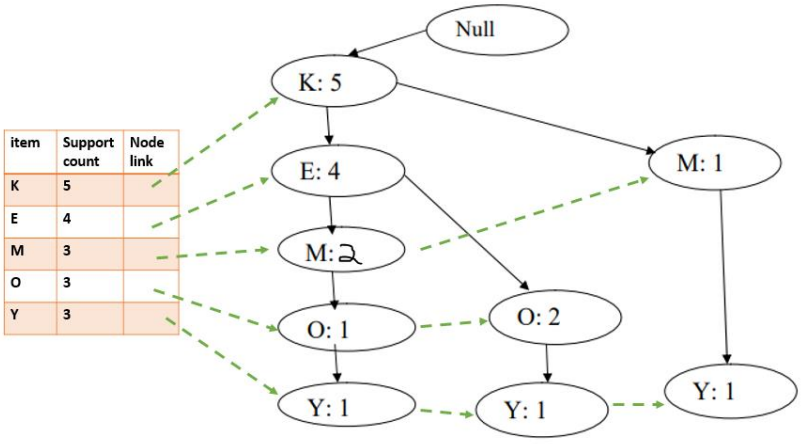
item	Support count
M	3
O	3
K	5
E	4
Y	3

item	Support count
K	5
E	4
M	3
O	3
Y	3

Step 2: Consider the above frequent pattern i.e. “KEMOY” and compare the pattern with original data set then generate the ordered itemsets

TID	Itemset	Ordered itemset
100	{ M, O, N, K, E, Y }	{ K, E, M, O, Y }
200	{ D, O, N, K, E, Y }	{ K ,E ,O ,Y }
300	{ M, A, K, E }	{ K ,E ,M }
400	{ M, U, C, K, Y }	{ K ,M ,Y }
500	{ C, O, K, I, E }	{ K ,E ,O }

Step 3: Construct the FP Tree for above Ordered Itemsets



Step 4: Arrange the item in reverse order then find the frequent itemsets ending in Y, the algorithm proceeds to look for frequent item sets in O. this process continues until all the paths are processed and construct the conditional FP Tree by counting the repeated item in conditional pattern base

Item	Conditional Pattern Base	Conditional FP- tree	Frequent Pattern Generated
Y	{{KEMO:1}, {KEO:1}, {KM:1}}	<K:3>	{K,Y:3}
O	{{KEM:1}, {KE:2}}	<K:3, E:3>	{K,O:3}, {E,O:3}, {K,E,O:3}

M	{{KE:2}, {K:1}}	<K:3>	{K,M:3}
E	{K:4}	<K:4>	{K,E:4}
K	---	---	---

1.12 Evaluation of Association Pattern

The Association analysis algorithm have the potential to generate the large number of patterns, we could easily end up with thousands or even millions of patterns, many of which may not be interesting.

Different types of association pattern evaluation criteria:
Support- Confidence framework
Interest factor
Correlation Analysis
IS Measure

Consider the following Example:
Following table shows an example of contingency table for a pair of binary variable A and B.
Each entry f_{ij} in this 2 X 2 table denotes a frequency count.
Example, f_{11} is the number of times A and B appear together in the same transaction, while f_{01} is the number of transaction that contain B but not A. The row sum f_{1+} represents the support count for A, while the column sum f_{+1} represents the support count for B.

Table 5.6. A 2-way contingency table for variables A and B.

	B	$\overline{B}$	
A	f_{11}	f_{10}	f_{1+}
$\overline{A}$	f_{01}	f_{00}	f_{0+}
	f_{+1}	f_{+0}	N

Limitation of Support confidence framework:
→ Existing association rule mining formulation relies on the support and confidence measure to eliminate uninteresting patterns.
→ the drawback of support is that potentially interesting patterns involving low support items might be eliminated by the support threshold.
The drawback of confidence is demonstrated using the following example.

Suppose we are interested in analyzing the relationship between people who drink tea and coffee

Table 5.7. Beverage preferences among a group of 1000 people.

	$Coffee$	$\overline{Coffee}$	
Tea	150	50	200
$\overline{Tea}$	650	150	800
	800	200	1000

To evaluate the association rule
{Tea} →{Coffee}

It may appear that people who drink tea also tend to drink coffee because the rule's support (15%) and confidence (75%) values are reasonably high
The fraction of people who drink coffee, regardless of whether they drink tea, is 80%, while the fraction of tea drinkers who drink coffee is only 75%.
The rule {Tea} → {Coffee} is therefore misleading despite its high confidence value.

Example 2:

Table 5.8. Information about people who drink tea and people who use honey in their beverage.

	$Honey$	$\overline{Honey}$	
Tea	100	100	200
$\overline{Tea}$	20	780	800
	120	880	1000

If we evaluate the association rule {Tea} → {Honey}
Confidence value of this rule is merely 50%, which might be easily rejected

Interest Factor
The interest factor, which is also called as the "lift," can be defined as follows:

$$I(A,B) = \frac{s(A,B)}{s(A) \times s(B)} = \frac{Nf_{11}}{f_{1+}f_{+1}}.$$

$$I(A,B) \begin{cases} = 1, & \text{if } A \text{ and } B \text{ are independent;} \\ > 1, & \text{if } A \text{ and } B \text{ are positively related;} \\ < 1, & \text{if } A \text{ and } B \text{ are negatively related.} \end{cases}$$

For tea-coffee,

$$I = \frac{0.15}{0.2 \times 0.8} = 0.9375,$$

Suggesting a slight negative relationship between tea drinkers and coffee drinkers

For the tea-honey example,

$$I = \frac{0.1}{0.12 \times 0.2} = 4.1667,$$

Suggesting a strong positive relationship between people who drink tea and people who use honey in their beverage.

Correlation Analysis
Correlation analysis is one of the most popular techniques for analyzing relationships between a pair of variables

$$\phi = \frac{f_{11}f_{00} - f_{01}f_{10}}{\sqrt{f_{1+}f_{+1}f_{0+}f_{+0}}}.$$

Value of the φ-coefficient ranges from −1 to +1.
A correlation value of 0 means no relationship

while a value of +1 suggests a perfect positive relationship and
value of −1 suggests a perfect negative relationship.

tea and coffee drinkers example,
−0.0625, which is slightly less than 0.
On the other hand, the correlation between people who drink tea and people who use honey is 0.5847, suggesting a positive relationship.

IS Measure

$$IS(A,B) = \sqrt{I(A,B) \times s(A,B)} = \frac{s(A,B)}{\sqrt{s(A)s(B)}} = \frac{f_{11}}{\sqrt{f_{1+}f_{+1}}}.$$

The value of IS thus varies from 0 to 1, where an IS value of 0 corresponds to no co-occurrence of the two variables, while an IS value of 1 denotes perfect relationship

For the tea-coffee example, the value of IS is equal to 0.375,
while the value of IS for the tea-honey example is 0.6455.
The IS measure thus gives a higher value for the {Tea}→ {Honey} rule than the {Tea}→ {Coffee} rule,

Table 5.9. Examples of objective measures for the itemset $\{A, B\}$.

Measure (Symbol)	Definition
Correlation (ϕ)	$\frac{Nf_{11}-f_{1+}f_{+1}}{\sqrt{f_{1+}f_{+1}f_{0+}f_{+0}}}$
Odds ratio (α)	$(f_{11}f_{00})/(f_{10}f_{01})$
Kappa (κ)	$\frac{Nf_{11}+Nf_{00}-f_{1+}f_{+1}-f_{0+}f_{+0}}{N^2-f_{1+}f_{+1}-f_{0+}f_{+0}}$
Interest (I)	$(Nf_{11})/(f_{1+}f_{+1})$
Cosine (IS)	$(f_{11})/(\sqrt{f_{1+}f_{+1}})$
Piatetsky-Shapiro (PS)	$\frac{f_{11}}{N} - \frac{f_{1+}f_{+1}}{N^2}$
Collective strength (S)	$\frac{f_{11}+f_{00}}{f_{1+}f_{+1}+f_{0+}f_{+0}} \times \frac{N-f_{1+}f_{+1}-f_{0+}f_{+0}}{N-f_{11}-f_{00}}$
Jaccard (ζ)	$f_{11}/(f_{1+}+f_{+1}-f_{11})$
All-confidence (h)	$\min\left[\frac{f_{11}}{f_{1+}}, \frac{f_{11}}{f_{+1}}\right]$

MODULE 4

Classification

Classification is the task of assigning objects to one of several predefined categories
Example:
→Determining spam email messages based upon the message header and content
→Classifying galaxies based upon their shapes
→A bank loan officer wants to analyze the data in order to know which customer are risky or safe (loan applicant)
→A marketing manager at a company needs to analyze a customer with a given profile, who will buy a computer or not

4.1 Classification- Definition

Classification is a task of learning a *target function* f that maps each attribute set x to one of the predefined class labels y

Figure 4.2. Classification as the task of mapping an input attribute set $\mathbf{x}$ into its class label y.

	categorical	categorical	continuous	class
Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

One of the attributes is the class attribute
In this case: Cheat

Two class labels (or classes): Yes (1), No (0)

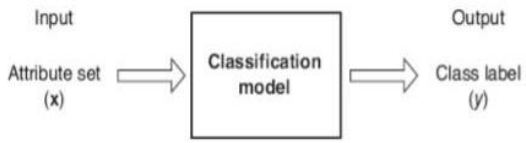


Figure 4.2. Classification as the task of mapping an input attribute set $\mathbf{x}$ into its class label y.

A classification model is useful for the following purposes:

- Descriptive modeling
- Predictive Modeling

Descriptive modeling

A classification model can serve as an explanatory tool to distinguish between objects of different classes

Name	Body Temperature	Skin Cover	Gives Birth	Aquatic Creature	Aerial Creature	Has Legs	Hibernates	Class
Human	Warm	hair	yes	no	no	yes	no	Mammal
Python	Cold	scales	no	no	no	no	yes	Reptile
Salmon	Cold	scales	no	yes	no	no	no	Fish
Whale	Warm	hair	yes	yes	no	no	no	Mammal
Frog	Cold	none	no	semi	no	yes	yes	Amphibian
Komodo	Cold	scales	no	no	no	yes	no	Reptile
Bat	Warm	hair	yes	no	yes	yes	yes	Mammal
Pigeon	Warm	feathers	no	no	yes	yes	no	Bird
Cat	Warm	fur	yes	no	no	yes	no	Mammal
Leopard	Cold	scales	yes	yes	no	no	no	Fish
Turtle	Cold	scales	no	semi	no	yes	no	Reptile
Penguin	Warm	feathers	no	semi	no	yes	no	Bird
Porcupine	Warm	quills	yes	no	no	yes	yes	Mammal
Eel	Cold	scales	no	yes	no	no	no	Fish
Salamander	Cold	none	no	semi	no	yes	yes	Amphibian

Table 4.1 Vertebrate classification

Predictive modeling

A classification model can also be used to predict the class label of unknown records as shown below

Name	Body Temperature	Skin Cover	Gives Birth	Aquatic Creature	Aerial Creature	Has Legs	Hibernates	Class
Gila Monster	cold	scale	no	no	no	yes	yes	?

→A classification model can be treated as a black box that automatically assigns the class label when presented with an attribute set of an unknown record

→Suppose we are given the characteristics of a creature known as Gila Monster

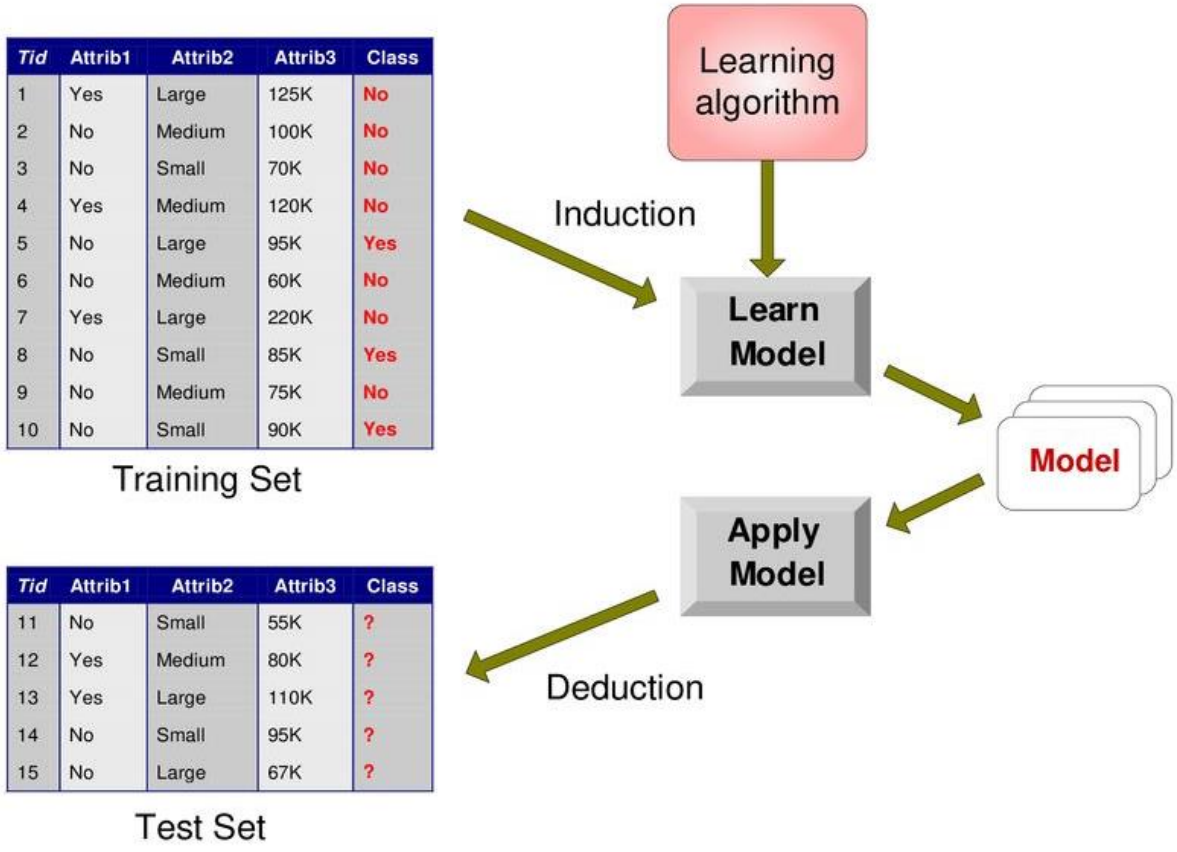
→We can use a classification model built from the data set given in Table 4.1 to determine the class to which the creature belongs

4.2 General Approach for solving a classification problem

A classification technique (or classifier) is a systematic approach to building classification models from an input data set.

The model generated by a learning algorithm should understand the input data well and correctly predict the class labels of records it has never seen before.

Therefore, a key objective of the learning algorithm is to build models with good generalization capability; i.e., models that accurately predict the class labels of previously unknown records.



Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Figure 4.1 General Approach for building a classification model

Figure 4.2 shows a general approach for solving classification problems.

First, a training set consisting of records whose class labels are known must be provided.

The training set is used to build a classification model, which is subsequently applied to the test set, which consists of records with unknown class labels.

Evaluation of the performance of a classification model is based on the counts of test records correctly and incorrectly predicted by the model. These counts are tabulated in a table known as a confusion matrix.

Confusion matrix

A table used to describe the performance of the classification model on as set of test data for which the true values are known

		Predicted Class	
		$Class = 1$	$Class = 0$
Actual Class	$Class = 1$	f_{11}	f_{10}
	$Class = 0$	f_{01}	f_{00}

Table 4.2 Confusion matrix for a 2-class problem

Table 4.2 depicts the confusion matrix for a binary classification problem. Each entry fij in this table denotes the number of records from class i predicted to be of class j. For instance, f01 is the number of records from class 0 incorrectly predicted as class 1.

Based on the entries in the confusion matrix, the total number of correct predictions made by the model is (f11 + f00) and the total number of incorrect predictions is (f10 + f01).

$$\text{Accuracy} = \frac{\#\text{ correct predictions}}{\text{total }\#\text{ of predictions}} = \frac{f_{11} + f_{00}}{f_{11} + f_{10} + f_{01} + f_{00}}$$

$$\text{Error rate} = \frac{\#\text{ wrong predictions}}{\text{total }\#\text{ of predictions}} = \frac{f_{10} + f_{01}}{f_{11} + f_{10} + f_{01} + f_{00}}$$

4.3 Types of Classifiers

Decision tree classifier

Rule based classifier

Nearest Neighbor classifier

Bayesian Classifier

4.3.1 Decision Tree Induction

Decision tree builds classification or regression models in the form of a tree structure. It breaks down a dataset into smaller and smaller subsets while at the same time an associated decision tree is incrementally developed. The final result is a tree with decision nodes and leaf nodes

How a Decision Tree works?

Figure 4.2 shows the decision tree for the mammal classification problem.

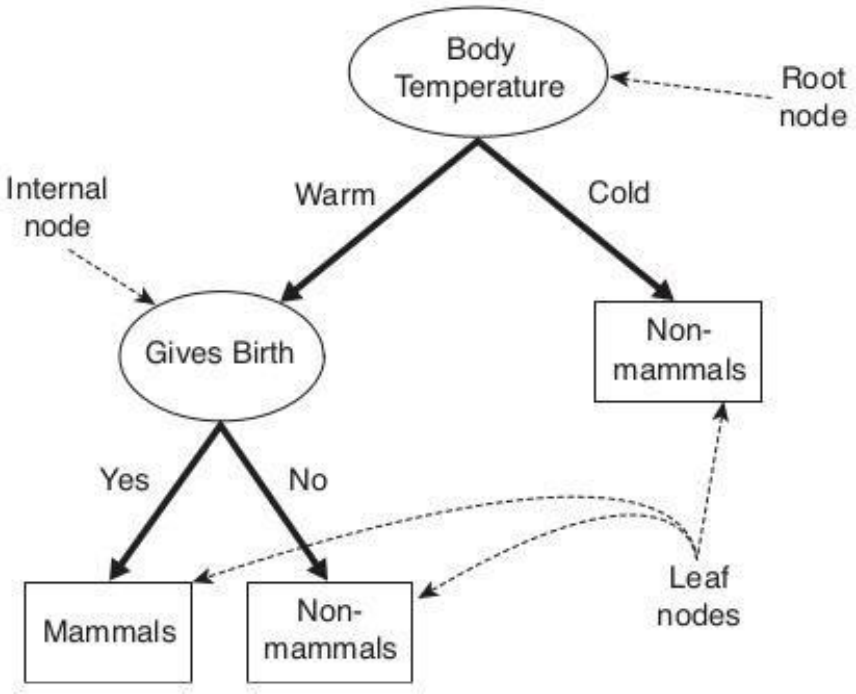


Figure 4.2: A Decision tree for the mammal classification problem

The tree has three types of nodes:

Root node

Internal nodes

Leaf or terminal nodes

Root node: Has no incoming edges and zero or more outgoing edges

Internal node: each of which has exactly one incoming edge and two or more outgoing edges

Leaf or terminal nodes: each of which has exactly one incoming edge and no outgoing edges

→ Each leaf node is assigned a class label

→ Non terminal nodes (root or internal nodes) contains attribute test condition

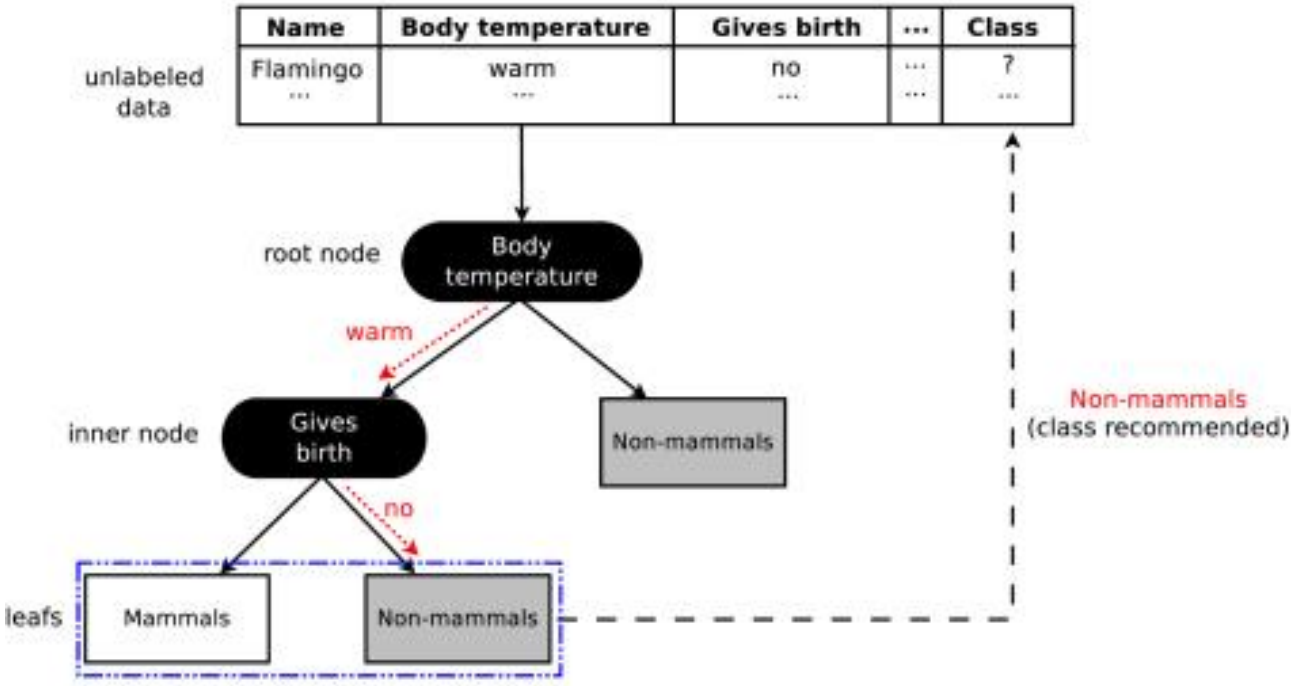


Figure 4.3: Classifying an unlabelled vertebrate

How to build a decision tree

→ Many decision trees can be constructed from a given set of attributes

→ Some of the trees are more accurate than others

→ Hunts algorithm can be used

Hunts Algorithm

Hunts algorithm grows a decision tree in a recursive fashion by partitioning the training records into successively purer subsets

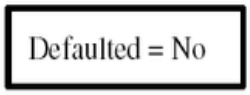
Let Dt be the set of training records that reach a node t

-If Dt contains records that belong the same class yt , then t is a leaf node labeled as yt

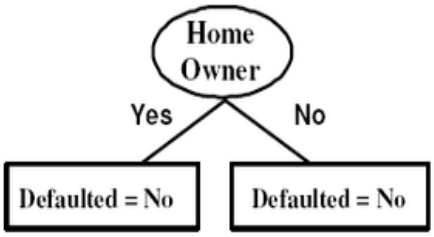
-If Dt contains records that belong to more than one class, use an attribute test to split the data into smaller subsets. Recursively apply the procedure to each subset

Consider the following example of predicting whether loan applicant will repay the loan

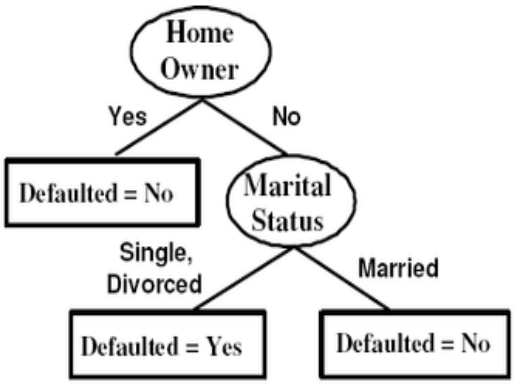
Tid	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



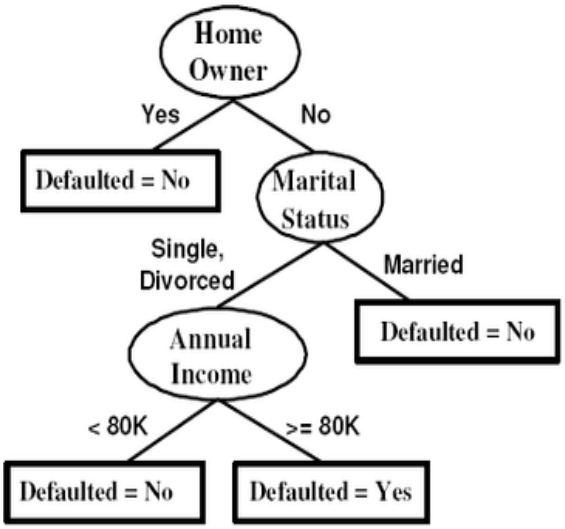
(a) Step 1



(b) Step 2



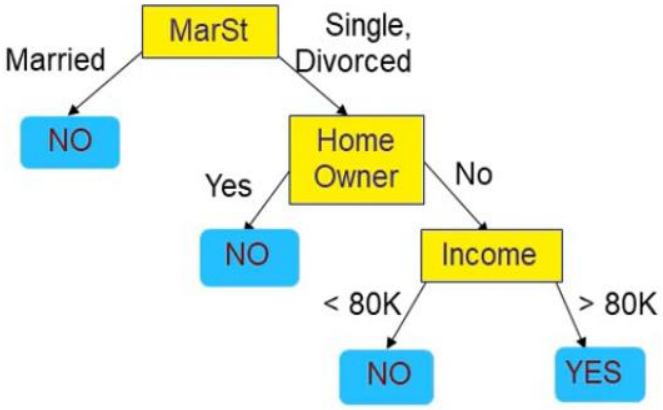
(c) Step 3



(d) Step 4

There could be more than one tree that fits the same data as shown in the Figure below

	categorical	categorical	continuous	class
ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

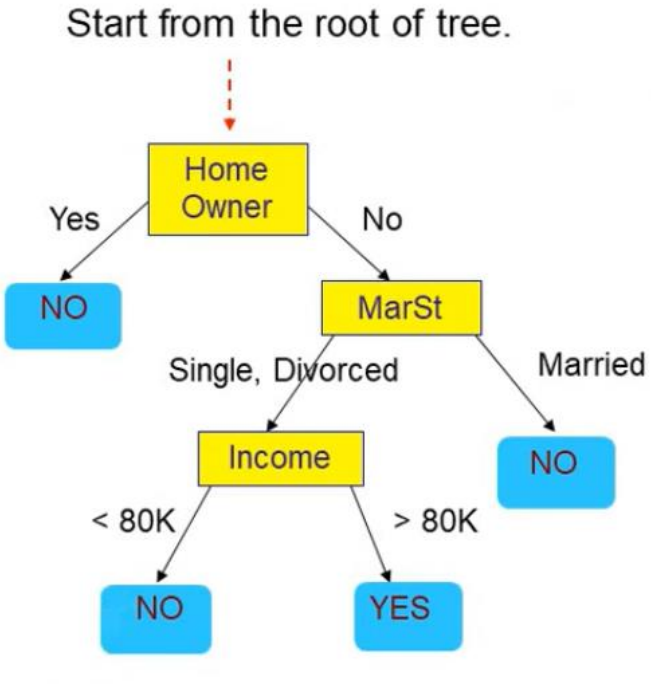


There could be more than one tree that fits the same data!

Apply Model to Test Data

Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Above test data is classified as Defaulted borrower is NO.

Methods for Expressing attribute test conditions:

Decision tree induction algorithm must provide a method for expressing an attribute test condition and its corresponding outcomes for different attributes types.

1. Binary Attributes: The test condition for a binary attribute generates two potential outcomes, as shown in Figure 4.4

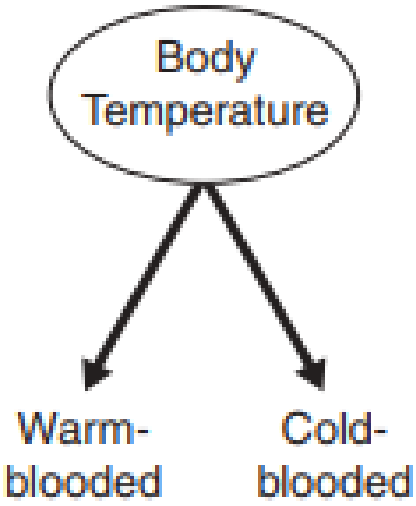


Figure 4.4 Test condition for binary attribute

2. Nominal Attributes: Since a nominal attribute can have many values, its test condition can be expressed in two ways, as shown in Figure 4.5.

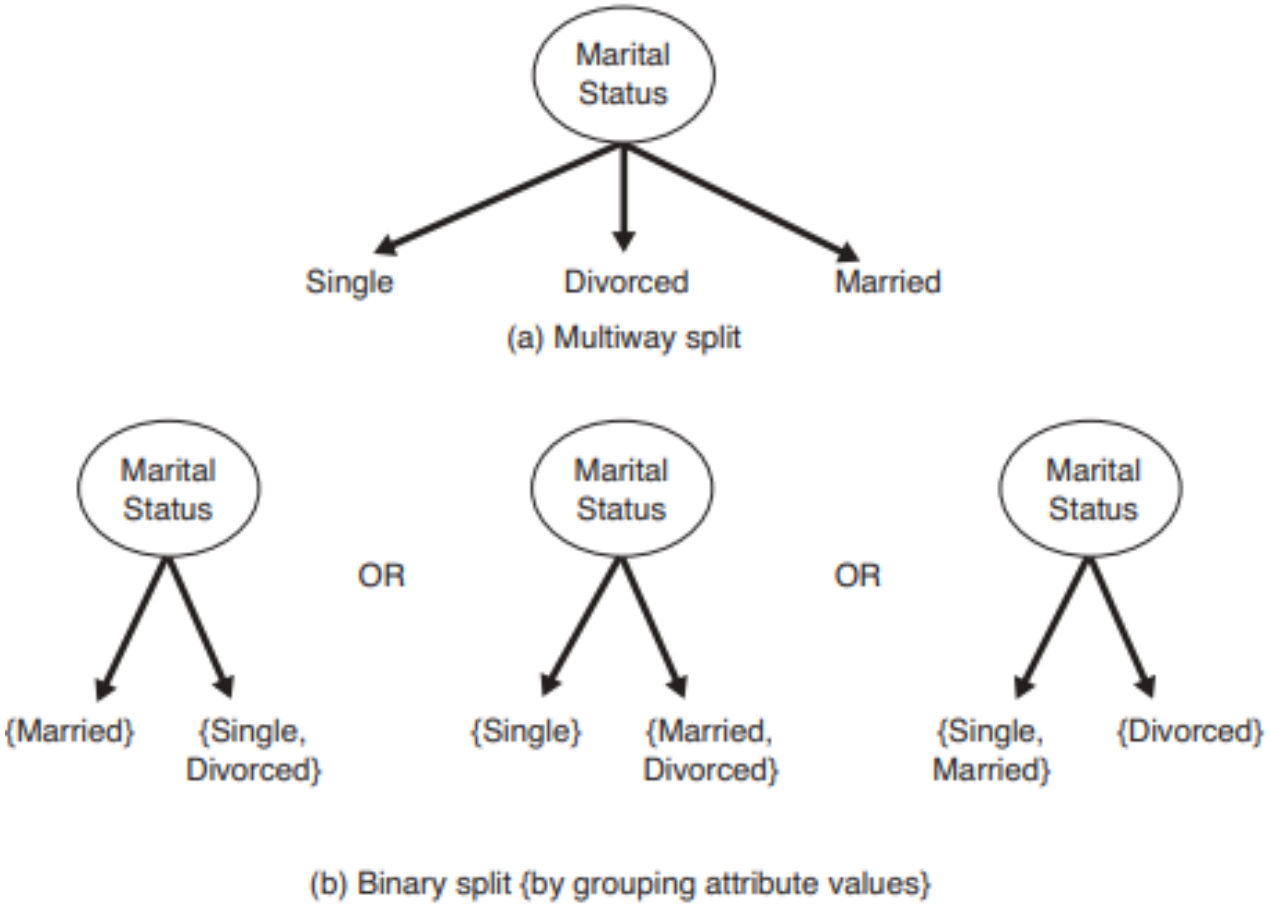


Figure 4.5: Test condition for nominal attribute

3. Ordinal Attribute

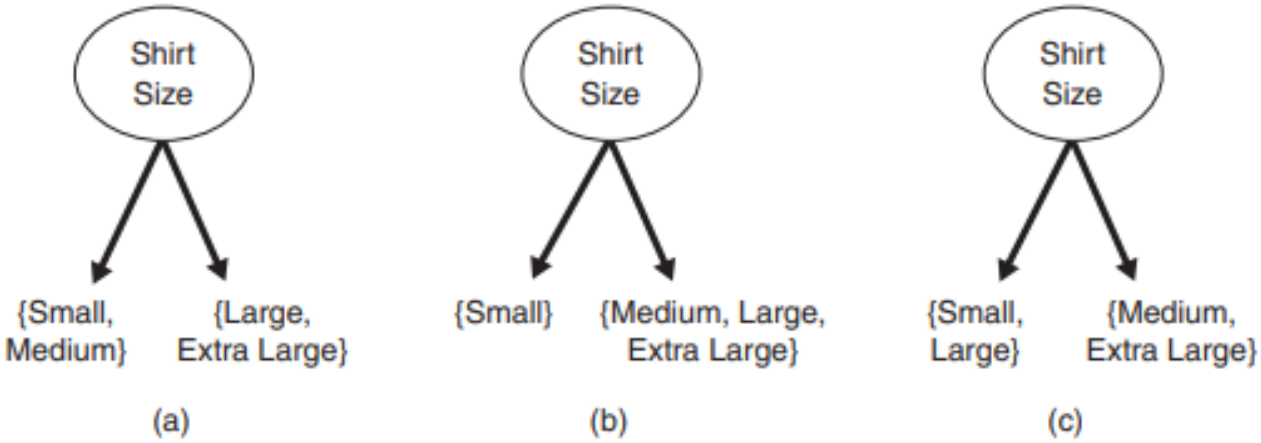


Figure 4.6: Different ways of grouping ordinal attribute values

Ordinal attributes can also produce binary or multiway splits

Ordinal attribute values can be grouped as long as the grouping does not violate the order property of the attribute values.

figure 4.6 illustrates various ways of splitting training records based on the Shirt Size attribute. The groupings shown in Figures 4.6(a) and (b) preserve the order among the attribute values, whereas the grouping shown in Figure 4.6(c) violates this property because it combines the attribute values Small and Large into the same partition while Medium and Extra Large are combined into another partition.

4. Continuous Attributes: For continuous attributes, the test condition can be expressed as a comparison test or with binary outcomes

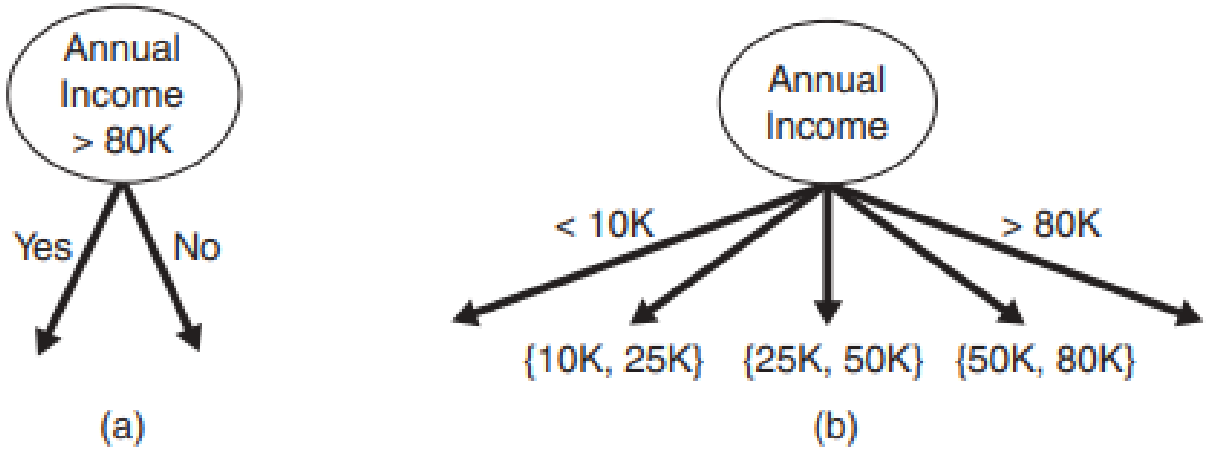


Figure 4.7 Test condition for continuous attribute

Measures for selecting the best split

- → There are many measures that can be used to determine the best way to split the records
- → The measures developed for selecting the best split are often based on the degree of impurity of the child node.
- → Smaller the degree of impurity the more biased the class distribution is.
- → A node with class distribution(0,1) has zero impurity, whereas a node with uniform distribution (0.5, 0.5) has the highest impurity

→ P(i|t)- denote the fraction of records belonging to class i at a given node t.

Example of impurity measures

$$\text{Entropy}(t) = -\sum_{i=1}^{c} p(i \mid t) \log p(i \mid t)$$

$$\text{Gini}(t) = 1 - \sum_{i=1}^{c} [p(i \mid t)]^2$$

$$\text{Classification error}(t) = 1 - \max_i [p(i \mid t)]$$

Where c is the number of classes

0 log 0 =0 in entropy calculations

Node N_1	Count
Class=0	0
Class=1	6

$\text{Gini} = 1 - (0/6)^2 - (6/6)^2 = 0$
$\text{Entropy} = -(0/6)\log_2(0/6) - (6/6)\log_2(6/6) = 0$
$\text{Error} = 1 - \max[0/6, 6/6] = 0$

Node N_2	Count
Class=0	1
Class=1	5

$\text{Gini} = 1 - (1/6)^2 - (5/6)^2 = 0.278$
$\text{Entropy} = -(1/6)\log_2(1/6) - (5/6)\log_2(5/6) = 0.650$
$\text{Error} = 1 - \max[1/6, 5/6] = 0.167$

Node N_3	Count
Class=0	3
Class=1	3

$\text{Gini} = 1 - (3/6)^2 - (3/6)^2 = 0.5$
$\text{Entropy} = -(3/6)\log_2(3/6) - (3/6)\log_2(3/6) = 1$
$\text{Error} = 1 - \max[3/6, 3/6] = 0.5$

Consider the candidate attribute test condition shown in Figures 4.8(a) and (b) for the loan borrower classification problem (Table 4.2: Data sample for loan borrower classification problem)

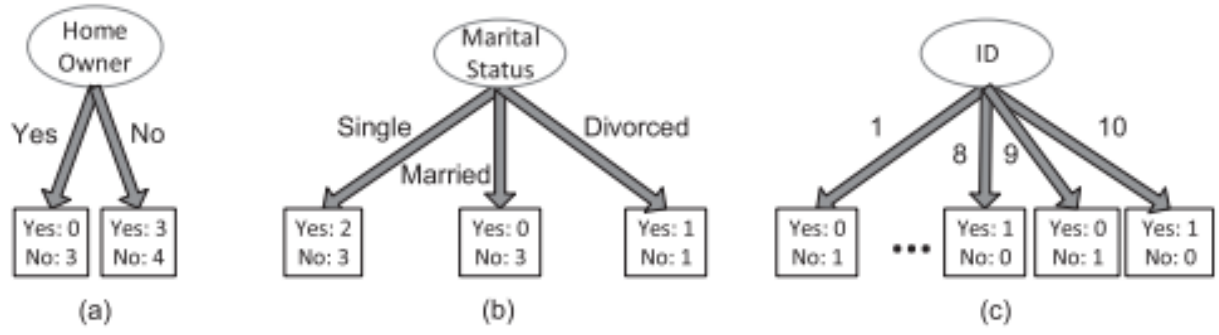


Figure 4.8: Examples of candidate attribute test conditions

ID	Home Owner	Marital Status	Annual Income	Defaulted?
1	Yes	Single	125000	No
2	No	Married	100000	No
3	No	Single	70000	No
4	Yes	Married	120000	No
5	No	Divorced	95000	Yes
6	No	Single	60000	No
7	Yes	Divorced	220000	No
8	No	Single	85000	Yes
9	No	Married	75000	No
10	No	Single	90000	Yes

Table 4.2: Data sample for loan borrower classification problem

Entropy can be calculated as follows:

$$I(\texttt{Home Owner = yes}) = -\frac{0}{3}\log_2\frac{0}{3} - \frac{3}{3}\log_2\frac{3}{3} = 0$$

$$I(\texttt{Home Owner = no}) = -\frac{3}{7}\log_2\frac{3}{7} - \frac{4}{7}\log_2\frac{4}{7} = 0.985$$

$$I(\texttt{Home Owner}) = \frac{3}{10}\times 0 + \frac{7}{10}\times 0.985 = 0.690$$

$$I(\texttt{Marital Status = Single}) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.971$$

$$I(\texttt{Marital Status = Married}) = -\frac{0}{3}\log_2\frac{0}{3} - \frac{3}{3}\log_2\frac{3}{3} = 0$$

$$I(\texttt{Marital Status = Divorced}) = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1.000$$

$$I(\texttt{Marital Status}) = \frac{5}{10}\times 0.971 + \frac{3}{10}\times 0 + \frac{2}{10}\times 1 = 0.686$$

Thus, Marital Status has a lower weighted entropy than Home Owner.

To determine how well the test condition performs, we need to compare the degree of impurity of parent node (before splitting) with the degree of impurity of the child node (after splitting)

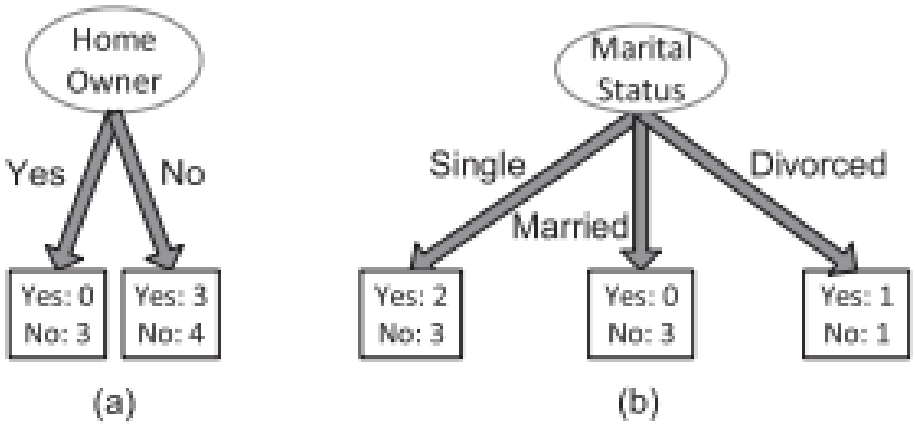
Larger the difference, better the test condition

Gain is the criteria used to determine the goodness of the split

$$\Delta = I(\text{parent}) - I(\text{children}),$$

When entropy is used as the impurity measure, the difference in entropy is commonly known as information gain, Δ_{info}.

Example



$$I(\text{parent}) = -\frac{3}{10}\log_2\frac{3}{10} - \frac{7}{10}\log_2\frac{7}{10} = 0.881$$

The information gains for Home Owner and Marital Status are each given by

$$\Delta_{\text{info}}(\texttt{Home Owner}) = 0.881 - 0.690 = 0.191$$
$$\Delta_{\text{info}}(\texttt{Marital Status}) = 0.881 - 0.686 = 0.195$$

The information gain for Marital Status is thus higher due to its lower weighted entropy, which will thus be considered for splitting.

Gain ratio

Used to compensate for attributes that produce a large number of child nodes.

$$\text{Gain ratio} = \frac{\Delta_{\text{info}}}{\text{Split Info}}$$

$$\text{Split info} = -\sum_{i=1}^{k}\frac{N(v_i)}{N}\log_2\frac{N(v_i)}{N}$$

Where N(vi) is the number of instances assigned to node vi and k is the total number of splits.

Example:

Consider the data set given below

We want to select the best attribute test condition among the following three attributes: Gender, Car Type, and Customer ID.

Customer ID	Gender	Car Type	Shirt Size	Class
1	M	Family	Small	C0
2	M	Sports	Medium	C0
3	M	Sports	Medium	C0
4	M	Sports	Large	C0
5	M	Sports	Extra Large	C0
6	M	Sports	Extra Large	C0
7	F	Sports	Small	C0
8	F	Sports	Small	C0
9	F	Sports	Medium	C0
10	F	Luxury	Large	C0
11	M	Family	Large	C1
12	M	Family	Extra Large	C1
13	M	Family	Medium	C1
14	M	Luxury	Extra Large	C1
15	F	Luxury	Small	C1
16	F	Luxury	Small	C1
17	F	Luxury	Medium	C1
18	F	Luxury	Medium	C1
19	F	Luxury	Medium	C1
20	F	Luxury	Large	C1

Table 4.3:Data set

$$\text{Entropy(parent)} = -\frac{10}{20}\log_2\frac{10}{20} - \frac{10}{20}\log_2\frac{10}{20} = 1.$$

If Gender is used as attribute test condition:

$$\text{Entropy(children)} = \frac{10}{20}\left[-\frac{6}{10}\log_2\frac{6}{10} - \frac{4}{10}\log_2\frac{4}{10}\right] \times 2 = 0.971$$

$$\text{Gain Ratio} = \frac{1 - 0.971}{-\frac{10}{20}\log_2\frac{10}{20} - \frac{10}{20}\log_2\frac{10}{20}} = \frac{0.029}{1} = 0.029$$

If Car Type is used as attribute test condition:

$$\text{Entropy(children)} = \frac{4}{20}\left[-\frac{1}{4}\log_2\frac{1}{4} - \frac{3}{4}\log_2\frac{3}{4}\right] + \frac{8}{20}\times 0 + \frac{8}{20}\left[-\frac{1}{8}\log_2\frac{1}{8} - \frac{7}{8}\log_2\frac{7}{8}\right] = 0.380$$

$$\text{Gain Ratio} = \frac{1-0.380}{-\frac{4}{20}\log_2\frac{4}{20} - \frac{8}{20}\log_2\frac{8}{20} - \frac{8}{20}\log_2\frac{8}{20}} = \frac{0.620}{1.52} = 0.41$$

Finally, if Customer ID is used as attribute test condition:

$$\text{Entropy(children)} = \frac{1}{20}\left[-\frac{1}{1}\log_2\frac{1}{1} - \frac{0}{1}\log_2\frac{0}{1}\right]\times 20 = 0$$

$$\text{Gain Ratio} = \frac{1-0}{-\frac{1}{20}\log_2\frac{1}{20}\times 20} = \frac{1}{4.32} = 0.23$$

Thus, even though Customer ID has the highest information gain, its gain ratio is lower than Car Type since it produces a larger number of splits.

Exercises

Consider the training examples shown in Table 4.3 for a binary classification problem.

(a) Compute the Gini index for the overall collection of training examples.

(b) Compute the Gini index for the **Customer** ID attribute.

(c) Compute the Gini index for the **Gender** attribute.

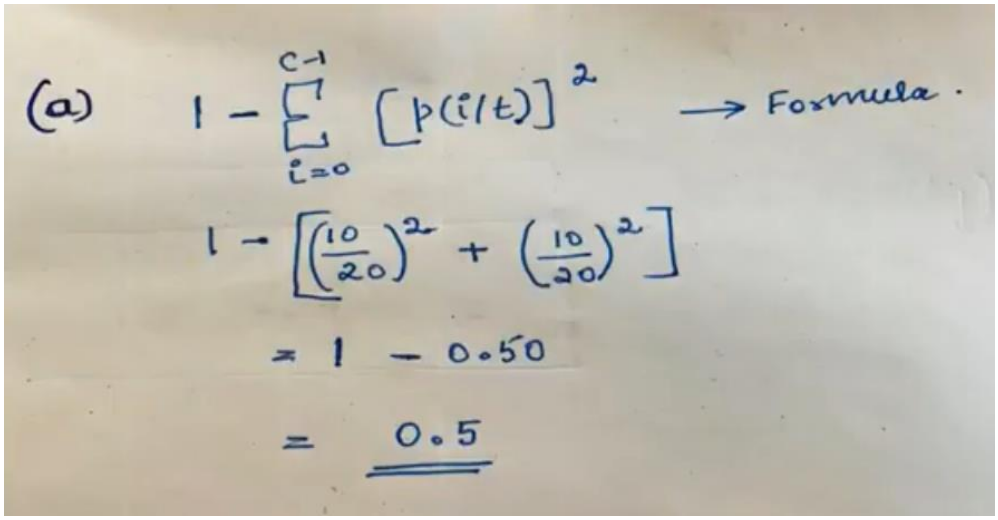
(d) Compute the Gini index for the **Car Type** attribute using multiway split.

(e) Compute the Gini index for the **Shirt Size** attribute using multiway split.

(f) Which attribute is better, **Gender**, **Car Type**, or **Shirt Size**?

(g) Explain why Customer ID should not be used as the attribute test condition even though it has the lowest Gini.

→Compute the Gini index for the overall collection of training examples



→Compute the Gini index for the **Customer** ID attribute.

The Gini for each customer ID is zero

→ Compute the Gini index for the **Gender** attribute

Gini (Male)

Male	Count
class = 0	6
class = 1	4

$$1 - \left[\left(\frac{6}{10}\right)^2 + \left(\frac{4}{10}\right)^2\right]$$
$$= 1 - [0.36 + 0.16]$$
$$= 0.48$$

Gini (Female)

Female	Count
class = 0	4
class = 1	6

$$1 - \left[\left(\frac{4}{10}\right)^2 + \left(\frac{6}{10}\right)^2\right]$$
$$1 - [0.16 + 0.36]$$
$$= 0.48$$

$$\text{Overall Gini} = \frac{10}{20} \times 0.48 + \frac{10}{20} \times 0.48$$
$$= 0.48$$

→ Compute the Gini index for the **Car Type** attribute using multiway split.

Gini (Family)

Family	Count
class = 0	1
class = 1	3

$$1 - \left[\left(\frac{1}{4}\right)^2 + \left(\frac{3}{4}\right)^2\right]$$
$$= 1 - [0.0625 + 0.562]$$
$$= 0.375$$

Gini (Sports)

Sports	Count
class = 0	8
class = 1	0

$$1 - \left[\left(\frac{8}{8}\right)^2 + \left(\frac{0}{8}\right)^2\right]$$
$$= 1 - 1$$
$$= 0$$

Gini (Luxury)

Luxury	count
class = 0	1
class = 1	7

$$1 - \left[\left(\frac{1}{8}\right)^2 + \left(\frac{7}{8}\right)^2\right]$$
$$= 0.21375$$

$$\text{Overall Gini} = \frac{4}{20} \times 0.375 + \frac{8}{20} \times 0 + \frac{8}{20} \times 0.2137$$
$$= 0.1625$$

→ Compute the Gini index for the **Shirt Size** attribute using multiway split.

Gini (Small)

Small	Count
class = 0	3
class = 1	2

$1 - \left[\left(\frac{3}{5}\right)^2 + \left(\frac{2}{5}\right)^2\right]$

$= 0.48$

Gini (Medium)

Medium	Count
class = 0	3
class = 1	4

$1 - \left[\left(\frac{3}{7}\right)^2 + \left(\frac{4}{7}\right)^2\right]$

$= 0.49$

Gini (Large)

Large	Count
class = 0	2
class = 1	2

$1 - \left[\left(\frac{2}{4}\right)^2 + \left(\frac{2}{4}\right)^2\right]$

$= 0.5$

Gini (Extra Large)

E.Large	Count
class = 0	2
class = 1	2

$1 - \left[\left(\frac{2}{4}\right)^2 + \left(\frac{2}{4}\right)^2\right]$

$= 0.5$

Overall Gini Index =

$\frac{5}{20} \times 0.48 + \frac{7}{20} \times 0.49 + \frac{4}{20} \times 0.5 + \frac{4}{20} \times 0.5$

$= 0.4915$

→Which attribute is better, **Gender**, **Car Type**, or **Shirt Size**?

Car type, because it has the lowest gini compared to the three attributes

→Explain why Customer ID should not be used as the attribute test condition even though it has the lowest Gini.

Attribute has no predictive power since the new customers are assigned to the new customer ID's

Algorithm for decision tree induction

A skeleton decision tree induction algorithm called TreeGrowth is shown in Algorithm below

The input to this algorithm consists of the training records E and the attribute set F.

The algorithm works by recursively selecting the best attribute to split the data (Step 7) and expanding the leaf nodes of the tree (step 11 and 12) until the stopping criterion is met (step 1).

Algorithm 4.1: A skeleton decision tree induction algorithm

```
TreeGrowth (E, F)
1.  if stopping_cond(E,F) = true then
2.     leaf = createNode()
3.     leaf.label = Classify(E).
4.     return leaf.
5.  else
6.     root = createNode().
7.     root.test-cond = f ind_best_split(E, F).
8.     let V = {v\v is a possible outcome of root.test-cond }.
9.     for each v Є V do
10.          Ev = {e | root.test-cond(e) = v and e Є E).
11.          child = TreeGrowth(Ev, F).
12.          add child as descendent of root and label the edge (root —> child) as v.
13.    end for
14. end if
15. return root.
```

The Details of the Algorithm are explained below:

→The createNode() function extends the decision tree by creating a new node

→ The find_best_split() function determines which attribute should be selected as the test condition for splitting the training records. The choice of test condition depends on which impurity measure is used to determine the goodness of split. Some of them are Entropy, Gini Index etc..

→ The Classify() function determines the class label to be assigned to a leaf node.

→ The stopping_cond() function is used to terminate the tree-growing process by testing whether all the records have either the same class label or the same attribute value s.

Characteristics of decision tree induction

→Decision tree induction is a non-parametric approach for building classification models

→Finding an optimal decision tree is an NP-complete problem. Many decision tree algorithms employ a heuristic-based approach to guide their search in the vast hypothesis space.

→Techniques developed for constructing decision trees are computationally inexpensive, making it possible to quickly construct models even when the training set size is very large.

→Decision trees, especially smaller-sized trees, are relatively easy to interpret.

→Decision trees provide an expressive representation for learning discrete-valued functions.

→Decision tree algorithms are quite robust to the presence of noise, especially when methods for avoiding over-fitting.

→The presence of redundant attributes does not adversely affect the accuracy of decision trees

→Since most decision tree algorithms employ a top-down, recursive partitioning approach, the number of records becomes smaller as we traverse down the tree.

→These algorithms usually employ a greedy strategy that grows a decision tree by making a series of locally optimum decisions about which attribute to use for partitioning the data

→ Many Algorithms:

-Hunts Algorithm

-CART

-ID3, C4.5

-SLIQ, SPRINT

4.3.2 Rule based Classifiers

A rule based classifier is a technique for classifying records using a collection of **"if … then …"** rules.

Following table shows an example of a model generated by a rule based classifier for the vertebrate classification problem.

1) R1: (Give Birth = no) ^ (Aerial Creature = yes) → Birds
2) R2: (Give Birth = no) ^ (Aquatic Creature = yes)→ Fishes
3) R3: (Give Birth = yes) ^ (Blood Type = warm) →Mammals
4) R4: (Give Birth = no) ^ (Aerial Creature = no) →Reptiles
5) R5: (Live in Water = sometimes) → Amphibians

Each classification rule can be expressed in the following ways:

r_i: (Condition$_i$) → y_i

Conditioni= (A1 op v1) ^ (A2 op v2) ^…(Ak op vk)

Where (Aj, vj) is an attribute-value pair and op is a logical operator chosen from the set {=, !=, <, <=, >, >=}

Left hand side of the rule is called the rule **antecedent** or **precondition**

Right hand side of the rule is called the rule **consequent**, which contains the predicted class yi.

Name	Blood Type	Give Birth	Can Fly	Live in Water	Class
human	warm	yes	no	no	mammals
python	cold	no	no	no	reptiles
salmon	cold	no	no	yes	fishes
whale	warm	yes	no	yes	mammals
frog	cold	no	no	sometimes	amphibians
komodo	cold	no	no	no	reptiles
bat	warm	yes	yes	no	mammals
pigeon	warm	no	yes	no	birds
cat	warm	yes	no	no	mammals
leopard shark	cold	yes	no	yes	fishes
turtle	cold	no	no	sometimes	reptiles
penguin	warm	no	no	sometimes	birds
porcupine	warm	yes	no	no	mammals
eel	cold	no	no	yes	fishes
salamander	cold	no	no	sometimes	amphibians
gila monster	cold	no	no	no	reptiles
platypus	warm	no	no	no	mammals
owl	warm	no	yes	no	birds
dolphin	warm	yes	no	yes	mammals
eagle	warm	no	yes	no	birds

Application of rule based classifier

R1: (Give Birth = no) $\wedge$ (Can Fly = yes) $\rightarrow$ Birds

R2: (Give Birth = no) $\wedge$ (Live in Water = yes) $\rightarrow$ Fishes

R3: (Give Birth = yes) $\wedge$ (Blood Type = warm) $\rightarrow$ Mammals

R4: (Give Birth = no) $\wedge$ (Can Fly = no) $\rightarrow$ Reptiles

R5: (Live in Water = sometimes) $\rightarrow$ Amphibians

Name	Blood Type	Give Birth	Can Fly	Live in Water	Class
hawk	warm	no	yes	no	?
grizzly bear	warm	yes	no	no	?

First vertebrate triggers rule R1, hence classified as Bird

Second vertebrate triggers rule R3, hence classified as Mammals

Rule coverage and accuracy

Coverage of a rule:

Fraction of records satisfy the antecedent of a rule

Accuracy of a rule:

Fraction of records that satisfy the antecedent that also satisfy the consequent of a rule

$$\text{Coverage}(r) = \frac{|A|}{|D|}$$

$$\text{Accuracy}(r) = \frac{|A \cap y|}{|A|}, \qquad (5.3)$$

where $|A|$ is the number of records that satisfy the rule antecedent, $|A \cap y|$ is the number of records that satisfy both the antecedent and consequent, and $|D|$ is the total number of records.

Example

(Status= Single) → No

Coverage=4/10= 40%
Accuracy=2/4=50%

Tid	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

How does a rule based classifier works

A rule-based classifier classifies a test record based on the rule triggered by the record. To illustrate how a rule-based classifier works, consider the rule set given below Table and the following vertebrates

Name	Blood type	Skin cover	Gives birth	Lives in water	can fly	Has legs	Hibernate
Lemur	Warm	Fur	Yes	No	No	Yes	Yes
Turtle	Cold	Scales	No	Semi	No	Yes	No
Dogfish shark	cold	scales	Yes	Yes	No	No	no

Lemur triggers rule r3, hence it is classified as a mammal
Turtle triggers both r4 and r5
Dogfish shark triggers none of the rules

Characteristics of a rule set

Mutually Exclusive rules:

- classifier contains mutually exclusive rules if the rules are independent of each other

- every record is covered by at most one rule

Exhaustive rules

-classifier has exhaustive coverage if there is a rule for every possible combination of attribute values

- each record is covered by at least one rules

If the rule set is not exhaustive, then a default rule

Rd: () → yd

Must be added to cover the remaining cases.

If the rule set is not mutually exclusive, there are two ways to overcome the problem

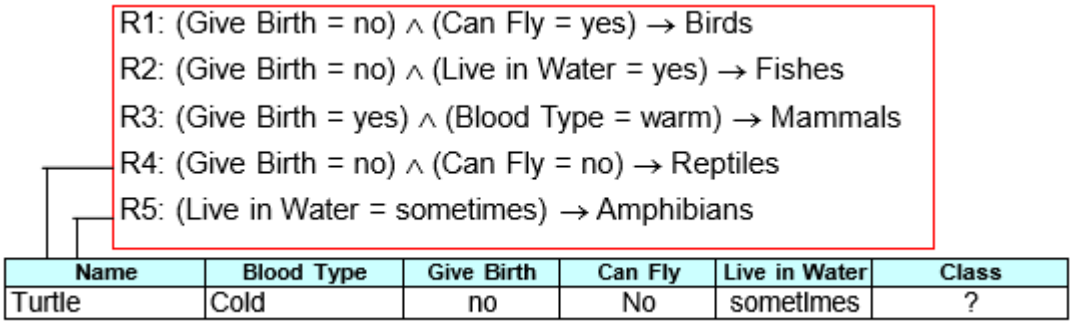
Ordered rules

Unordered rules

Ordered rules

In this approach, the rules in a rule set are ordered in decreasing order of their priority, which can be defined in many ways {e.g., based on accuracy, coverage, total description length, or the order in which the rules are generated). An ordered rule set is also known as a decision list

Ex: Rules are ordered based on the way they are generated.



Name	Blood Type	Give Birth	Can Fly	Live in Water	Class
Turtle	Cold	no	No	sometimes	?

Unordered rules

 This approach allows a test record to trigger multiple classification rules and considers the consequent of each rule as a vote for a particular class.

 The votes are then tallied to determine the class label of the test record. The record is usually assigned to the class that receives the highest number of votes.

  In some cases, the vote may be weighted by the rule's accuracy.

Rule Ordering Scheme

This ordering scheme ensures that every test record is classified by the "best" rule covering it and Individual rules are ranked based on their quality

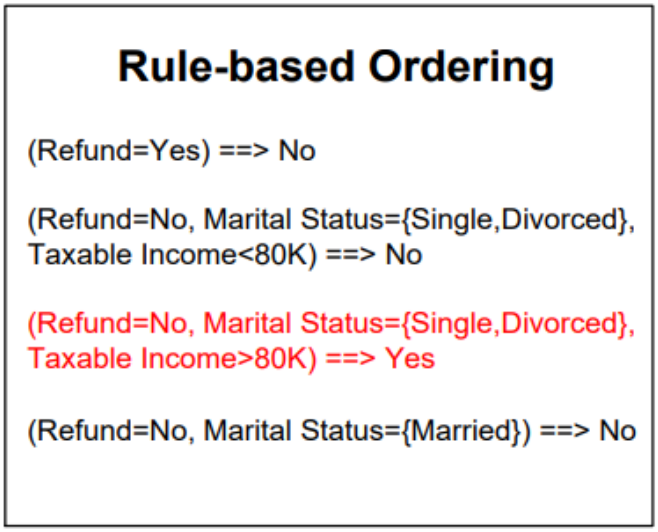
Two ordering schemes:

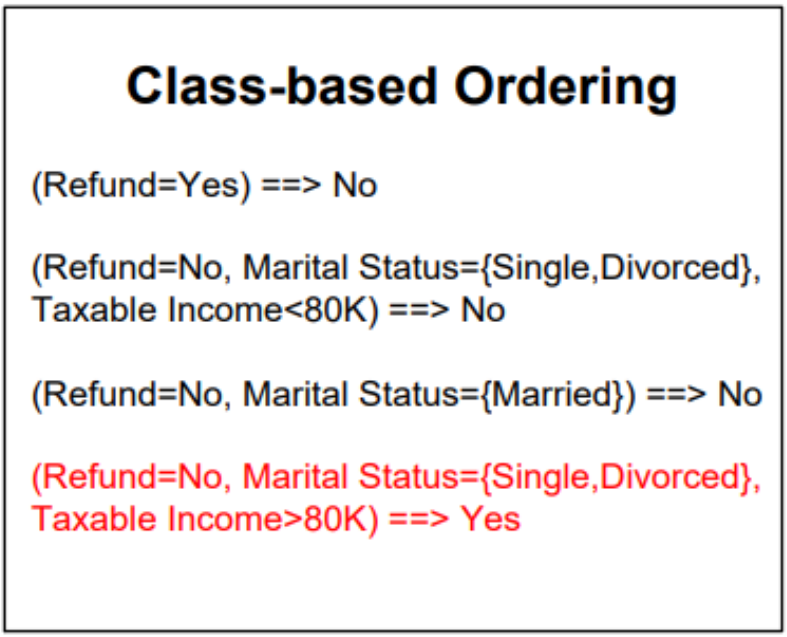
→ Rule based ordering

→ Class based ordering

In rule based ordering, the rules are ordered based on their priority

In class based ordering, Rules that belong to the same class appear together





How to build a Rule Based Classifier?

There are two broad classes of methods for extracting classification rules:

(1) **Direct method**, which extract classification rules directly from data

(2) **Indirect method**, which extract classification rules from other classification models, such as decision trees and neural networks

Direct method (Sequential covering)

The sequential covering algorithm is often used to extract rules directly from data.

The algorithm extracts the rules one class at a time for data sets that contain more than two classes. For the vertebrate classification problem, the sequential covering algorithm may generate rules for classifying birds first, followed by rules for classifying mammals, amphibians, reptiles, and finally, fishes

Sequential Covering Algorithm steps:

1. Start from an empty rule

2. Grow a rule for a record

3. Remove training records covered by the rule

4. Repeat Step (2) and (3) until stopping criterion is met

Algorithm 5.1 Sequential covering algorithm.

```
1: Let E be the training records and A be the set of attribute-value pairs, {(A_j, v_j)}.
2: Let Y_o be an ordered set of classes {y_1, y_2, ..., y_k}.
3: Let R = { } be the initial rule list.
4: for each class y ∈ Y_o − {y_k} do
5:    while stopping condition is not met do
6:       r ← Learn-One-Rule (E, A, y).
7:       Remove training records from E that are covered by r.
8:       Add r to the bottom of the rule list: R ⟶ R ∨ r.
9:    end while
10: end for
11: Insert the default rule, {} ⟶ y_k, to the bottom of the rule list R.
```

Example of Sequential covering

Below figure demonstrates how the sequential covering algorithm works for a data set that contains a collection of positive and negative examples.

The rule R1, whose coverage is shown in Figure ii), is extracted first because it covers the largest fraction of positive examples.

All the training records covered by Rl are subsequently removed and the algorithm proceeds to look for the next best rule, which is R2.

Rule growing

Two common strategies:

General-to-Specific

Specific-to-General

General-to-Specific

→ An initial rule is created

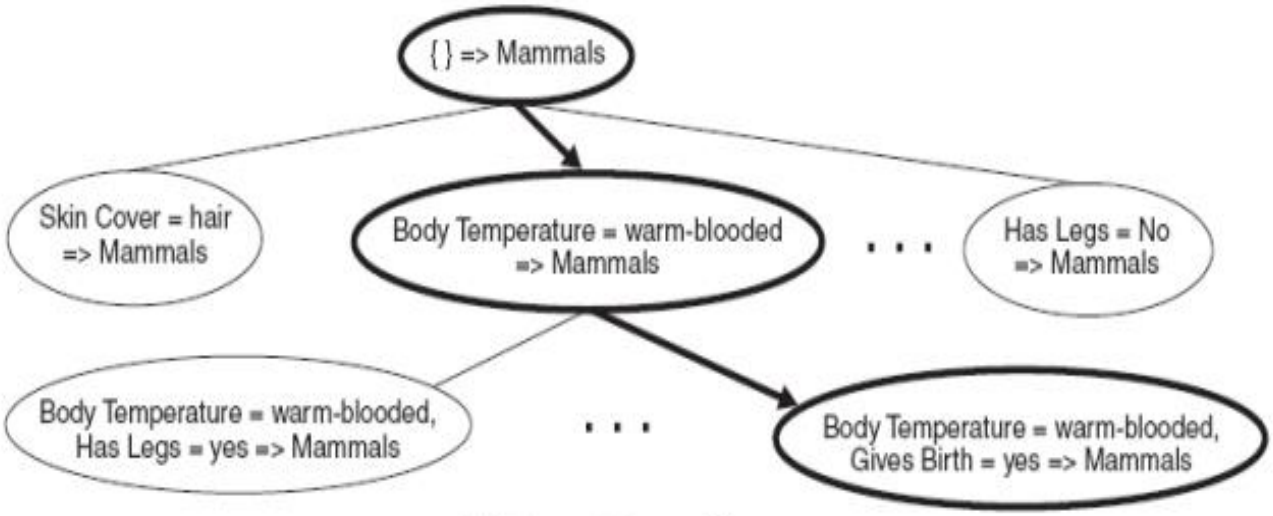
r: () → y

→ Where left hand side is an empty set

→ The rule has poor quality because it covers all the records in training set

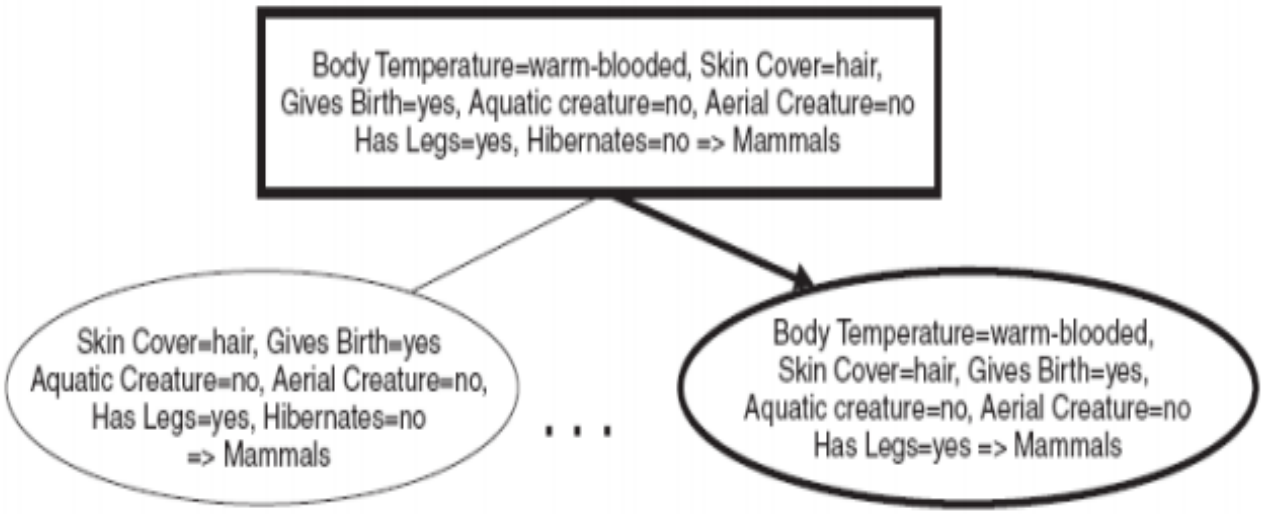
→ New conjuncts are subsequently added to improve the rules quality

Example: Vertebrate classification problem



(a) General-to-specific

Specific-to-General

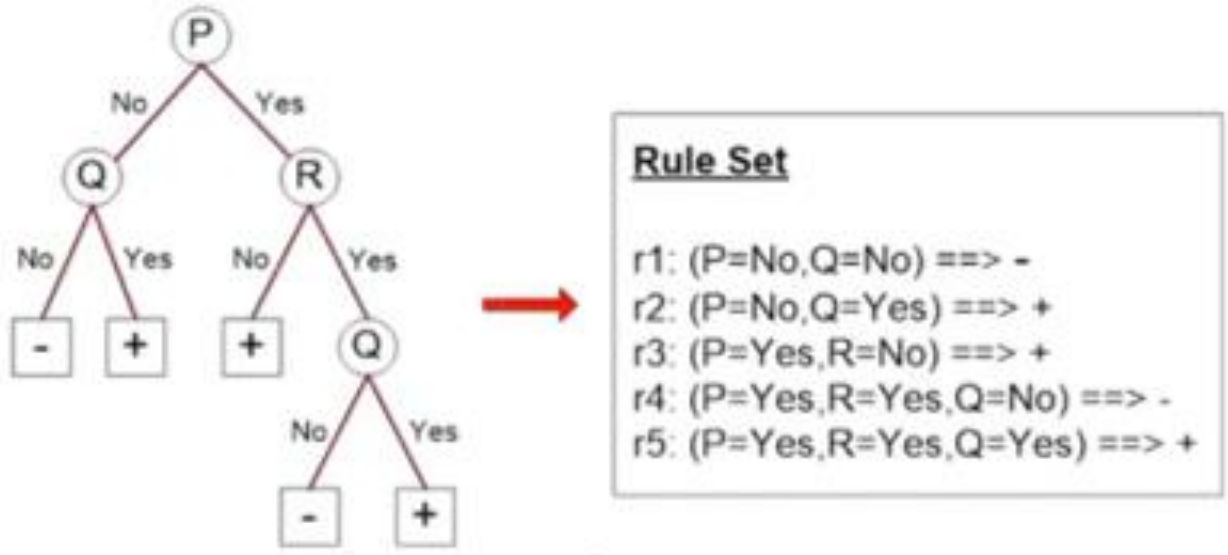


(b) Specific-to-general

Indirect method

Generating a rule set from decision tree

Example:



Advantages of rule based classifiers

As highly expressive as decision tree

Easy to interpret

Easy to generate

Can classify new instances rapidly

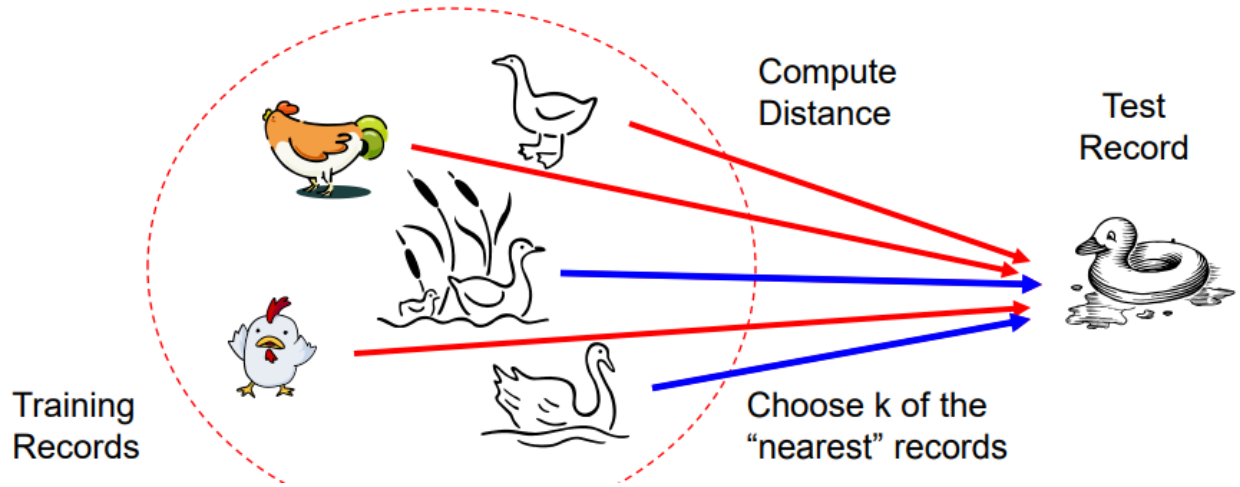
Performance comparable to decision tree

4.3.3 Nearest Neighbor classifier

Uses K- closest points (nearest neighbors) for performing classification

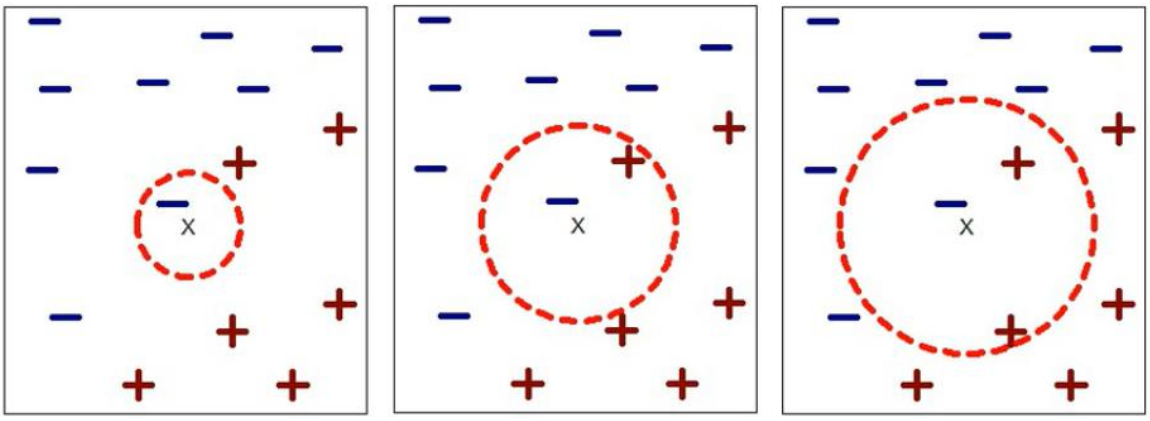
Basic Idea:

If it walks like a duck, quacks like a duck, then its probably a duck



Requires three things

- The set of stored records
- Distance Metric to compute distance between records
- The value of *k*, the number of nearest neighbors to retrieve



(a) 1-nearest neighbor (b) 2-nearest neighbor (c) 3-nearest neighbor

K-nearest neighbors of a record x are data points that have the k smallest distances to x

Above Figure illustrates the 1-, 2-, and 3-nearest neighbors of a data point located at the center of each circle. The data point is classified based on the class labels of its neighbors.

In the case where the neighbors have more than one label, the data point is assigned to the majority class of its nearest neighbors.

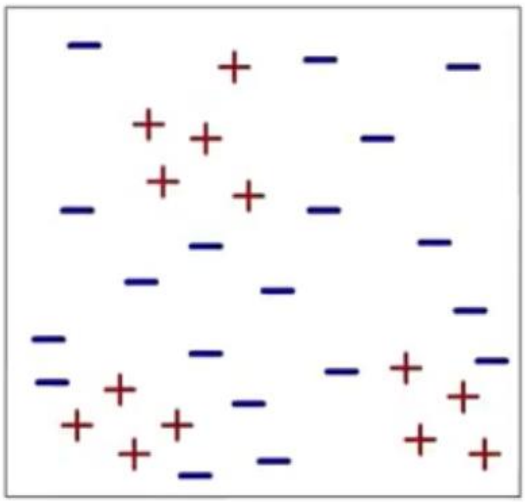
In Figure (a), the 1-nearest neighbor of the data point is a negative example. Therefore, the data point is assigned to the negative class.

If the number of nearest neighbors is three, as shown in Figure (c), then the neighborhood contains two positive examples and one negative example. Using the majority voting scheme, the data point is assigned to the positive class.

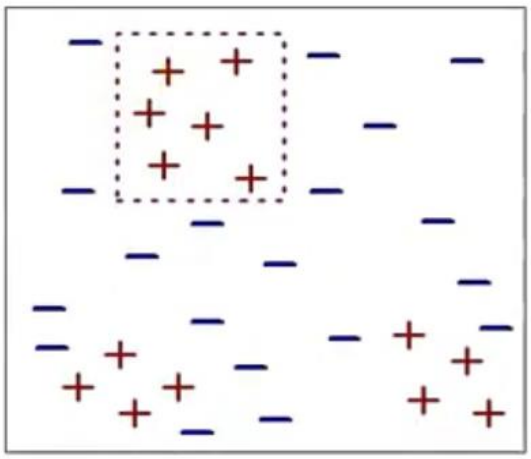
In the case where there is a tie between the classes (see Figure (b), we may randomly choose one of them to classify the data point.

The preceding discussion underscores the importance of choosing the right value for k. If k is too small, then the nearest-neighbor classifier may be susceptible to over-fitting because of noise in the training data.

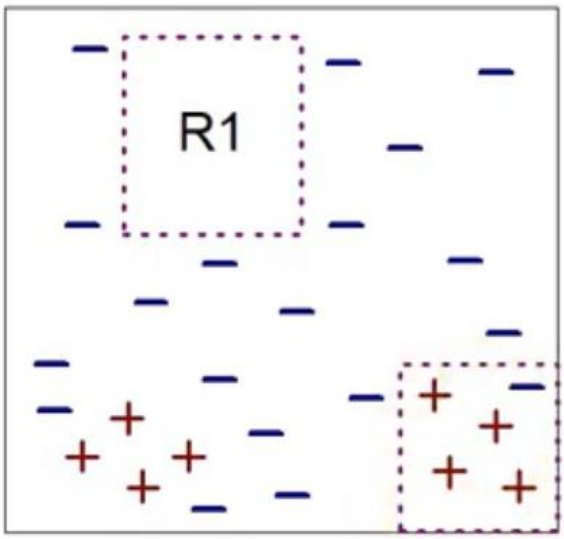
On the other hand, if k is too large, the nearest-neighbor classifier may misclassify the test instance because its list of nearest neighbors may include data points that are located far away from its neighborhood.



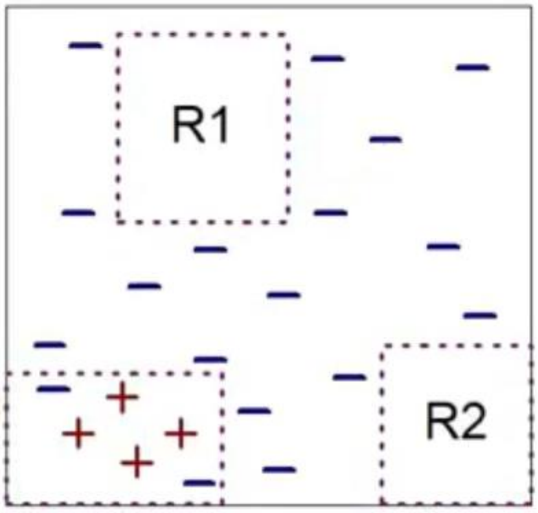
(i) Original Data



(ii) Step 1



(iii) Step 2



(iv) Step 3

K-NN algorithm

Algorithm 5.2 The k-nearest neighbor classification algorithm.

1: Let k be the number of nearest neighbors and D be the set of training examples.
2: **for** each test example $z = (\mathbf{x}', y')$ **do**
3: Compute $d(\mathbf{x}', \mathbf{x})$, the distance between z and every example, $(\mathbf{x}, y) \in D$.
4: Select $D_z \subseteq D$, the set of k closest training examples to z.
5: $y' = \underset{v}{\operatorname{argmax}} \sum_{(\mathbf{x}_i, y_i) \in D_z} I(v = y_i)$
6: **end for**

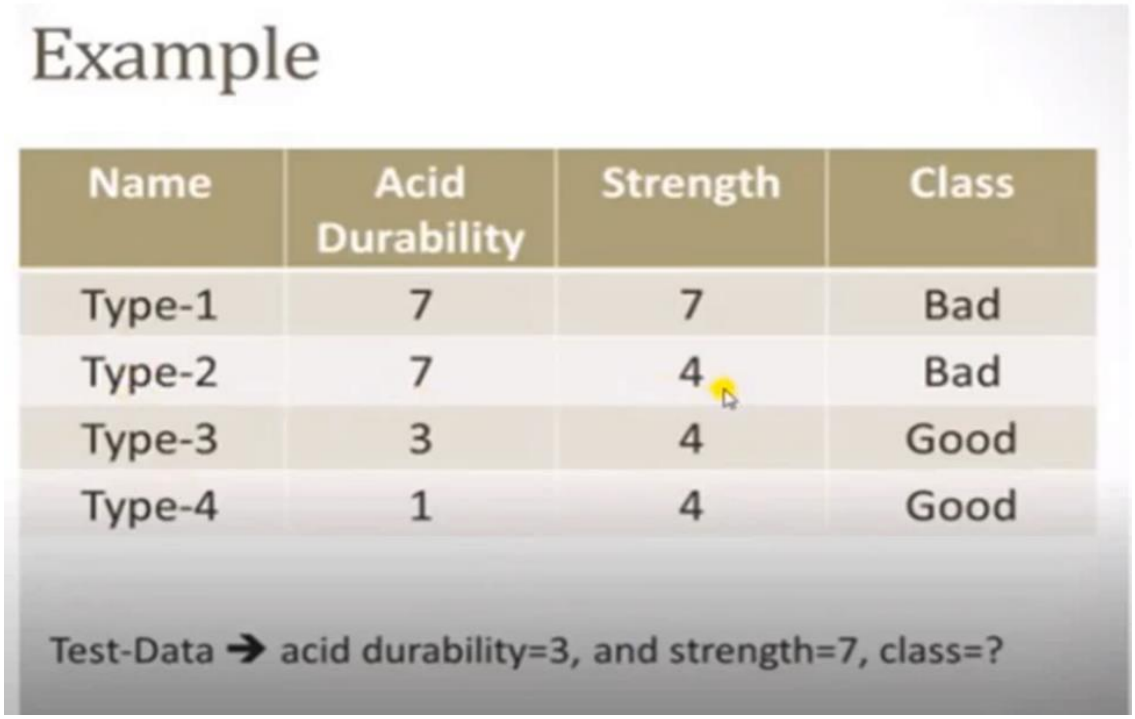
Once the nearest neighbor list is obtained, the test example is classified based on majority class of its nearest neighbour

- Majority Voting: $y' = \underset{v}{\operatorname{argmax}} \sum_{(x_i, y_i) \in D_z} I(v = y_i)$

where, v is a class label

yi is the class label for one of the nearest neighbors

I is an indicator function that returns the value 1 if its argument is true and 0 otherwise.



Characteristics of Nearest-Neighbor Classifiers

Nearest-neighbor classification is part of a more general technique known as instance-based learning, which uses specific training instances to make predictions without having to maintain an abstraction (or model) derived from data.

Lazy learners such as nearest-neighbor classifiers do not require model building. However, classifying a test example can be quite expensive because we need to compute the proximity values individually between the test and training examples.

Nearest-neighbor classifiers make their predictions based on local information, whereas decision tree and rule-based classifiers attempt to find a global model that fits the entire input space.

Nearest-neighbor classifiers can produce arbitrarily shaped decision boundaries. Such boundaries provide a more flexible model representation compared to decision tree and rule - based classifiers

Nearest-neighbor classifiers can produce wrong predictions unless the appropriate proximity measure and data preprocessing steps are taken.

4.3.4 Bayesian classifiers

Bayes Theorem: Bayes theorem is a statistical principle for combining prior knowledge of the classes with new evidence gathered from data.

Let X and Y be a pair of random variables

Their,

Joint probability, P(X=x, Y=y) refers to the probability that variable X will take on the value x and variable Y will take on the value y.

Conditional probability, P(Y=y | X=x) refers to the probability that the variable Y will take on the value y, given that the variable X is observed to have the value x

Formula of Bayes theorem

- The joint and conditional probabilities for X and Y are related in the following way:

$$P(X, Y) = P(Y|X) \times P(X) = P(X|Y) \times P(Y)$$

- Rearranging the last two expressions leads to the following formula, known as the Bayes theorem:

$$P(Y \mid X) = \frac{P(X \mid Y)P(Y)}{P(X)}$$

Using Bayes theorem for classification

Consider the following data set:

Tid	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Figure 3.6. Training set for predicting borrowers who will default on loan payments.

Suppose any test tuple comes with

marital status= married we need to classify them as defaulted borrower = yes or defaulted borrower= no

P(Defaulted borrower=No| marital status = married)

$$\rightarrow \frac{P(marital\ status = married | default\ borrower = No) X}{P(default\ borrower=No)} \over P(marital\ status=married)$$

$$\rightarrow \frac{4/7X\ 7/10}{4/10} = 1.00$$

P(Defaulted borrower=Yes| marital status = married)

$$\rightarrow \frac{P(marital\ status = married | default\ borrower = yes) X}{P(default\ borrower=yes)} \over P(marital\ status=married)$$

$$\rightarrow \frac{0/3X\ 3/10}{4/10} = 0$$

Conclusion:

Since P(Defaulted borrower=No| marital status = married) > P(Defaulted borrower=Yes| marital status = married)

The test tuple with attribute marital status=married is classified as default borrower=No

Naive Bayes classifier

A naive Bayes classifier estimates the class-conditional probability by assuming that the attributes are conditionally independent.

$$P(\mathbf{X}|Y = y) = \prod_{i=1}^{d} P(X_i|Y = y),$$

- Suppose we are given a test record with the following attribute set: X = (Home Owner = No, Marital Status = Married, Annual Income = $120K).

X =(Hone Owner : No, Marital Status : Married, Annual Income : $120K)

- P(X | No) = P(Home Owner=No | No)
  × P(Status=Married| No)
  × P(Annual Income=120K | No)
  = 4/7 × 4/7 × 0.0072 = 0.0006

- P(X | Yes) = P(Home Owner=No | Yes)
  × P(Status=Married| Yes)
  × P(Income=120K | Yes)
  = 1 × 0× 1.2 × 10^{-9} = 0

For the continuous variable

$$P(X_i = x_i|Y = y_j) = \frac{1}{\sqrt{2\pi}\sigma_{ij}} \exp^{-\frac{(x_i-\mu_{ij})^2}{2\sigma_{ij}^2}}.$$

$$\overline{x} = \frac{125 + 100 + 70 + \ldots + 75}{7} = 110$$

$$s^2 = \frac{(125 - 110)^2 + (100 - 110)^2 + \ldots + (75 - 110)^2}{7(6)} = 2975$$

$$s = \sqrt{2975} = 54.54.$$

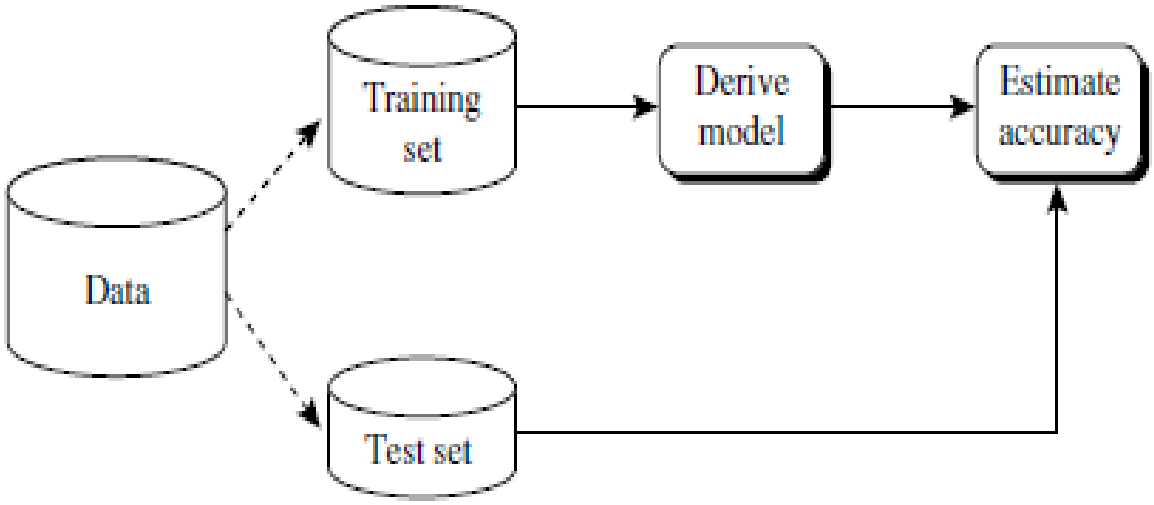
$$P(\text{Income=120|No}) = \frac{1}{\sqrt{2\pi}(54.54)} \exp^{-\frac{(120-110)^2}{2\times 2975}} = 0.0072.$$

Estimating predictive accuracy of classification method

The following are the different methods for estimating the accuracy of a method.

Holdout method (Test sample method):

- In this method, the given data are randomly partitioned into two independent sets, a *training set* and a *test set*. Typically, two-thirds of the data are allocated to the training set, and the remaining one-third is allocated to the test set. The training set is used to derive the model. The model's accuracy is then estimated with the test set



Random sub-sampling method:

- **Random subsampling** is a variation of the holdout method in which the holdout method is repeated *k* times.
- The overall accuracy estimate is taken as the average of the accuracies obtained from each iteration.

K-fold cross validation method:

- Training and testing is performed *k* times.
- In the first iteration, subsets *D*2, … , *Dk* collectively serve as the training set to obtain a first model, which is tested on *D*1; the second iteration is trained on subsets *D*1, *D*3, … , *Dk* and tested on *D*2; and so on.
- The accuracy estimate is the overall number of correct classifications from the *k* iterations, divided by the total number of tuples in the initial data.

Boot strap method:

- In this method, given a data set of size n a bootstrap sample is randomly selected uniformly with replacement (a sample may be selected more than once) and build a model.

Other evaluation criteria for classification methods

Speed:

Speed involves not just the time or computation cost of constructing a model it also includes the time required to learn to use the model.

Robustness:

Even if there are some data error, the classification method should be able to produce good results.

Scalability:

As the data set increases the method should work efficiently for large data bases

Flexibility:

Flexible enough for small modifications on the training data set

Time complexity:

Should not be exponential nor factorial. It should be in terms of log or linear or N^2

Interpretability

It is desirable that the end-user be able to understand and gain insight from the results produced by the classification method

MODULE 5

Cluster Analysis

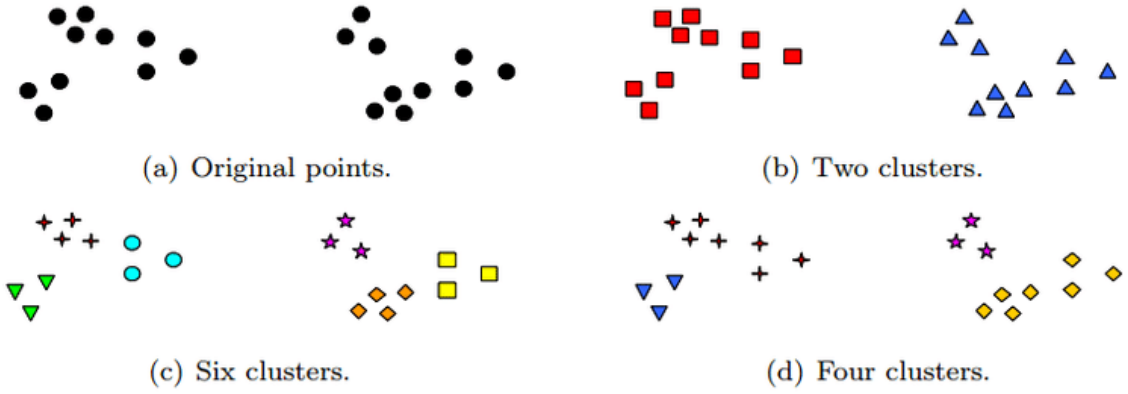
5.1 Introduction

Clustering is the task of dividing the population or data points into a number of groups such that data points in the same groups are more similar to other data points in the same group than those in other groups. In simple words, the aim is to segregate groups with similar traits and assign them into clusters.

Cluster: It is said to be "Collection of data objects"

Applications of Clustering

- ✓ Data Mining
- ✓ Pattern recognition
- ✓ Image analysis
- ✓ Bioinformatics
- ✓ Machine Learning
- ✓ Voice mining
- ✓ Image processing
- ✓ Text mining
- ✓ Web cluster engines
- ✓ Whether report analysis



(a) Original points. (b) Two clusters.

(c) Six clusters. (d) Four clusters.

5.2 Different types of Clustering

Hierarchical clustering Vs Partitioning clustering

Exclusive vs Overlapping Vs Fuzzy

Complete vs Partial

Hierarchical clustering Vs Partitioning clustering

Partitioning Clustering are clustering techniques that subdivide the data sets into a set of k groups, where k is the number of groups pre-specified by the analyst.

Hierarchical clustering is an alternative approach to partitioning clustering.

The result of hierarchical clustering is a tree-based representation of the objects

Each node(cluster) in the tree is a union of the children (sub clusters) and the root of the tree is the cluster containing all objects but leaf of the tree is single cluster of individual data objects

Exclusive vs Overlapping Vs Fuzzy

In Exclusive clustering we assign each object to a single cluster.

There are many situations in which a point could reasonably be placed in more than one cluster and these situations are better addressed by Non-Exclusive clustering

In Non-Exclusive clustering or overlapping clustering an object can simultaneously belong to more than one group(class)

Ex: A person at a university can be both an enrolled student and employee of university

Every object belongs to every cluster with membership weight that is between 0 and 1.

0- Absolutely does not belong

1- Absolutely belongs

Complete vs Partial

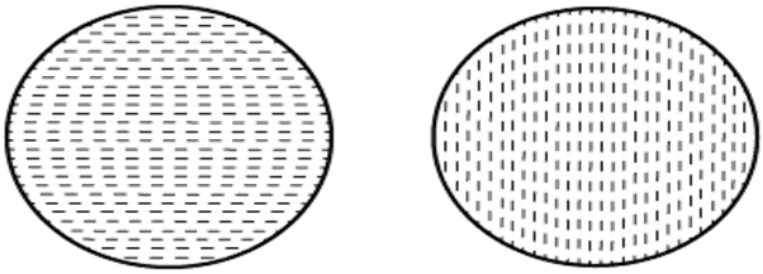
A complete clustering assigns every object to a cluster where as partial clustering does not

Example: Some newspaper stories may have a common theme, while other stories are more generic

5.3 Types of clusters

Well separated cluster

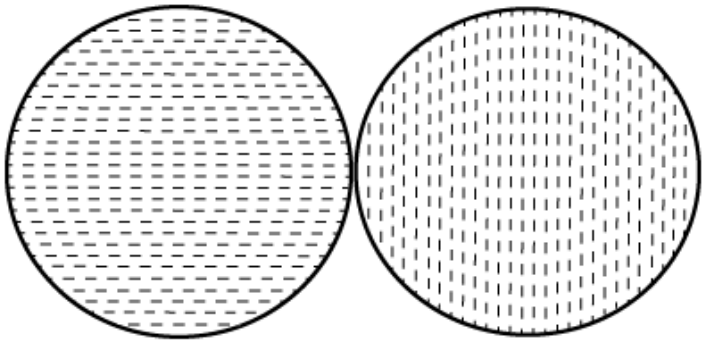
The data object within a cluster must have small distance and distance between two cluster must be high.



Well-separated clusters

Prototype based cluster (centre based cluster)

Each object is closer(similar) to the prototype that define the cluster than to the prototype of any other cluster



Center-based clusters

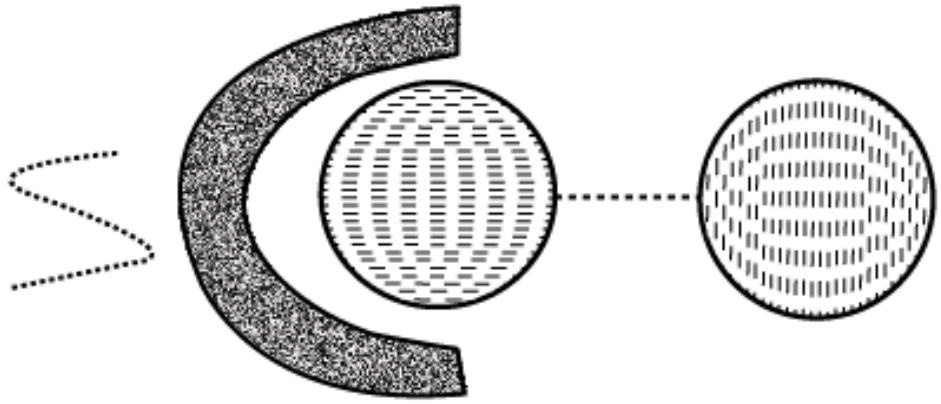
Graph based cluster (contiguity based)

Two objects are connected only if they are within a specified distance of each other.

· Each point in a cluster is closer to at least one point in the same cluster than to any point in a different cluster.

· Useful when clusters are irregular and intertwined.

· This does not work efficiently when there is noise in the data, a small bridge of points can merge two distinct clusters into one.



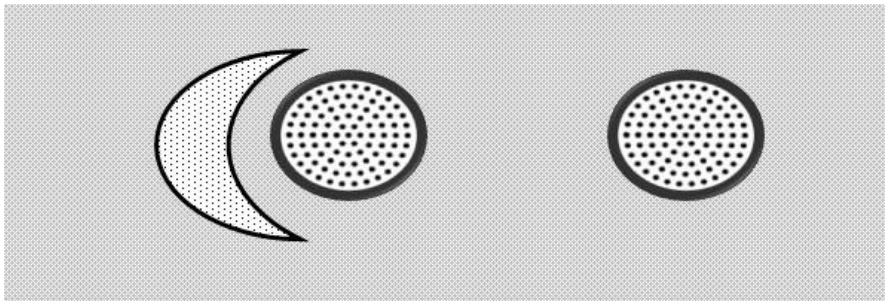
Contiguity-based clusters

Density Based Clusters:

Cluster is a dense region of objects that is surrounded by a region of low density.

Density based clusters are employed when the clusters are irregular, intertwined and when noise and outliers are present.

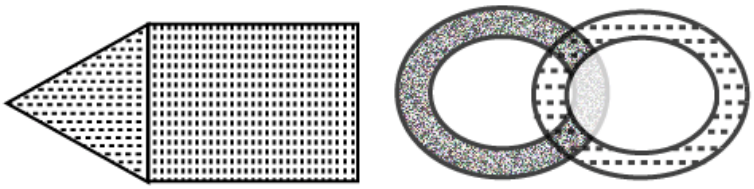
Points in low density region are classified as noise and omitted.



Shared Property cluster (Conceptual cluster)

→More general

→ Set of object that share same property



Conceptual clusters

5.4 The k-means method:

Also called the centroid method since at each step the centroid point of each cluster is assumed to be known

The k-means method uses the Euclidean distance measure

If Instead of Euclidean distance measure, the Manhattan distance is used then the method is called the k-median method

The K-means method is described as follows:

* Select the number of clusters. Let this number be k

* Pick k seed as the centroid of the k-cluster. Seed may be picked randomly

* Compute the distance of each object in the data set from each of the centroids

* Allocate each object to the cluster it is nearest to based on the distances computed in the previous step

* Compute the centroid of the cluster by computing the means of the attribute values of the object in each cluster

* Check if the stopping criteria has been met (ex. If the cluster membership is unchanged). If yes go to step7. If no go to step3.

* Stop

STUDENT	AGE	MARK 1	MARK 2	MARK 3
S1	18	73	75	57
S2	18	79	85	75
S3	23	70	70	52
S4	20	55	55	55
S5	22	85	86	87
S6	19	91	90	89
S7	20	70	65	60
S8	21	53	56	59
S9	19	82	82	60
S10	47	75	76	77

Step 1: let k=3(clusters)

Step 2: let seed be the first 3 students

STUDENT	AGE	MARK 1	MARK 2	MARK 3
S1	18	73	75	57
S2	18	79	85	75
S3	23	70	70	52

1- iteration(step 3 and 4)								
C1	**18**	**73**	**75**	**57**	**Distance from cluster**			**Allocation to the nearest cluster**
C2	**18**	**79**	**85**	**75**	**C1**	**C2**	**C3**	
C3	**23**	**70**	**70**	**52**				
S1	18	73	75	57	0	34	18	C1
S2	18	79	85	75	34	0	52	C2
S3	23	70	70	52	18	52	0	C3
S4	20	55	55	55	42	76	36	C3
S5	22	85	86	87	57	23	67	C2
S6	19	91	90	89	66	32	82	C2
S7	20	70	65	60	18	46	16	C3
S8	21	53	56	59	44	74	40	C3
S9	19	82	82	60	20	22	36	C1
S10	47	75	76	77	52	44	60	C2

Step 5: Compute the centroid by computing the mean of the attribute values of the objects in each cluster

	AGE	MARK 1	MARK 2	MARK 3
C1	18.5	77.5	78.5	58.5
C2	26.5	82.5	84.3	82
C3	21	61.5	61.5	56.5

2- iteration(step 3 and 4)								
C1	18.5	77.5	78.5	58.5	Distance from cluster			Allocation to the nearest cluster
C2	26.5	82.5	84.3	82	C1	C2	C3	
C3	21	61.5	61.5	56.5				
S1	18	73	75	57	10	52.3	28	C1
S2	18	79	85	75	25	19.8	62	C2
S3	23	70	70	52	27	60.3	23	C3
S4	20	55	55	55	51	90.3	16	C3
S5	22	85	86	87	47	13.8	79	C2
S6	19	91	90	89	56	28.8	92	C2
S7	20	70	65	60	24	60.3	16	C3
S8	21	53	56	59	50	86.3	17	C3
S9	19	82	82	60	10	32.3	46	C1
S10	47	75	76	77	52	41.3	74	C2

Note that the clusters have not changed

Cluster 1 → c1→ s1, s9

Cluster 2 → c2→s2, s5, s6, s10

Cluster 3 → c3→s3, s4, s7, s8

Algorithm: *k*-means. The *k*-means algorithm for partitioning, where each cluster's center is represented by the mean value of the objects in the cluster.

Input:

- *k*: the number of clusters,
- *D*: a data set containing *n* objects.

Output: A set of *k* clusters.

Method:

(1) arbitrarily choose *k* objects from *D* as the initial cluster centers;
(2) **repeat**
(3) (re)assign each object to the cluster to which the object is the most similar, based on the mean value of the objects in the cluster;
(4) update the cluster means, that is, calculate the mean value of the objects for each cluster;
(5) **until** no change;

The *k*-means partitioning algorithm.

Strength and weakness of k-mean method:

→ k-mean is simple

→ Quite efficient even though multiple runs are often performed

→ k-mean is not suitable for all types of data

→ k-mean also have trouble in clustering data that contains outliers

→ k-mean is restricted to data for which there is a notion of centre (centroid)

5.5 Agglomerative Hierarchical clustering

There are two basic approaches for generating a hierarchical clustering

Agglomerative

Divisive

Agglomerative:

Start with a point as an individual cluster and at each step, merge the closest pair of clusters

Divisive:

Start with one all-inclusive cluster and at each step split a cluster until only a singleton cluster of individual parts remains

Agglomerative hierarchical clustering Algorithm:

Compute the proximity matrix, if necessary

repeat

merge the closest two clusters

update the proximity matrix to reflect the proximity between the new cluster and the original cluster

until only one cluster remains

Defining proximity between the clusters:

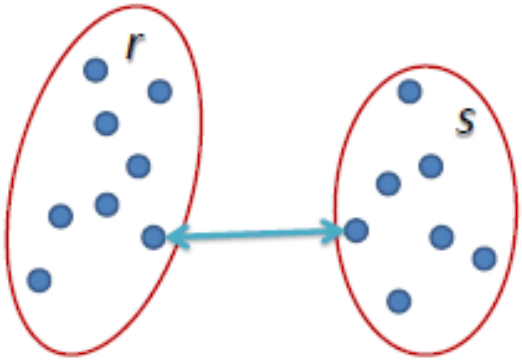
The different proximity measures in agglomerative clustering are:

MIN

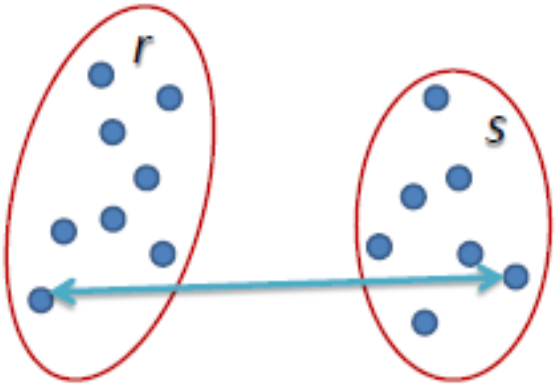
MAX

Group Average

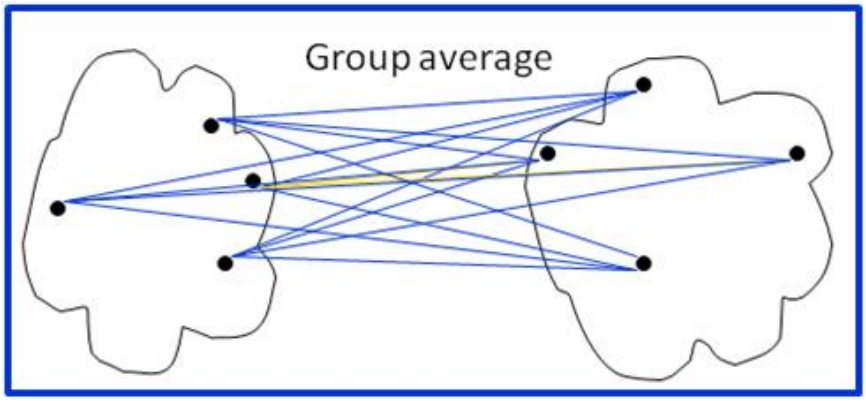
MIN defines the Proximity between the closest two points that are in different clusters



MAX defines the Proximity between the farthest two points that are in different clusters



Group Average defines the cluster proximity to be the average pairwise proximities of all pair of points from different clusters



Strengths and weaknesses

Agglomerative algorithm can produce better quality clusters

Agglomerative algorithm is expensive in terms of their computation and storage requirement

All merge can cause trouble to noise in high dimension data such as document data

5.6 DBSCAN

Density based clustering locates region of high dimensionality that are separated from one another by the region of low density.

DBSCAN is based on centre based approach

In centre based approach, density is estimated for a particular point in a data set by counting the number of points within a specified radius of that point

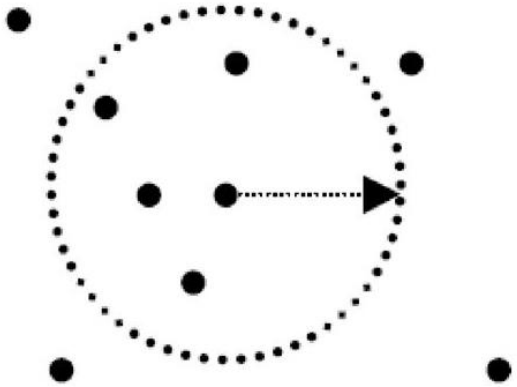


Illustration of center-based density.

Classification of points according to centre based approaches:

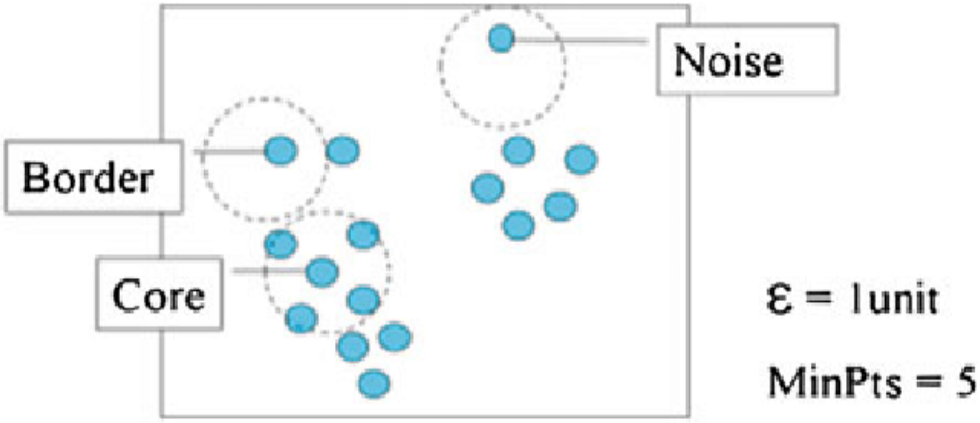
This approach allows us to classify the points into following categories:

- Core point
- Border point
- Noise or Background point

Core points are in the interior points of a density based cluster

Border points is not a core point but falls within the neighbourhood of the core point

Noise point is any point that is neither a core point nor a border point



DBSCAN Algorithm

* Label all the points as core, border and noise

* Eliminate noise points

* Put an edge between all the core points that are within Eps

* Make each group of connected core points to separate cluster

* Assign each border point to one of the cluster of its Associated core points

Strength and weakness

It is resistant to noise

Can handle cluster of arbitrary shapes and sizes

It has trouble with high dimensional data because density is more difficult to define for such data

DBSCAN can be expensive when the computation of nearest neighbor requires computing all pairwise proximities

Density based clustering

DBSCAN, a simple but effective algorithm for finding density based cluster.

Dense regions of objects that are surrounded by low density regions

Grid based cluster

Breaks the data space into grid cells and then forms cluster from cells that are sufficiently dense

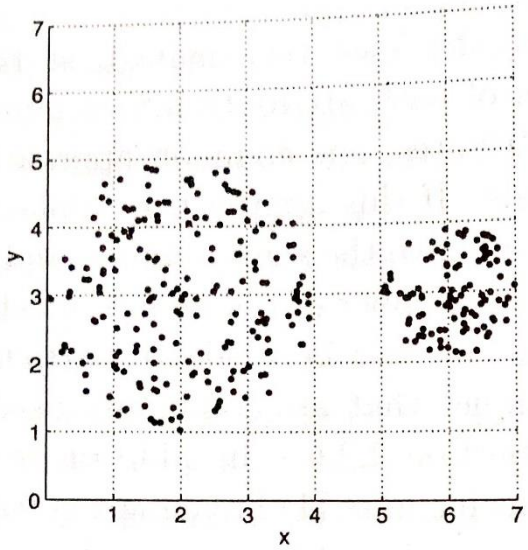


Figure 9.10. Grid-based density.

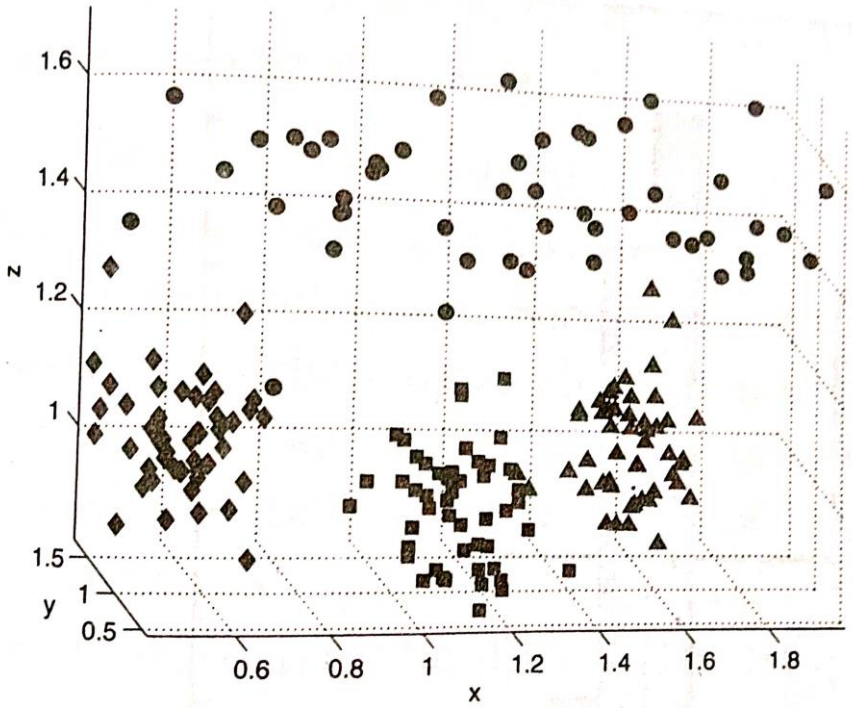
0	0	0	0	0	0	0
0	0	0	0	0	0	0
4	17	18	6	0	0	0
14	14	13	13	0	18	27
11	18	10	21	0	24	31
3	20	14	4	0	0	0
0	0	0	0	0	0	0

Table 9.2. Point counts for grid cells.

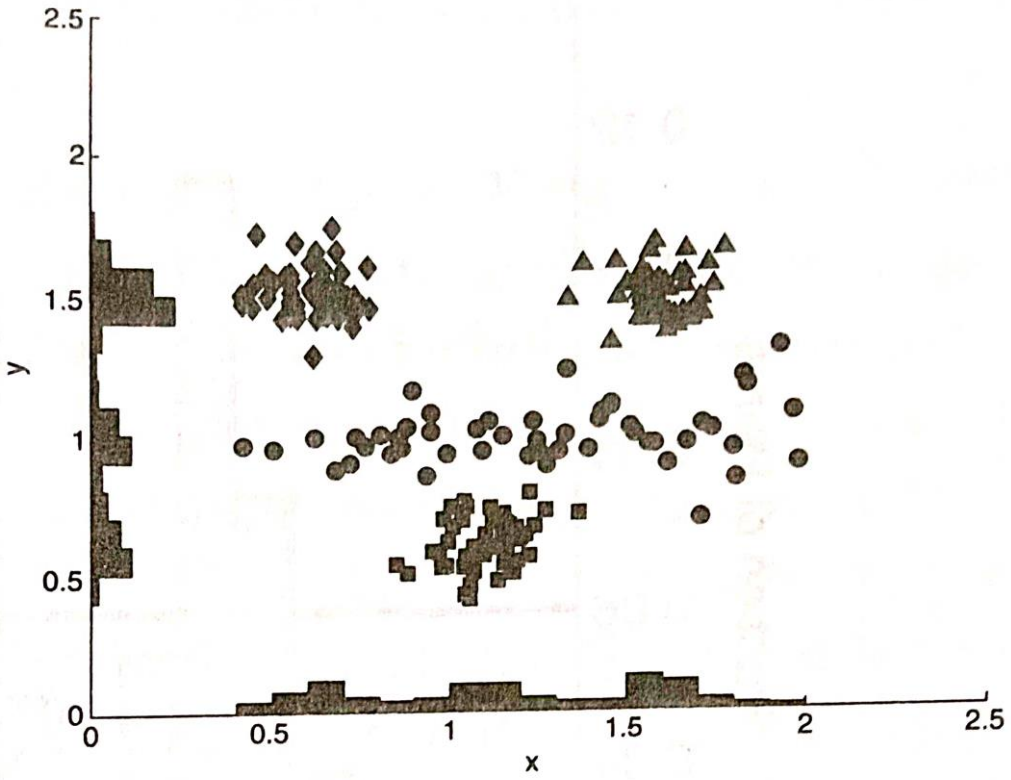
Subspace clustering

The clustering techniques considered until now found clusters by using all of the attributes

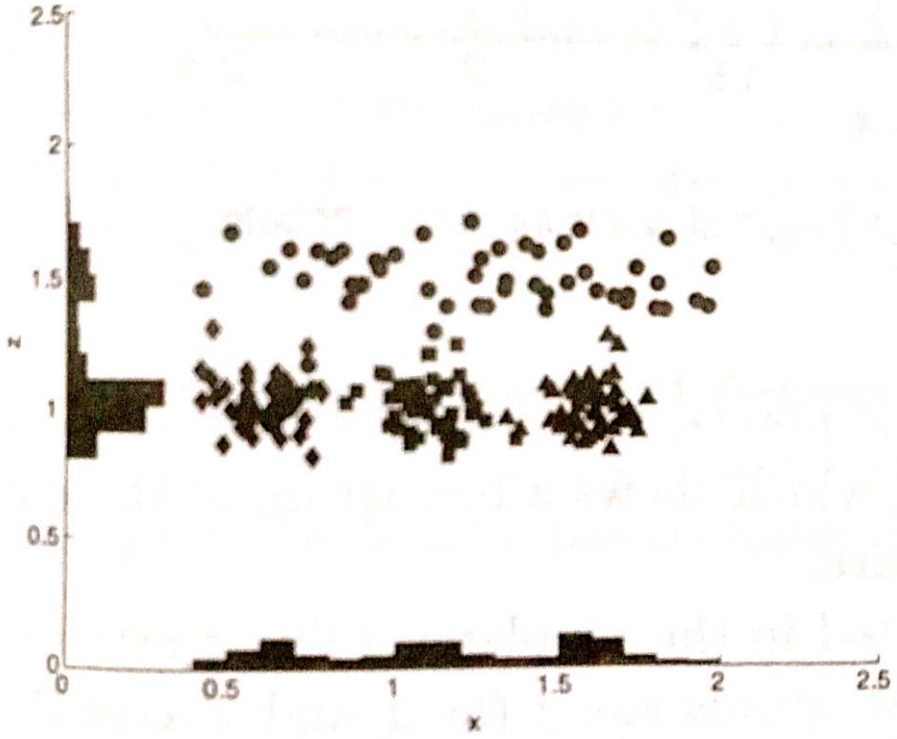
If only subset of the attributes are considered then the clusters that we find can be quite different



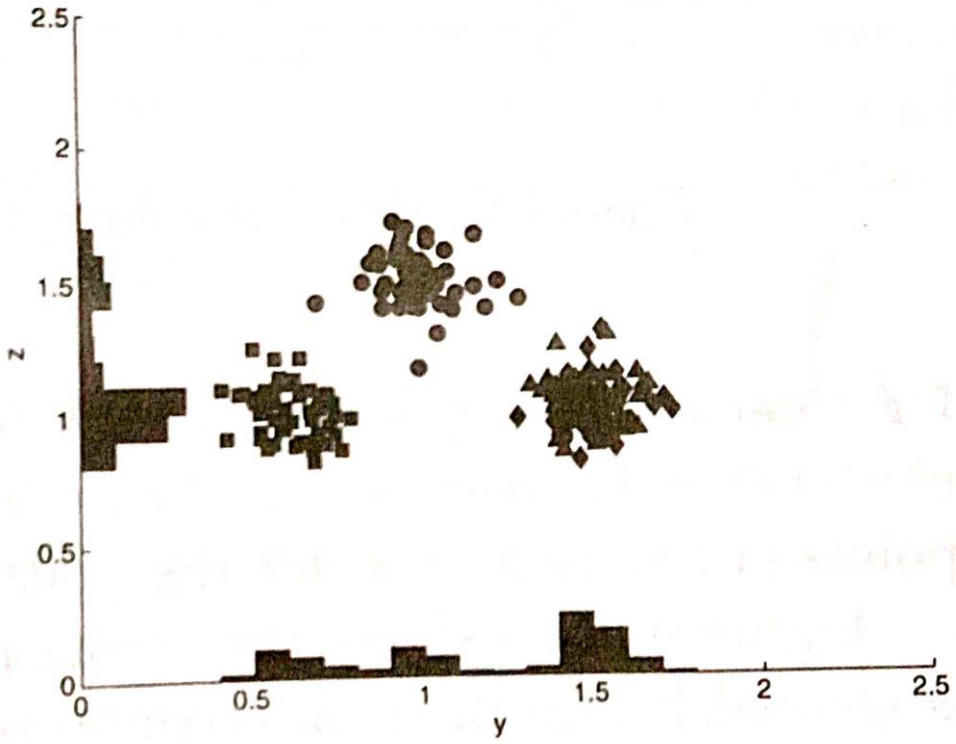
(a) Four clusters in three dimensions.



(b) View in the xy plane.



(c) View in the xy plane.



(d) View in the xy plane.

5.7 Cluster Evaluation

Cluster Evaluation should be a part of any of the Cluster Analysis

Cluster evaluation/Validation are traditionally classified into following types:

Unsupervised cluster Evaluation

Supervised cluster Evaluation

Unsupervised cluster Evaluation

Measure the goodness of the clustering structure without considering external information

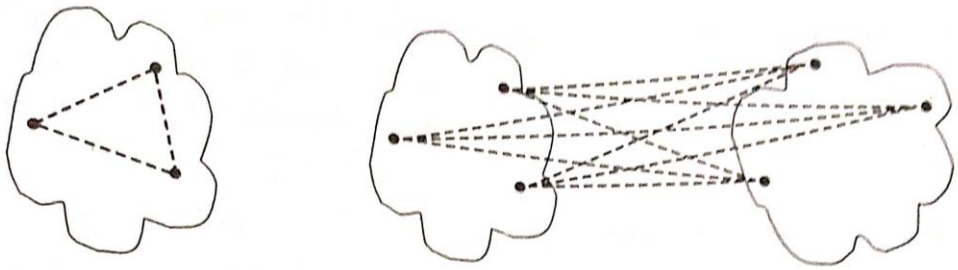
Unsupervised measure is further divided into two classes:

Measure of cluster cohesion

Measure of cluster separation

Cluster cohesion determines how closely related to objects in a cluster

Cluster separation determines how distinct or well separated a cluster is from other clusters



(a) Cohesion. (b) Separation.

Figure 8.27. Graph-based view of cluster cohesion and separation.

$$cohesion(C_i) = \sum_{\substack{\mathbf{x} \in C_i \\ \mathbf{y} \in C_i}} proximity(\mathbf{x}, \mathbf{y})$$

$$separation(C_i, C_j) = \sum_{\substack{\mathbf{x} \in C_i \\ \mathbf{y} \in C_j}} proximity(\mathbf{x}, \mathbf{y})$$

Supervised cluster Evaluation

This approach measures the extent to which two objects that are in the same class are in the same cluster and vice versa

There are two types of supervised measures:

Classification oriented

Similarity oriented

Classification oriented

No. of measures such as “Entropy”, “Purity”, “Precision”, “Recall”, “F measure” are commonly used to evaluate the performance

Entropy:

The degree to which each cluster consists of objects of a single class

$$e = \sum_{i=1}^{K} \frac{m_i}{m} e_i,$$

where K is the number of clusters

m is the total number of data points

$$e_i = -\sum_{j=1}^{L} p_{ij} \log_2 p_{ij},$$

pij=mij/mi

where mi is the number of objects in cluster i

mij is the no of objects of class j in cluster i

L is the number of classes

Purity:

$$purity = \sum_{i=1}^{K} \frac{m_i}{m} p_i.$$

where pi= max pij

Precision:

The fraction of a cluster that consists of objects of a specified class

precision(i,j)=pij

Recall

The extent to which a cluster contains all the classes

F measure:

combination of both precision and recall

Extent to which a cluster contains only objects of a particular class and all the objects of that class

Similarity oriented

Cluster evaluation will be done using comparison of two matrices

Ideal cluster similarity matrix

Ideal class similarity matrix

Ideal cluster similarity matrix

Has 1 in ijth entry if two objects i and j are in the same cluster, and 0 otherwise

Ideal class similarity matrix

Has 1 in ijth entry if two objects i and j belong to the same class, and 0 otherwise

Table 8.10. Ideal cluster similarity matrix.

Point	p1	p2	p3	p4	p5
p1	1	1	1	0	0
p2	1	1	1	0	0
p3	1	1	1	0	0
p4	0	0	0	1	1
p5	0	0	0	1	1

Table 8.11. Ideal class similarity matrix.

Point	p1	p2	p3	p4	p5
p1	1	1	0	0	0
p2	1	1	0	0	0
p3	0	0	1	1	1
p4	0	0	1	1	1
p5	0	0	1	1	1